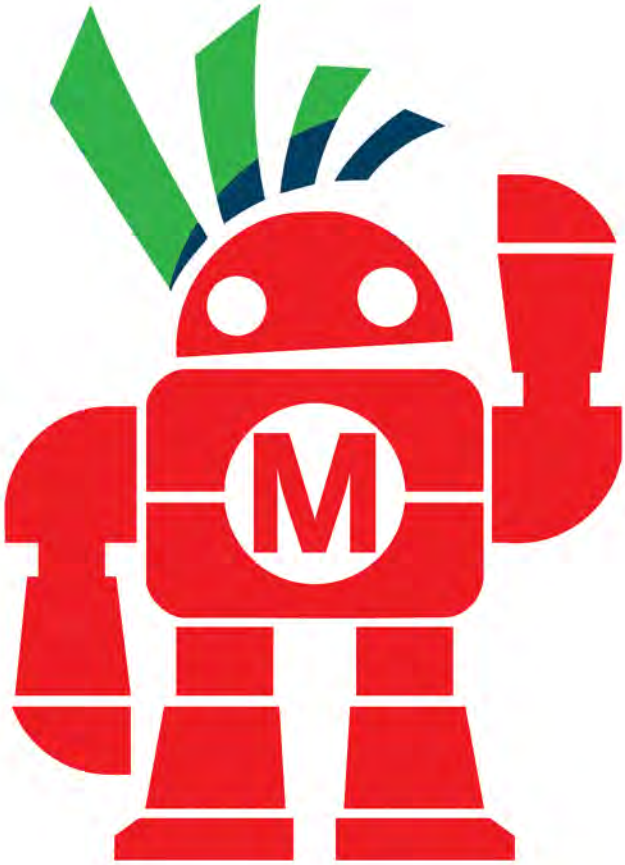
Makey:Lab

Playbook 1



Make:

Umwelt-Campus Birkenfeld

HOCHSCHULE TRIER

Makey:Lab – Playbook 1

Version 1.2026 | 1. Auflage

Erstellt vom Make Magazin in Kooperation mit dem Umwelt-Campus Birkenfeld der Hochschule Trier.

Autor: Alexander Panknin, Make Magazin
Redaktionelle Leitung: Daniel Bachfeld, Make Magazin
Layout und Satz: Alexander Panknin, Make Magazin
Bilder: Alexander Panknin, Make Magazin
Software: Prof. Dr. Klaus-Uwe Gollmer, IoT²-Werkstatt
Hardware-Design: Guido Burger, Fab-Lab.eu

© 2026 Maker Media GmbH. Alle Rechte vorbehalten.

Impressum:

Maker Media GmbH
Postfach 61 04 07, 30604 Hannover
Karl-Wiechert-Allee 10, 30625 Hannover
Telefon: 05 11 / 53 52-0
Internet: www.make-magazin.de

Inhaltsverzeichnis

Teil 1 - Einführung

Dein Technik-Versuchslabor 1
Die Idee hinter Makey:Lab 4
Ein Werkzeug für alles 8
Was ist alles dabei? 10
Makey:Lab einrichten 11
Die IoT-Werkstatt verstehen 15
Dein erstes Programm 19
Deine Welt steckt voller Projekte 23
Spickzettel: Orte für Projekte 26
So denkt ein Programm 29

Teil 2 – Projekte mit Makey:Lab

#01 Bau dir einen eigenen Wetterfrosch 35
#02 Frische Luft fürs Hirn 77
#03 Makey:Lab schnuppert am Obstkorb 91
#04 Die Duschpetze 119
#05 Die Lärmampel 141
#06 Zimmer-Alarmanlage 153
#07 Tageslicht-Wecker 165
#08 Makey:Lab sieht Farbe 195
#09 Makey:Lab nutzt WLAN 205
#10 Makey:Lab schreibt dir übers Internet! 213

Teil 3 – Anhang

Nachwort	229
Betriebsanleitung	231

schau genau hin,
vielleicht gibt es
noch mehr zu entdecken :)

117
151

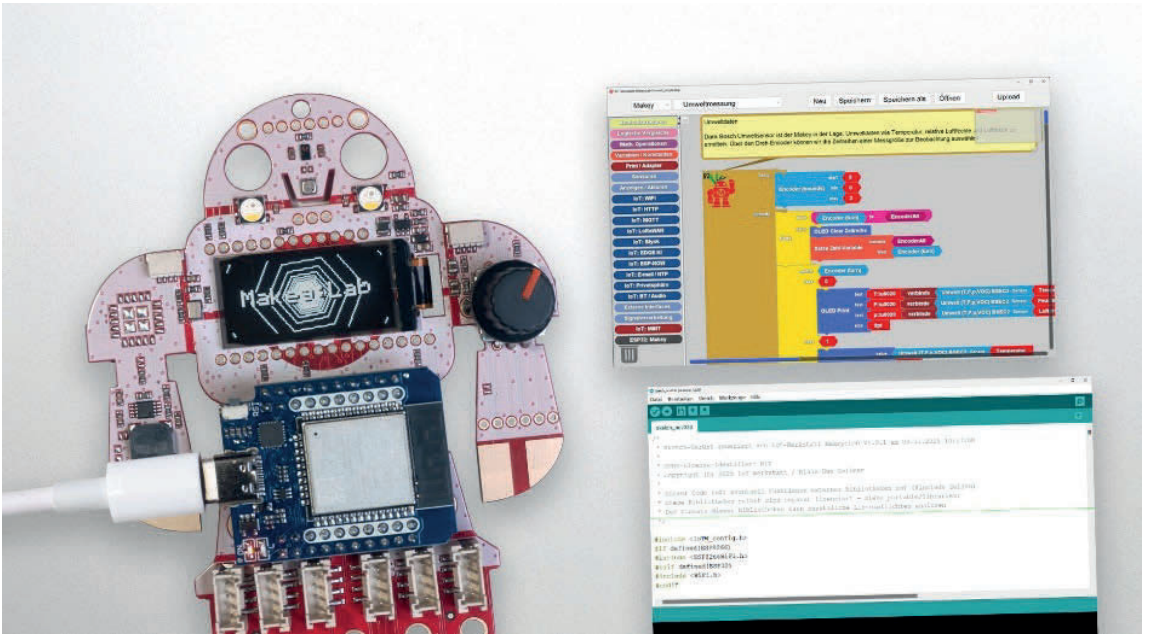
Hinweis

Makey:Lab wird kontinuierlich weiterentwickelt.
Die in diesem Playbook beschriebenen Projekte und Anleitungen wurden sorgfältig getestet und entsprechen dem Stand der aktuell ausgelieferten Hard- und Software.
Da sich Makey:Lab durch laufende Verbesserungen weiterentwickelt, können sich einzelne Details – etwa Blocknamen, Menüpunkte oder Funktionsbezeichnungen – mit zukünftigen Updates geringfügig ändern.

Dieses Playbook enthält zugleich die **Betriebs- und Sicherheitshinweise** zum Makey:Lab.
Die entsprechenden technischen Informationen, rechtlichen Hinweise und Sicherheitsvorgaben finden sich im Anhang.

Alle aktuellen Informationen, Software-Updates, ergänzenden Dokumentationen und Hinweise zu neuen Funktionen sind zu finden unter:
https://www.makeylab.de

Rückmeldungen zu Inhalten, Ideen oder neuen Projekten sind jederzeit willkommen.
Anregungen aus der Community helfen dabei, Makey:Lab kontinuierlich zu verbessern.



Makey:Lab – dein Technik-Versuchslabor

Entdecke, was möglich ist

Die Welt um dich herum ist voll von Technik: Smartphones, Sprachassistenten, Tablets und Computer – alles ist vernetzt, misst Daten oder reagiert auf Befehle. Aber wie genau funktioniert das? Und wer entscheidet eigentlich, was diese Technik macht?

Mit dem Makey:Lab bestimmst du das.
Hier verstehst du, wie Technik funktioniert – und wie du sie nach deinen Ideen gestalten kannst.
Makey:Lab ist deine Plattform, um auszuprobieren, zu verändern und zu zeigen, was möglich ist.

Ganz egal, ob du schon Erfahrung hast oder nicht – das spielt keine Rolle. Du brauchst kein Vorwissen, keine

Programmierkenntnisse. Nur Neugier und Lust, die Dinge selbst in die Hand zu nehmen.

Mit dem Makey:Lab kannst du dir in kurzer Zeit eigene Geräte bauen: von der Frischluft-Ampel fürs Klassenzimmer bis zur Duschpetze, die piept, wenn mal wieder jemand zu lange im Bad ist – oder ein Zimmer-Alarm, der sofort anschlägt, wenn jemand deine Tür öffnet.

Ob witzig oder nützlich: Jedes Projekt zeigt, was Technik kann – Dinge erkennen, reagieren und im richtigen Moment handeln.

Das Beste: Du musst dafür keinen komplizierten Code schreiben. Mit der Blockprogrammierung baust du deine eigenen Programme wie ein Puzzle – Schritt für Schritt. Und das, was du auf dem Bildschirm zusammensteckst, passiert direkt in der echten Welt.

In diesem Playbook zeigen wir dir, wie du startest. Du kannst alles in deinem eigenen Tempo entdecken – ohne Stress, ohne Noten, einfach loslegen. Was du daraus machst, ist dein Ding.

Experimentiere, mach Fehler, versteh, wie’s funktioniert – und dann baue, was du willst.

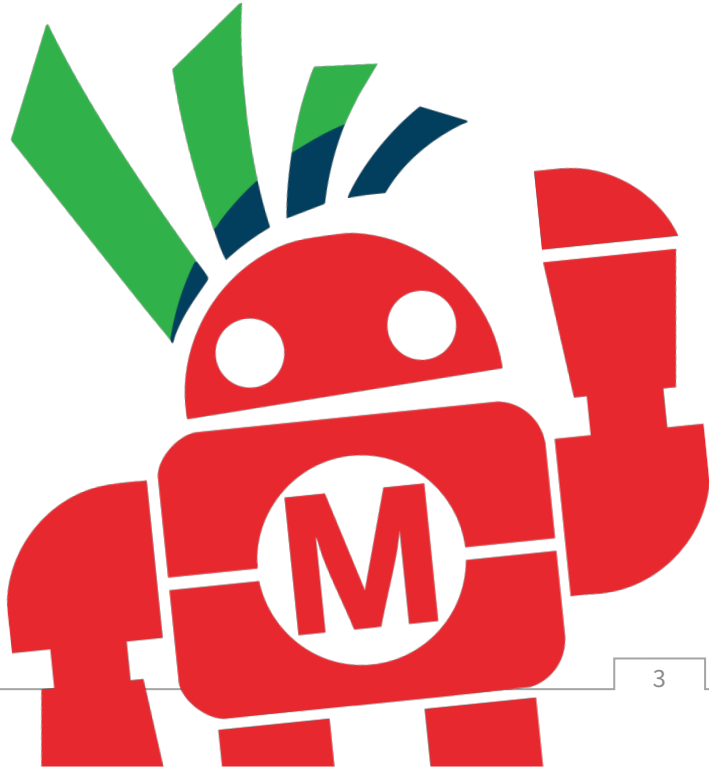
Und du bist nicht allein: Makey:Lab ist auch eine Community – ein Ort für technikbegeisterte Menschen, die ihre Ideen teilen und voneinander lernen.

Denn: Da draußen gibt es viele, die genauso gern tüfteln und ausprobieren wie du.

Übrigens: Es lohnt sich, früh damit anzufangen, sich mit Technik zu befassen. Das kann dir später sogar einen echten Vorteil verschaffen. Du lernst, Probleme zu erkennen, eigene Lösungen zu finden – und Technik als Werkzeug zu begreifen, statt sie nur zu benutzen.

Makey:Lab ist bereit für die Zukunft. Es ist bestens ausgestattet, damit du moderne Projekte auf dem neuesten Stand der Technologie umsetzen kannst.

Willkommen im Makey:Lab – und viel Spaß beim Entdecken!



Die Idee hinter Makey:Lab

Lernen durch Machen

Makey:Lab ist eine gemeinsame Initiative des Umwelt-Campus Birkenfeld der Hochschule Trier und des Make Magazins. Gemeinsam verfolgen sie das Ziel, junge Menschen zu ermutigen, Technik nicht nur zu benutzen, sondern sie zu verstehen, zu hinterfragen und kreativ zu gestalten.

Am Umwelt-Campus entstehen seit Jahren viele Projekte, die digitale Technologien mit Innovation und Nachhaltigkeit verbinden – etwa der Blaue Engel für Software, KI-gestützte Prognosetools für erneuerbare Energien oder eine künstliche Nase zum Erkennen von Weinsorten.

Die Initiative zur IoT^2-Werkstatt (Internet of Things and Thinking – eine vom Umwelt-Campus entwickelte Erweiterung des klassischen IoT-Konzepts) und ihrer Anwendung z. B. bei CO_2-Ampeln, Starkregenpegel oder Solartischen geht auf die Arbeit von Prof. Dr. Klaus-Uwe Gollmer zurück, der seit Jahren am Umwelt-Campus Birkenfeld Wege erforscht, digitale Bildung, technische Informatik und Nachhaltigkeit praktisch zu verbinden.

Seit seiner frühen Jugend als Maker unterwegs, entwickelte er in seiner Freizeit eine grafische Programmierplattform, die auch komplexe Szenarien

typischer IoT²-Aufgaben spielerisch einfach abbildet (LoRaWAN, MQTT, NTP, Edge-KI). Mit dem *IoT-Octopus* – einem Lernsystem für Sensorik und IoT, das gemeinsam mit dem Entwickler Guido Burger entstand – legten die beiden Maker den Grundstein für das heutige Makey:Lab.

Zusammen mit der Do-it-yourself-Erfahrung von Make – also der Freude am Selbermachen – entstand daraus nun das Makey:Lab – eine offene Lern- und Entdeckerplattform, die zeigt, wie moderne Technik begreifbar und gestaltbar wird.

Makey:Lab führt Schritt für Schritt in die Welt des IoT² – also in Systeme, die Daten erfassen, austauschen und intelligent auf ihre Umgebung reagieren.

Hier wird ausprobiert, programmiert, gemessen, verbessert: Lernen passiert durch Machen. Wer Makey:Lab nutzt, entdeckt, dass Technik keine Blackbox ist, sondern etwas, das sich verstehen, verändern und für eigene Ideen einsetzen lässt.

Damit stärkt Makey:Lab Fähigkeiten, die in Zeiten quasi allwissender, generativer KI heute wichtiger sind denn je, etwa Selbstständigkeit, Innovationsfreude, Problemlösekompetenz und digitales Verständnis.

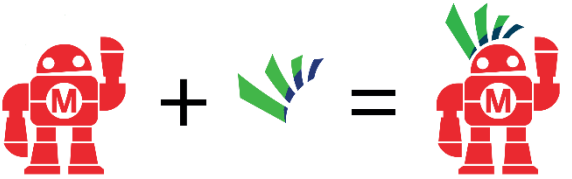
Gleichzeitig vermittelt es spielerisch MINT-Wissen (Mathematik, Informatik, Naturwissenschaften, Technik) und macht deutlich, wie eng Technik, Umwelt und Zukunft miteinander verbunden sind.

Ob im Unterricht, in Makerspaces oder einfach zu Hause – Makey:Lab funktioniert ohne Vorwissen und lädt zum gemeinsamen Entdecken ein. Lehrkräfte und lernbegleitende Erwachsene finden darin eine praxiserprobte Grundlage für moderne Lernformen – junge Menschen ein Werkzeug, um eigene Ideen Wirklichkeit werden zu lassen.

Mit Makey:Lab verbindet sich die kreative Welt der Maker mit der Innovationskraft der Hochschulen. Diese einmalige Partnerschaft eröffnet neue durchgängige Lernwege – vom ersten Experiment am Schultisch über Ausbildung und Studium bis hinein in die Forschung.

Makey:Lab ist die Brücke zwischen Machen und Denken, zwischen Werkstatt und Wissenschaft – ein Ort, an dem Ideen wachsen, Wissen entsteht und Zukunft gestaltet wird.

Das erkennt man übrigens auch an der Makey:Lab-Platine. Sie orientiert sich in ihrer Form am roten Makey-Roboter aus dem Make Magazin.

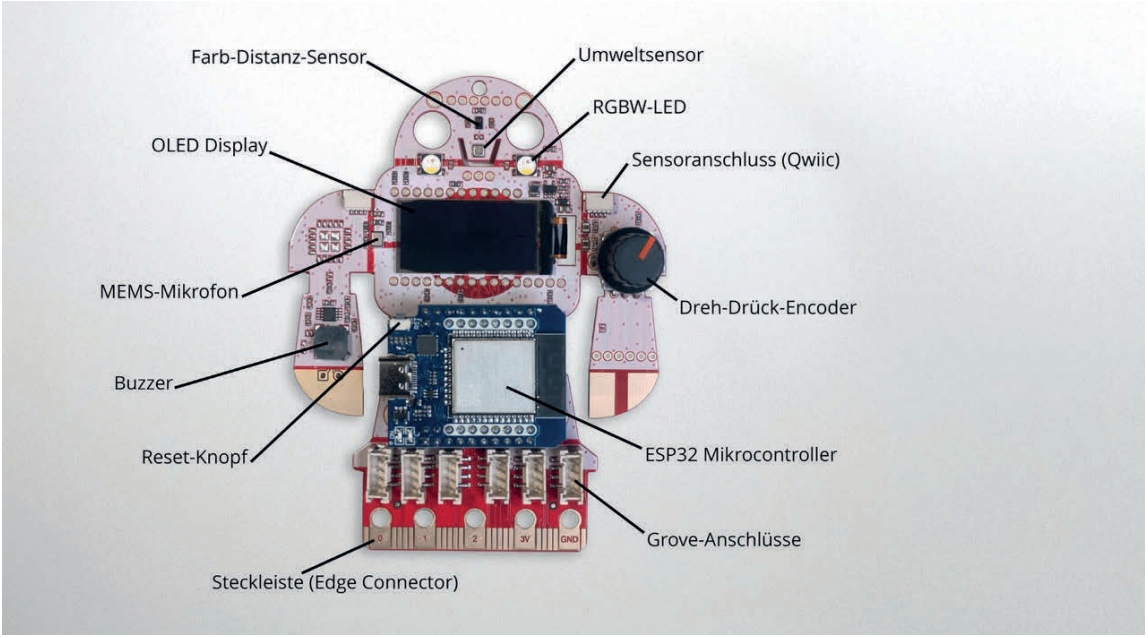


Im Logo trägt er das stilisierte grün-blaue Symbol des Umwelt-Campus Birkenfeld – ein Zeichen für Nachhaltigkeit, Technologie und die gemeinsame Entwicklung des Projekts.

Mehr über Makey:Lab am Umwelt-Campus

SCAN MICH

Hier findest du die Makey:Lab-Seite des Umwelt-Campus Birkenfeld. Dort erfährst du, warum *Making* und *Internet of Things (IoT)* als Schlüsselkompetenzen der Zukunft gelten – für Innovation, Bildung, Energie, Klimafolgen und Nachhaltigkeit. Die Seite führt dich zu Studienangeboten, zur IoT-Werkstatt und zeigt, wie deine eigenen Projekte eventuell sogar im Studium anerkannt werden können.



Makey:Lab – Ein Werkzeug für alles

Ein ganzes Labor – auf einer einzigen Platine

Makey:Lab ist dein eigenes kleines Techniklabor. Ein Ort, an dem du experimentieren, ausprobieren und sogar eigene Geräte entwickeln kannst – ganz ohne Vorwissen.

Damit Technik die Welt begreift, braucht sie Sensoren. Und davon hat das Makey:Lab jede Menge – direkt auf der Platine. Temperatur, Luftdruck, Stoffe in der Luft (sogenannte VOCs), Licht, Geräusche – alles lässt sich messen.

Damit es noch mehr Spaß macht, haben wir dem Makey:Lab besonders hochwertige Sensoren verpasst. Die sind nicht nur Spielzeug, sie werden auch in professionellen Geräten und der Industrie eingesetzt.

Das Herzstück ist der Mikrocontroller – ein kleiner, programmierbarer Computer. Wenn du Makey:Lab gerade vor dir hast: das ist das Bauteil mit der kleinen silbernen Aluabdeckung direkt unter dem Display. Er steuert alle Komponenten – und macht genau, was du ihm sagst.

Heutzutage ist fast alles vernetzt – Stichwort "IoT", also "Internet of Things". Unser Mikrocontroller hat WLAN und Bluetooth an Bord und kann so mit anderen Geräten (zum Beispiel mit deinem Tablet oder Smartphone) und sogar mit dem Internet kommunizieren. Das eröffnet dir viele Möglichkeiten: Daten sammeln, vergleichen, teilen oder darauf reagieren.

Mit dem Makey:Lab kannst du also direkt loslegen: Miss deine Umgebung, schalte Leuchtdioden (LEDs), gib Geräusche aus, reagiere auf Bewegung oder Luftqualität – alles ist möglich.

Und das ist nur der Anfang. Später wirst du eigene Projekte entwickeln, die ein Problem erkennen – und eine Lösung anbieten. Genau wie echte Geräte: Sie beobachten (über Sensoren), denken (mit dem Mikrocontroller) und handeln (über LEDs, Töne, Anzeigen auf dem Display). Du entscheidest dabei allerdings, was deine Technik tun soll.

Was ist alles dabei?

Dein Makey:Lab besteht aus einer einzigen Platine – aber die hat es in sich: Darauf sind schon alle wichtigen Komponenten für deine ersten Projekte. Du musst nichts löten, nichts zusammensetzen. Einfach starten.

Das findest du auf der Platine:

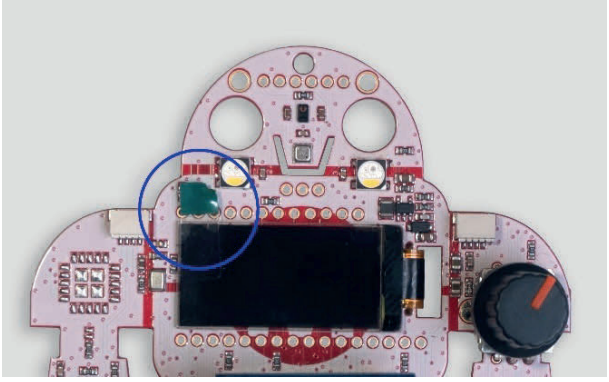
- Einen kleinen Bildschirm (OLED), der dir Infos anzeigen kann
- Zwei farbige LEDs (RGBW), die blinken, leuchten oder etwas anzeigen können
- Einen Dreh-Drück-Encoder – damit kannst du Inhalt scrollen, auswählen, steuern
- Einen Umweltsensor (BME680), der Temperatur, Luftfeuchtigkeit, Luftdruck und Luftqualität misst
- Einen Farbsensor (APDS-9999), der auch Helligkeit und sogar Entfernung misst
- Ein Mikrofon (MEMS), das Geräusche erkennt
- Einen Buzzer – für Töne, Signale, Sounds
- Mehrere Anschlüsse – damit du später noch weitere Sensoren anschließen kannst
- Und einen Mikrocontroller (ESP32), der alles steuert

Du musst nur das mitgelieferte USB-C-Kabel anschließen – darüber bekommt das Makey:Lab Strom und kann mit einem Computer oder Laptop programmiert werden.

Wenn später ein fertiges Programm aufgespielt ist, kannst du das Makey:Lab auch ganz ohne Computer betreiben:

Einfach eine Powerbank anschließen – und es läuft eigenständig weiter. Wenn du dein Programm neu starten willst, dann kannst du einfach auf den Reset-Knopf drücken. Makey:Lab merkt sich das Programm, bis du ein neues überträgst.

Hinweis: Wenn du dein Makey:Lab frisch aus der Verpackung nimmst, befindet sich auf dem Display noch eine Schutzfolie – erkennbar am kleinen grünen Fähnchen. Du kannst selbst entscheiden, ob du sie zunächst aufgeklebt lässt oder entfernst; die Folie beeinträchtigt den Betrieb nicht.



Wenn du magst, kannst du am Fähnchen ziehen, um die Schutzfolie zu entfernen.

Makey:Lab einrichten

Bevor es losgeht, musst du dein Makey:Lab einmal richtig vorbereiten. Keine Sorge – das ist kein Hexenwerk, du brauchst nur ein bisschen Geduld und einen Laptop oder

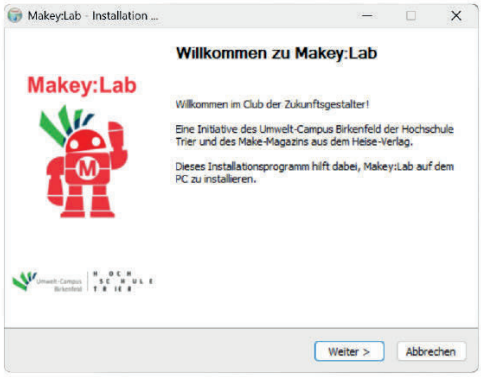
PC. Ein Tablet reicht hier noch nicht (vielleicht später mal).

Schritt für Schritt: Zuerst brauchen wir unsere Makey:Lab-Software. Sie nennt sich **IoT-Werkstatt** und du kannst sie dir auf der Makey:Lab-Webseite herunterladen. Alle wichtigen Informationen dazu findest du unter dem QR-Code. Einfach scannen und loslegen!

Scan mich!

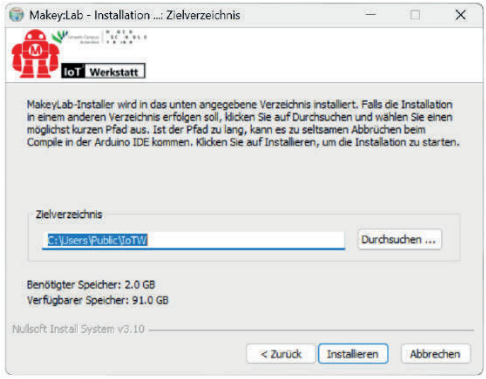
Makey:Lab wurde zunächst für Windows-Rechner entwickelt. Deshalb zeigen wir hier die Einrichtung unter Windows. Es gibt aber auch eine Linux-Version und du kannst sogar einen Raspberry Pi nutzen.

Über den Download erhältst du das Installationsprogramm (Win_*MakeyLab-InstallerV**xxx**.exe* – dabei stehen die **x** für die jeweilige Versionsnummer). Das ist eine ausführbare Datei, die du per Doppelklick startest. Standardmäßig wird die Software im Ordner *C:\Benutzer\Öffentlich\IoTW* (beziehungsweise *C:\Users\Public\IoTW*) installiert und benötigt ca. 2 GB Speicherplatz.



Das ist das Installationsprogramm.

Wenn du einen anderen Speicherort wählst, achte darauf, dass der Pfad nicht zu lang ist, damit es später keine Probleme beim Betrieb des Programms gibt.



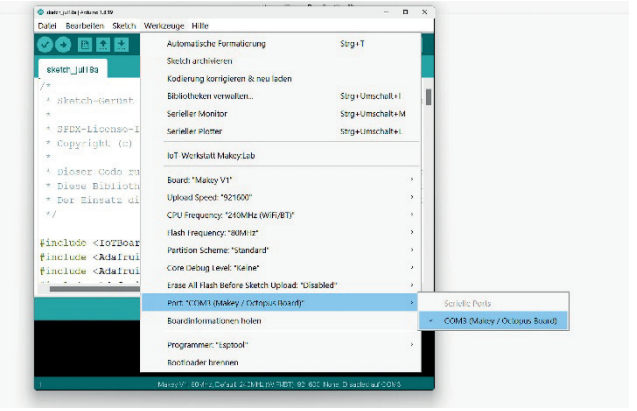
Hier kannst du den Installationspfad wählen.

Hinweis: Sollte es bei der Installation der Software zu Problemen kommen, dann kann es helfen, das Setup als Administrator auszuführen. Dafür einfach mit rechter Maustaste den Installer anklicken und "Als Administrator ausführen" anwählen.

Schließe jetzt dein Makey:Lab am USB-Port deines Rechners an. Achte darauf, ein Kabel zu verwenden, das auch eine Datenleitung hat – einige Kabel liefern nur Strom und eignen sich nicht für die Verbindung (der Unterschied ist äußerlich nicht sichtbar). Am besten nutzt du das mitgelieferte Kabel, da es garantiert funktioniert.

Du findest eine Verknüpfung mit dem Namen *MakeyLab* auf deinem Desktop. Starte die Software aber erst, wenn du alles richtig verkabelt hast.

Es öffnet sich nun die sogenannte **Arduino IDE**. Das ist das Programm, welches die Kommunikation zwischen unserer IoT-Werkstatt und dem Makey:Lab ermöglicht. Hier musst du nur einstellen, wo das Makey:Lab angeschlossen ist. Dies geht unter *Werkzeug > Port*. Manchmal sind mehrere Ports aufgelistet, am Namen erkennst du allerdings dein **Makey:Lab**.



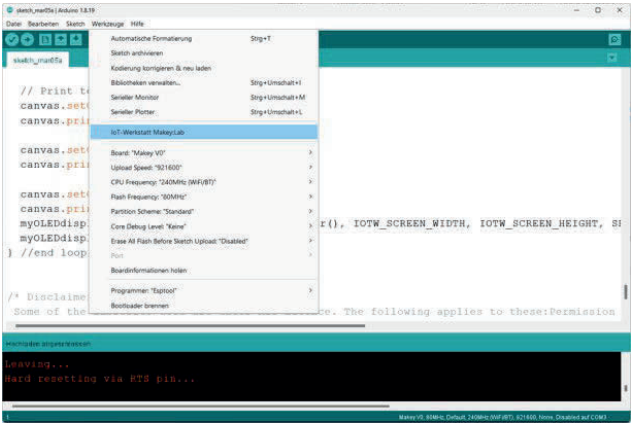
Hier findest du den richtigen Port.

Alternativ kannst du die Software übrigens auch direkt aus dem Installationsordner (*C:\Benutzer\Öffentlich\IoTW*) starten – *MakeyLab* anklicken.

Die IoT-Werkstatt verstehen

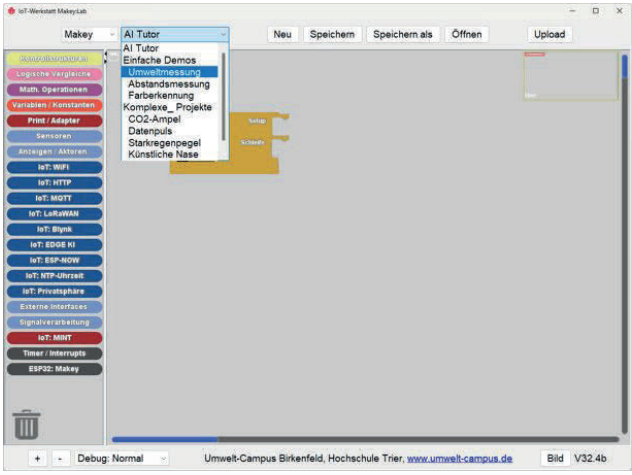
Im Herzen des Makey:Lab steckt ein ESP32-Mikrocontroller. Viele Maker nutzen die Arduino IDE, mit der man den ESP32 in der Programmiersprache C++ programmieren kann.

Das kannst du später auch machen, aber für den Anfang nutzen wir dafür unsere Programmier-Blöcke. Die sind viel einfacher – und machen den Einstieg besonders leicht.



Hier geht es zur Blockprogrammierung, der Iot-Werkstatt.

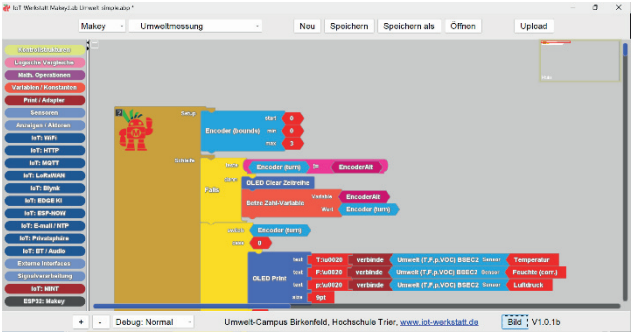
Die Oberfläche zur Blockprogrammierung findest du hier: Klicke im Arduino-Fenster einfach auf *Werkzeuge* und dann *IoT Werkstatt Makey:Lab*. Es öffnet sich ein weiteres Fenster. Dort erkennst du im Arbeitsbereich in der Mitte schon den ersten Startblock. Hier legen wir später unser Programm an. Eine Auswahl für weitere Blöcke findest du auf der linken Seite unter den einzelnen Reitern. Diese kann man ganz einfach in den Arbeitsbereich ziehen und dort mit anderen Blöcken verbinden. Du kannst dir das wie einen Werkzeugkasten vorstellen, in dem alles liegt, was du brauchst.



Hier kannst du ein paar Demo-Programme ausprobieren.

Damit du einen Eindruck bekommst, wie so ein fertiges Programm aussieht, haben wir eine Reihe von Beispielen in der IoT-Werkstatt hinterlegt. Am oberen Rand des Fensters findest du zwei Auswahlmenüs. Im linken kannst du auswählen, welche Blöcke angezeigt werden sollen. Hier wählen wir natürlich *Makey* aus. Daneben gibt es ein Menü mit mehreren Beispielprogrammen. Wähle hier einmal *Umweltmessung* aus.

Nun siehst du ein bereits fertiges Programm, welches aus vielen Blöcken besteht. Wenn nicht alles auf den Bildschirm passt, kannst du dich mit dem Mausrad oder per gedrückter Maustaste durch die Blöcke bewegen. Klick einfach auf einen freien Bereich und zieh die Maus, um dich umzusehen.



Nicht erschrecken, sowas baust du später mit links!

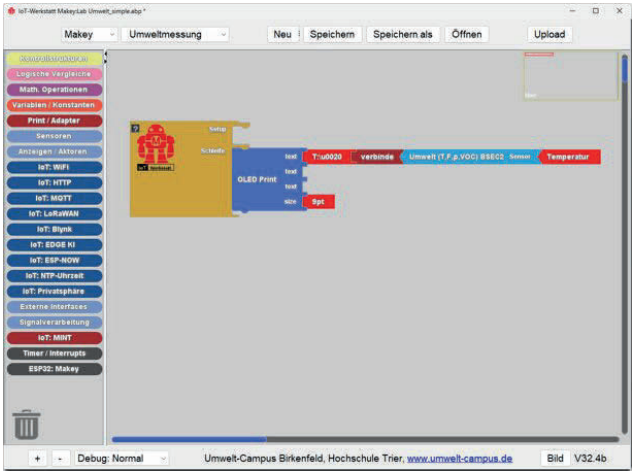
Das Programm liest den Umweltsensor aus und zeigt Temperatur, Feuchtigkeit oder Luftdruck auf dem Bildschirm des Makey:Lab an. Du kannst mit dem Encoder (Drehregler) auswählen, was du sehen möchtest. Um das Programm, auch Sketch genannt, auf dein Makey:Lab zu übertragen musst du einfach oben rechts auf *Upload* klicken. Nun wird der Code automatisch aus den Blöcken zusammengesetzt, kompiliert (also in Sprache übersetzt, die der Microcontroller versteht) und über das USB Kabel aufgespielt.

Du hast nun dein erstes Programm ausgeführt. Klick dich gerne auf gleiche Weise durch die anderen Beispiele. Kannst du schon erkennen, was sie machen?

Dein erstes eigenes Programm

Nachdem du mit der Oberfläche nun ein wenig vertraut bist, willst du bestimmt endlich dein erstes eigenes Programm zusammensetzen, richtig? Sorge also für einen frischen Arbeitsbereich, indem du auf *Neu* klickst - es bleibt nur der **Startblock**.

Zunächst musst du dir Gedanken darüber machen, was dein Programm können soll. Für ein einfaches Beispiel können wir einen Teil der Umweltmessung von eben nachbauen. Am Ende soll auf dem Display des Makey:Lab die Umgebungstemperatur angezeigt werden.



Das ist dein ersten Programm. Gar nicht so schwer, oder?

Dafür brauchst du nicht viele Bausteine: Den Block **OLED Print** findest du unter dem Reiter *Anzeigen / Aktoren*, damit kannst du etwas auf dem Display anzeigen lassen.

Zieh ihn in den Arbeitsbereich und lasse ihn unter *Schleife* am Startblock einrasten. Was du später darauf siehst, bestimmst du über weitere Blöcke. Ein roter Text-Block wird automatisch als Platzhalter gesetzt. Den kannst du entweder entfernen oder den Inhalt (*text\u0020*) ersetzen - beispielsweise könntest du dort *Temperatur* schreiben - das steht dann später vor dem Sensorwert. Klick den Text einfach doppelt, lösche ihn oder überschreibe ihn. Zum Entfernen ziehst du einen Block einfach in die Blockspalte links.

Damit die Temperatur angezeigt wird, musst du diesen Block mit der Ausgabe des Umweltsensors verbinden, welchen du unter *Sensoren* findest - **Umwelt (T,F,p,VO) BSEC2**. Doch noch passt das nicht. Dir ist bestimmt schon aufgefallen, dass die Blöcke verschiedene Formen haben - grundsätzlich geht es dabei darum, welche Daten ein Block senden und empfangen kann. Damit der Umweltsensor (Dreieck) an das Display (Rechteck) passt, brauchen wir noch den entsprechenden Adapter-Block **verbinden** (der befindet sich unter *Print / Adapter*). Bau alles so zusammen, wie auf dem Bild.

Hinweis: Wenn du die Blöcke verbindest, achte auf das klackende Geräusch - nur wenn du es hörst, ist alles richtig eingerastet. Legst du den Block leicht neben der Verbindung ab, dann funktioniert dein Programm nicht. Achte auch darauf, dass Blöcke nicht übereinander liegen - sonst kann man schnell durcheinander kommen.

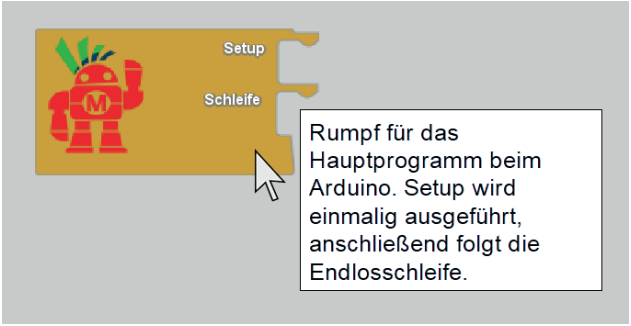
Jetzt musst du am Block des Umweltsensors nur noch einstellen, welchen Sensorwert du anzeigen lassen möchtest. Dafür kannst du über das Drop-Down-Menü (kleiner Pfeil nach unten) im roten Block hinter dem Umwelt-Sensorblock den richtigen Wert auswählen: *Temperatur*. Überträgst du den Sketch jetzt auf dein Makey:Lab, wird der entsprechende Wert auf dem Display angezeigt.

Prima, du hast dein erstes eigenes Gerät gebaut: Einen **Temperatur-Anzeiger**. Jetzt kennst du die Grundlagen des Makey:Lab – und eine spannende Welt voller Experimente wartet auf dich.

Jetzt bist du dran

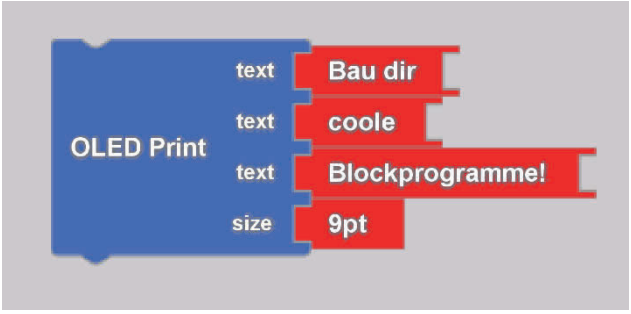
Bevor wir uns als nächstes mit einem richtigen Projekt befassen, beschäftige dich ruhig noch ein wenig mit der Programmoberfläche. Welchen Wert könntest du mit dem ersten Sketch noch anzeigen lassen? Ändere auch passend dazu den Text-Block, wenn du etwas anderes als die Temperatur anzeigen lässt. Zieh ein paar Blöcke in den Arbeitsbereich oder entferne sie wieder. Je vertrauter du mit der IoT-Werkstatt wirst, desto besser.

Wenn du es dir bereits zutraust, kannst du auch schon versuchen, eigene kleine Sketches zu bauen. Sobald du mit der Maus über einen Block fährst, zeigt dir ein kleiner Infotext, was dieser Block macht. Mit den Blöcken kannst du übrigens nichts kaputt machen – also keine Sorge.



Wenn du mit der Maus über einem Block schwebst, erscheint ein kleiner Erklärungstext.

Du hast jetzt das nötige Werkzeug: Ab jetzt kannst du deine eigenen Ideen umsetzen!



So einfach schreibst du etwas auf das Display!

Deine Welt steckt voller Projekte

Alles um dich herum kann zum Experiment werden

Vielleicht hast du noch keine Idee – oder fragst dich: „Wo soll ich überhaupt anfangen?“ Keine Sorge: Spannende Projekte lauern überall – manchmal direkt vor deiner Nase.

Überall dort, wo sich etwas verändert, wo du etwas beobachten, messen oder verbessern kannst, steckt Potenzial.

Warum beschlägt der Badezimmerspiegel? Wie laut ist es eigentlich im Klassenzimmer? Wird meine Pflanze regelmäßig gegossen?

Ein kleiner Moment der Neugier reicht – und plötzlich wird aus einer alltäglichen Beobachtung ein richtig spannendes Projekt.

Neugier ist nichts Neues

Schon seit Jahrhunderten bauen Menschen Instrumente, um ihre Welt besser zu verstehen: vom Sonnenstand in der Steinzeit bis zu modernen Wetterstationen. Beobachten, messen, verbessern – dieses Prinzip begleitet uns bis heute. Mit deinem Makey:Lab setzt du diese Tradition auf deine eigene Weise fort.

Technik begegnet uns überall: in deinem Zimmer, auf dem Balkon, im Garten oder im Jugendtreff. Mit dem Makey:Lab kannst du lernen, genauer hinzusehen: Was verändert sich? Was könnte man messen? Was kann daraus entstehen?

Zuhause findest du viele spannende Ansätze: In der Küche steigt beim Kochen die Luftfeuchtigkeit. Im Bad staut sich die warme Luft. Auf dem Balkon verändert sich die Sonneneinstrahlung. Im Keller zeigt sich vielleicht, ob die Luft zu feucht ist.

Auch in der Natur gibt es viel zu entdecken: Wie verändert sich die Temperatur im Laufe eines Tages? Wie feucht ist der Boden wirklich? Wie wirken sich Wetter und Licht auf Pflanzen aus?

In Gemeinschaftsräumen wie dem Vereinsheim, dem Jugendtreff oder der Schule kannst du messen, wie sich Luftqualität oder Lautstärke ändern, wenn viele Menschen zusammenkommen.

Überall, wo du misst, verstehst du besser – und kannst etwas verbessern. Kleine Entdeckungen führen oft zu neuen Ideen. Ideen, die etwas verändern können.

Veränderungen auf unserer Erde – vom Klimawandel bis zur Ressourcenknappheit – zeigen, wie wichtig es ist, Zusammenhänge zu erkennen.

Überall hörst du von Veränderungen. Aber welche Werte wirklich wichtig sind und sich auf dich und deine Umwelt

auswirken, kannst du mit deinem Makey:Lab selbst erforschen.

Sensoren: Deine Werkzeuge für neue Ideen

Sensoren sind wie kleine Sinne für Maschinen: Sie sehen, hören, fühlen oder messen, was um sie herum passiert. Mit deinem Makey:Lab kannst du deine eigenen technischen Sinnesorgane bauen und daraus Geräte entwickeln, die etwas bewirken.

Vielleicht hast du eine Idee, wie man Dinge verbessern oder Probleme lösen könnte – etwas, das es bisher noch nicht gibt. Dein Makey:Lab ist dein Werkzeug, um Neues zu schaffen: eigene Vorrichtungen, neue Geräte, kreative Lösungen.

Also: Wo möchtest du starten? Welche Ecke deiner Welt willst du als erstes genauer unter die Lupe nehmen?

Damit du nicht lange überlegen musst, starten wir gleich gemeinsam mit deinem ersten Projekt. Ein kleines Experiment, das zeigt, was dein Makey:Lab alles kann – und wie du schon mit wenigen Bausteinen spannende Entdeckungen machst. Vorher findest du noch ein paar Tipps, an welchen Orten Makey:Lab zum Einsatz kommen könnte.

Spickzettel: Orte für Projekte

Achtung, hier lauern Projekte!

Manchmal braucht es nur eine kleine Idee, um den ersten Schritt zu machen. Hier findest du Orte aus deinem Alltag – und Beispiele, welche Werte dort spannend sein könnten. Vielleicht hast du ja bald deine ganz eigene Messstation!

Zuhause

Bad: Luftfeuchtigkeit nach dem Duschen messen

Küche: Temperatur- und Feuchtigkeitsänderungen beim Kochen verfolgen, Obstkorb überwachen, trockene Wäsche erkennen, Frostwarner, Reifegrad von Gemüse und Obst erkennen

Schlafzimmer: Luftqualität am Morgen prüfen

Balkon: Sonneneinstrahlung und Außentemperatur messen

Schule

Klassenzimmer: Verbrauchte Luft messen und herausfinden, wann gelüftet werden sollte

Flure: Lautstärke in den Pausen beobachten

Werkraum: Temperatur und Luftqualität während des Unterrichts erfassen

In der Freizeit

Gruppenräume: Im Vereinsheim, in der Sporthalle - Luftqualität messen, wenn viele Menschen zusammenkommen

Hobbyräume: Temperatur und Feuchtigkeit überwachen

Veranstaltungsräume: Lautstärke und Lärmspitzen erfassen

Draußen in der Natur

Garten: Bodenfeuchtigkeit und Temperatur überprüfen

Park: Lichtintensität an verschiedenen Orten vergleichen

Sportplatz: Temperaturveränderungen durch Sonne und Schatten beobachten

Du siehst: Egal wo du bist – Projekte warten überall auf dich. Jetzt liegt es an dir, wo du deinen ersten Sensor platzierst!

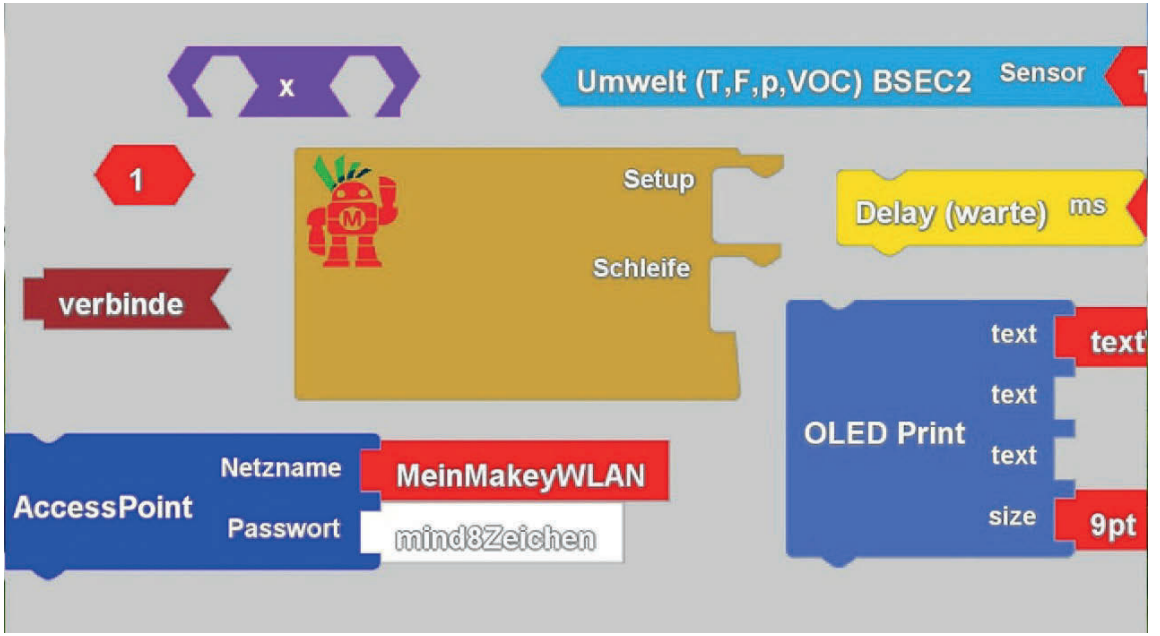
Und noch ein kleiner Hinweis: Du kannst dein Makey:Lab später durch unzählige weitere Sensoren erweitern oder sogar Motoren ansteuern, Daten an dein Smartphone schicken und vieles mehr.

Bevor du loslegst, schauen wir uns aber kurz an, wie das Programmieren mit Blöcken funktioniert – und wie dein Makey:Lab „denkt“.

Noch viel mehr tolle Idee:

SCAN MICH

Hinter dem QR-Code findest du eine *Welt voller Projekte* am Umwelt-Campus Birkenfeld. Dort lernst du spannende Ideen aus Hackathons, dem Innovationslabor und der IoT-Werkstatt kennen – und erfährst, wie Making als Hoffnungsträger für Innovation und Nachhaltigkeit wirkt. Lass dich von über 100 Maker-Projekten inspirieren und starte dein eigenes *Zukunftsexperiment*!



So denkt ein Programm

(Und was das mit Blöcken zu tun hat)

Vielleicht hast du dich schon gefragt, was du da eigentlich machst, wenn du mit Blöcken arbeitest. Ist das wirklich Programmieren? Und wie funktioniert das unter der Haube?

Die Antwort ist ganz einfach: Ja, das ist Programmieren – nur in einer Form, die du sehen und anfassen kannst. Statt komplizierter Wörter und Sonderzeichen nutzt du farbige Bausteine, die sich wie ein Puzzle zusammensetzen lassen. Doch hinter diesen Blöcken steckt echte Programmierlogik – und ziemlich mächtige Technik.

Wie sieht Programmieren „normalerweise“ aus?

In der klassischen Programmierung arbeitet man mit sogenanntem Quelltext – also geschriebenem Code. Das sind Texte, die dem Computer sagen, was er tun soll. Ein ganz einfaches Programm sieht zum Beispiel so aus:

```
void setup() {
  Serial.begin(9600);
}

void loop() {
  Serial.println("Hallo, Welt!");
}
```

Das ist eine Programmiersprache. In diesem Fall nennt sie sich Arduino C++. Sie ist mächtig – aber auch empfindlich. Ein fehlendes Semikolon oder eine falsch geschriebene Klammer, und das Programm funktioniert nicht mehr. Deshalb arbeiten viele Einsteiger:innen lieber mit grafischen Blöcken, wie sie das Makey:Lab bietet. Sie machen dieselben Dinge – aber in einer leichter verständlichen Form.

Blöcke statt Klammern

Beim Makey:Lab programmierst du mit der IoT-Werkstatt. Die bunten Blöcke stehen dabei für echte

Programmbefehle – nur viel verständlicher. Jeder Block hat eine Aufgabe, z. B.:

„*Zeige Text auf dem Display*“
„*Wenn der Knopf gedrückt wird, dann…*“
„*Warte 1 Sekunde*“

Du musst nichts tippen oder Klammern setzen – stattdessen ziehst du die passenden Bausteine an die richtige Stelle. Und genau wie beim klassischen Programmieren gibt es dabei zwei wichtige Bereiche: Setup und Schleife.

Der Startblock

Im Makey:Lab findest du ganz oben immer den **Programm**-Block, den wir auch **Startblock** nennen. Er ist das Herzstück deiner Anwendung, sozusagen das Haus, in dem dein ganzes Programm wohnt. Darin gibt es zwei besondere Bereiche.

Setup (Einrichten)

Dieser Bereich wird genau einmal ausgeführt, ganz am Anfang. Hier sagst du dem Makey:Lab, was es einmalig vorbereiten soll – z. B.:

- Den Bildschirm einschalten
- Sensoren aktivieren
- Einen Anfangswert setzen

Was hier passiert, wird nur einmal gemacht – beim Start. Danach geht's weiter zur Schleife.

Schleife (Loop)

Die Schleife läuft immer wieder und wieder – das ist der eigentliche „Motor" deines Programms. Was du hier hineinschreibst, wird ständig wiederholt, viele Male pro Sekunde. Du brauchst diese Schleife, damit dein Programm z. B. ständig Werte messen, auf Knöpfe reagieren oder etwas anzeigen kann.

Beispiel:

„*Wenn die Temperatur über 25 °C steigt, dann zeige eine Warnung.*"

„*Wenn Licht erkannt wird, schalte eine LED an.*"

„*Wenn jemand klatscht, gib einen Ton aus.*"

All das passiert in der Schleife – weil der Computer sonst sofort wieder schlafen gehen würde. Er braucht eine Aufgabe, die ihn wach hält. Und genau das ist deine Schleife.

Warum ist die Schleife so wichtig?

Stell dir vor, du willst regelmäßig prüfen, ob ein Fenster offen ist. Wenn du das nur einmal machst (wie im Setup), bekommst du nur einen Wert. Aber wenn du jede Sekunde nachsiehst (wie in der Schleife), dann weißt du

immer sofort, wenn sich etwas ändert und was der aktuelle Wert ist.

Warum Blöcke für den Anfang super sind

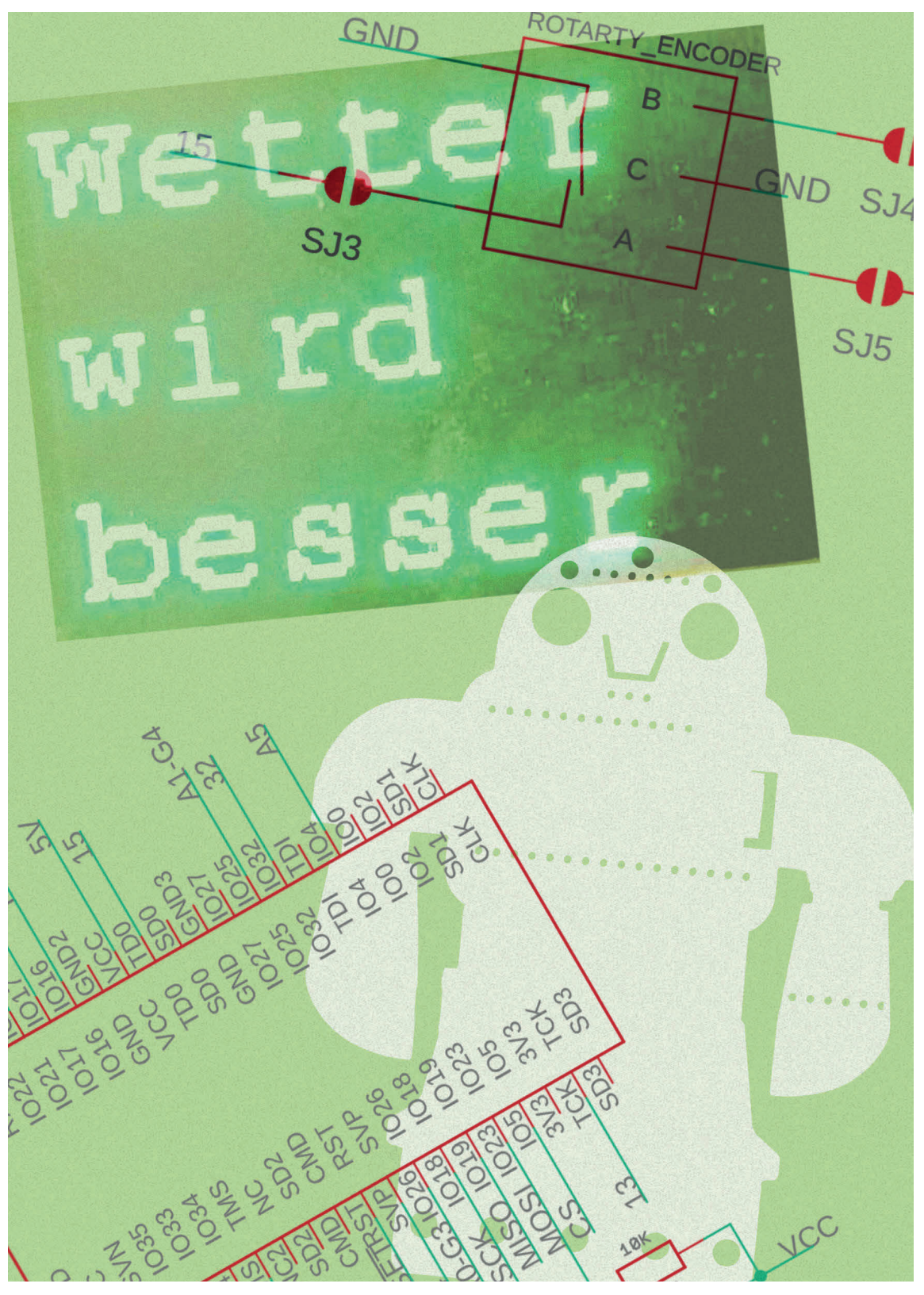
- Sie helfen dir, Fehler zu vermeiden (z. B. durch falsche Schreibweise)
- Du siehst sofort, was zusammenpasst
- Du verstehst die Struktur eines Programms – ohne komplizierte Sprache
- Du lernst nebenbei echte Programmierlogik: Bedingungen, Wiederholungen, Ereignisse

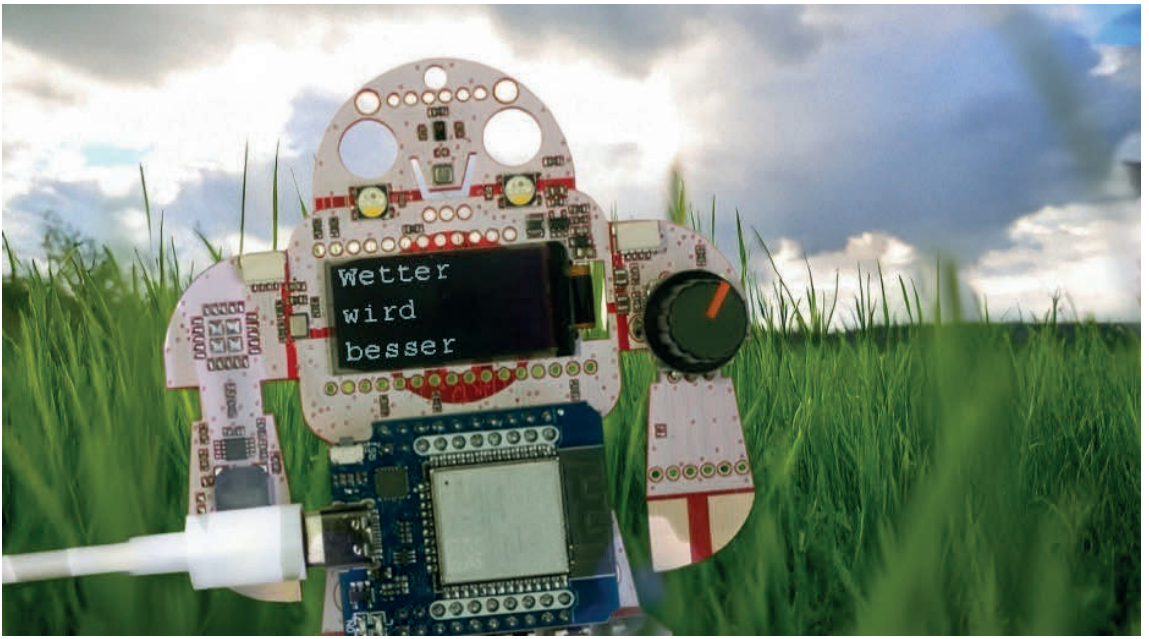
Selbst viele Profis nutzen Blöcke, um neue Ideen schnell auszuprobieren – und steigen später auf Textprogrammierung um. Du kannst also klein anfangen – und dabei trotzdem etwas Großes lernen.

Zusammengefasst:

- Programme bestehen aus zwei Teilen: Setup (einmalig) und Schleife (immer wieder)
- Der **Startblock** ist das Zuhause für alles
- Jeder Block ist ein echter Befehl
- Mit Blöcken kannst du Technik sichtbar steuern – ganz ohne Vorwissen
- Später kannst du alles auch in Textform lernen – wenn du willst

Bereit für dein erstes echtes Projekt? Dann geht’s jetzt los – mit deiner eigenen Wetterstation!


Wetter
wird
besser
GND
ROTARTY_ENCODER
B
C
A
GND
SJ4
SJ5
15
SJ3
VCC
10K
3
SD3
TCK
3V3
105
MOSI
MISO
SCK
SD2
CMD
RST
SVP
SVN
TMS
NC
SD1
CLK
SD0
TD0
VCC
GND
IO16
IO17
IO21
IO22
IO19
IO18
IO26
IO23
IO5
IO25
IO27
IO32
TDI
IO4
IO0
IO2
5V
A5
A2-G4
32



#01 Bau dir deinen eigenen Wetterfrosch

Mit Sensoren siehst du, wie das Wetter wird

Mit dem Makey:Lab brauchst du keine Wetter-App und auch keinen Wetterfrosch mehr. Jetzt baust du dir deine eigene Wetterstation, die dir genau anzeigt, wie das Wetter direkt vor deiner Tür ist. Dafür sammelst du einfach ein paar Daten mit dem Umweltsensor.

Wetterdaten verstehen

Du kannst beispielsweise Temperatur, Luftfeuchtigkeit und Luftdruck messen. Doch was sagen diese Werte eigentlich aus? Und warum sind sie für das Wetter so wichtig?

Temperatur zeigt dir, wie warm oder kalt die Luft ist. Je nach Sonnenstand, Tageszeit oder Jahreszeit kann sie stark schwanken. Wenn die Temperatur bei etwa 20 bis 22 Grad liegt, empfinden wir das in unseren Breiten meist als angenehm – das ist der sogenannte Komfortbereich. An besonders heißen Tagen reicht dann oft schon ein T-Shirt, während an kühlen Morgenstunden vielleicht noch eine Jacke nötig ist.

Wusstest du? Wenn die Temperatur nachts nicht unter 20 Grad fällt, spricht man von einer "Tropennacht". Viele Menschen empfinden das als unangenehm, weil der Körper sich schlechter abkühlen kann – das kann sogar die Gesundheit beeinträchtigen.

Luftfeuchtigkeit sagt dir, wie viel Wasserdampf sich in der Luft befindet. Wasserdampf kennst du zum Beispiel vom Kochen oder Duschen. Aber auch wenn du keinen sichtbaren Dampf siehst, steckt in der Luft immer etwas Wasser. Die Luftfeuchtigkeit zeigt dir, wie viel davon gerade enthalten ist. Das kann auch gesundheitlich relevant sein - gerade bei Atemwegserkrankungen wie Asthma oder Allergien.

Die Luftfeuchtigkeit ist aber nicht nur in Räumen spannend, sondern auch draußen: Hohe Luftfeuchtigkeit kann das Wetter verändern und macht die Luft manchmal spürbar "dicker". Du kennst das bestimmt: Schwüle Luft fühlt sich oft schwer an – das liegt an der Kombination

aus hoher Temperatur und hoher Luftfeuchtigkeit. Eine hohe Luftfeuchtigkeit kann übrigens auch ein Zeichen dafür sein, dass es bald regnet.

Luftdruck ist ein entscheidender Wert, wenn es ums Wetter geht. Er beschreibt, wie stark die Luft auf uns "drückt". Warme Luft steigt auf, kalte sinkt ab – dadurch bewegt sich die Atmosphäre und es entstehen Winde. Je nach Luftdrucklage kann sich also das Wetter deutlich ändern.

Wenn wir höher steigen, etwa auf einen Berg, wird der Luftdruck geringer – die Luft ist "dünner". Das kann man sogar beim Atmen spüren. Auch im Alltag kann sich der Luftdruck auf uns auswirken - bei manchem Menschen verursachen Luftdruckveränderungen sogar Kopfschmerzen Unwohlsein.

Für unsere Wetterstation ist wichtig: Ein sinkender Luftdruck bedeutet oft, dass Regen oder ein Gewitter bevorsteht. Deshalb ist er so hilfreich für Vorhersagen.

So liest du das Wetter

Je nachdem, wie Temperatur, Luftfeuchtigkeit und Luftdruck zusammenwirken, kannst du unterschiedliche Wetterszenarien erkennen:

Schönes Wetter: Konstante Temperaturen, niedrige bis mittlere Luftfeuchtigkeit und stabiler, hoher Luftdruck deuten oft auf ruhiges, sonniges Wetter hin.

Unwetter im Anmarsch: Steigende Luftfeuchtigkeit und gleichzeitig fallender Luftdruck sind ein Warnsignal. Es kann bald zu Regen oder Gewitter kommen. (Merk dir das schon mal für später!)

Drückend schwüle Tage: Hohe Temperaturen bei hoher Luftfeuchtigkeit machen die Luft besonders unangenehm. Das ist oft im Sommer der Fall und kann auf Hitzegewitter hindeuten.

Wetterumschwung: Wenn der Luftdruck schnell sinkt, kündigt sich meist eine Wetterveränderung an.

Später schauen wir uns das nochmal in genauen Messwerten an. Doch zunächst müssen wir noch das Makey:Lab vorbereiten.

Deine erste Mini-Wetterstation

Dein Makey:Lab sieht zwar nicht aus wie ein Frosch, aber wir machen es jetzt trotzdem zum Wetterfrosch – denn so nennt man traditionell Geräte (oder Tiere), die das Wetter vorhersagen können. Und genau das wollen wir jetzt tun: eine eigene kleine Wetterstation bauen. Alles, was wir brauchen, bringt Makey:Lab schon mit – wir müssen also nur das richtige Programm aufspielen.

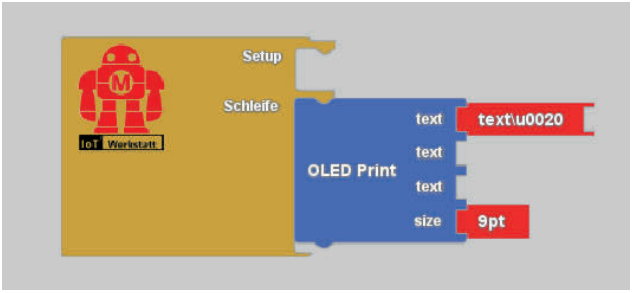
Mit dieser einfachen Wetterstation lernst du nicht nur, wie man misst – sondern du verstehst, wie das Wetter funktioniert. Und du bekommst ein Gefühl dafür, was Temperatur, Feuchte und Druck in deinem Alltag bedeuten.

Falls noch nicht geschehen, schließe das Makey:Lab wieder an deinen Computer an und öffne die Software. Du solltest nun die IoT-Werkstatt vor dir haben. Auf der Arbeitsfläche muss der *Startblock* liegen. Falls dort noch ein anderer Sketch geöffnet ist, kannst du ihn speichern – wenn du magst – und dann auf "*Neu*" klicken.

Display als Anzeige nutzen

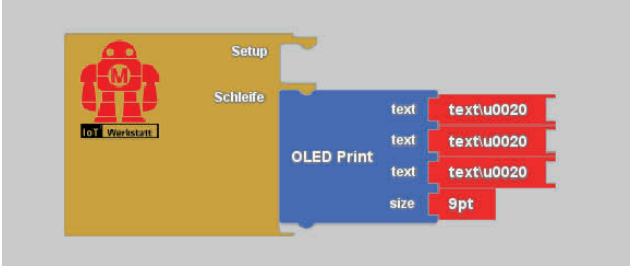
Da wir die Wetterdaten auf dem Bildschirm anzeigen lassen wollen, brauchst du wieder den **OLED Print**-Block. Du findest ihn unter dem Reiter *Anzeigen / Aktoren* und hängst ihn unter *Schleife* beim *Startblock* an. Er kann gleich drei Zeilen Text auf einmal anzeigen.

Zunächst siehst du in der ersten Zeile wieder den roten Text-Platzhalter (*text\u0020* - wobei *text* durch eine beliebigen eigenen Text ersetzt werden kann.). Den können wir gleich nutzen und dort angeben, was wir anzeigen lassen wollen.



Die Blöcke kannst du einfach ineinander einrasten lassen.

Zunächst kopieren wir ihn zweimal für die beiden anderen Zeilen. Dafür kannst du ihn mit der rechten Maustaste anklicken und *Klonen* wählen – ein zweiter Block erscheint. Schiebe ihn in die zweite Zeile und raste ihn ein. Den Vorgang wiederholst du dann für die dritte Zeile.



Der OLED Print-Block kann in jeder Zeile etwas anderes anzeigen.

Jetzt benennen wir die Text-Blöcke um: Mit Doppelklick auf den Text kannst du die Beschriftung auswählen und mit der Rücktaste (Backspace) löschen oder die Auswahl einfach überschreiben.

Vielleicht fragst du dich, was es mit dem *\u0020* auf sich hat? Das ist sogenannter **Unicode** – eine besondere Art für Computer, Zeichen wie Leerzeichen, Buchstaben oder Symbole als Zahlencode darzustellen. Statt das Zeichen direkt über die Tastatur einzugeben, wird es hier als Zahl aufgeschrieben. Warum? Damit das Zeichen später im Programm garantiert richtig erkannt und angezeigt wird – ganz egal, auf welchem Computer oder System das Programm läuft. So kommt nichts durcheinander.

Interessante Unicodes

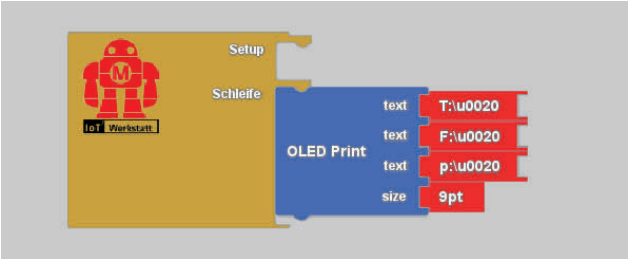
Hier sind ein paar Codes, die häufiger mal vorkommen:

- *\u0020* - das ist ein Leerzeichen
- *\u003A* - das ist ein Doppelpunkt
- *\u0022* - Anführungszeichen oben
- *\u002B* - Plus
- *\u002D* - Minus oder einfacher Bindestrich

Viele dieser Zeichen kannst du in der IoT-Werkstatt einfach trotzdem nutzen, aber gerade beim Leerzeichen brauchen wir den entsprechenden Code dafür.

Du kannst das mit dem Unicode gleich mal ausprobieren, schreibe in die Textblöcke jeweils, was du anzeigen willst. Also beispielsweise:

- *T:\u0020*
- *F:\u0020*
- *p:\u0020*



Mit der richtigen Beschriftung wird es übersichtlich.

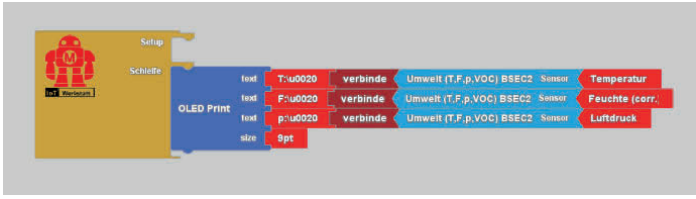
Hier sind die Ausdrücke abgekürzt, da auf dem Display nur begrenzt Platz ist. T steht für Temperatur, F für Luftfeuchtigkeit und für Luftdruck *p* – das steht entweder für das englische Wort *pressure*, was Druck heißt, oder lässt sich auf den Physiker Blaise Pascal zurückführen. Noch heute wird der Druck in der Maßeinheit *Pascal* angegeben.

Kleiner Tipp: Den Backslash (umgedrehter Schrägstrich) \ auf der Tastatur kannst du übrigens schreiben, wenn du die Alt-Taste gedrückt hältst und dann auf die ß-Taste rechts neben der Null tippst.

Umweltsensor einbauen

Nachdem wir die Beschriftung für die Werte angegeben haben, fehlen jetzt eigentlich nur noch die Messdaten aus dem Umweltsensor – den kennst du bereits. Wir nutzen auch wieder den **Verbinden**-Adapter. Davon holst du dir drei Stück und lässt sie jeweils am Textblock einrasten. Unter Sensoren findest du dann den blauen Block **Umwelt (T,F,p,VO) BSEC2**. Diesen hängst du ebenfalls dreimal an deine Adapter. Jetzt sollte alles passen. Wenn nicht, dann schau nochmal, ob du das richtige Verbindungsstück ausgewählt hast – es gibt verschiedene.

Hinter dem Umweltblock siehst du wieder die rote Auswahl. Stelle den Wert über das kleine Ausklappmenü (dreieckiger Pfeil nach unten) jeweils auf **Temperatur**, **Feuchte (corr.)** und **Luftdruck**. Jetzt ist alles eingerichtet.



Wähle rechts am Sensor, welche Daten er ausgeben soll.

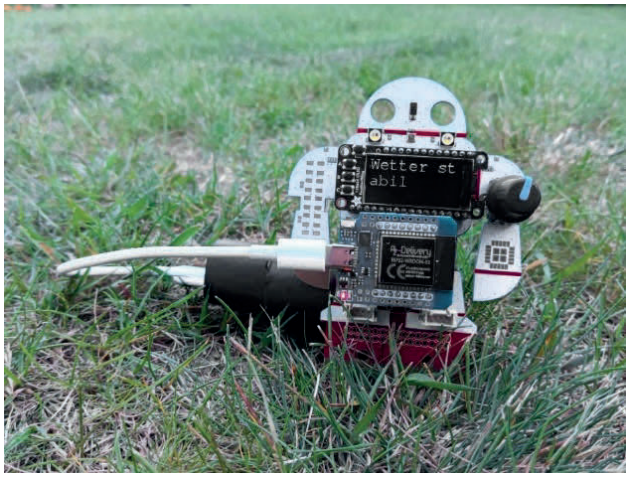
Wenn du magst, kannst du unter den drei Textzeilen die Größe der Schrift einstellen. Der Standardwert liegt bei *9pt* – je höher der Wert, desto größer die Schrift. Das ist

eine Angabe aus dem Schriftsatz, also Bücher und andere gedruckte Texte. *pt* steht für Point (deutsch: Punkt).

Es gibt einige Werte zur Auswahl. Da wir aber schon recht viel anzeigen lassen wollen, ist *9pt* bereits ein guter Wert.

Wenn du nun auf *Upload* drückst, wird dein Programm auf Makey:Lab übertragen und zeigt dir danach direkt die entsprechenden Werte an. Wenn du willst, kannst du die Werte auch aufschreiben oder mit denen deiner Wetter-App vergleichen. Vielleicht misst deine Station sogar schneller oder genauer!

Kleiner Versuch:
Jetzt, wo dein Makey:Lab die Werte anzeigt, kannst du ausprobieren, wie empfindlich der Luftdruck auf die Höhe reagiert. Lege dein Makey:Lab auf den Boden und lies den Druckwert ab. Heb es dann hoch – etwa auf den Tisch oder auf einen Stuhl – und beobachte die Anzeige. Du wirst sehen: Der Druck sinkt ein kleines bisschen, weil mit jedem Meter Höhe weniger Luft über dir lastet. Schon wenige Zentimeter machen messbar etwas aus!



Mit einer einfachen Powerbank, kannst du dein Makey:Lab mit nach draußen nehmen.

Daten messen: drinnen und draußen

Damit du ein gutes Gespür für die Zahlen auf dem Bildschirm bekommst, findest du hier eine Erklärung, was du daran ablesen kannst und welche Werte zu erwarten sind.

Temperatur wird bei uns in Grad Celsius gemessen. Beim Wetterbericht bezieht sich die Temperatur auf den Schatten, weil sie dort die Umgebungstemperatur ohne direkte Sonneneinstrahlung am besten widerspiegelt.

Luftfeuchtigkeit wird in Prozent angegeben. Sie zeigt, wie viel Wasserdampf aktuell in der Luft ist – im Verhältnis zur maximal möglichen Menge bei dieser Temperatur.

Beispiel: 45 % bedeutet, dass die Luft gerade 45 % der Feuchtigkeit enthält, die sie bei dieser Temperatur aufnehmen könnte.

Draußen (Wetter/Luft)

Situation	Typische rF (%)
Sonniger Sommertag, trocken	30–40 %
Angenehmes Wetter (Frühling/Herbst)	45–60 %
Schwüles, drückendes Wetter	70–90 %
Nebel, Regen oder direkt nach Regen	90–100 %
Wüste (z. B. „Sahara“)	<10 %

Wusstest du? Je wärmer die Luft, desto mehr Wasserdampf kann sie aufnehmen – deshalb fühlt sich warme, feuchte Luft oft „klebrig“ an. Sie enthält auch mehr Energie, weil der Wasserdampf bei der Kondensation Wärme freisetzt. Das kann starke Aufwinde und Luftwirbel erzeugen – daraus entstehen Gewitter, Stürme oder unter bestimmten Bedingungen sogar Tornados!

Verdampfen vs. Kondensieren – Was passiert mit der Energie?

Prozess	Was passiert?	Energiefluss	Wirkung auf die Umgebung
Verdampfen	Flüssiges Wasser wird zu Wasserdampf	Wärme wird aufgenommen	Umgebung kühlt ab (z. B. Schweiß auf der Haut)
Kondensieren	Wasserdampf wird wieder flüssig	Wärme wird freigesetzt	Umgebung wird wärmer (z. B. beschlagene Spiegel)

Merksatz: Beim **Verdampfen** verschwindet Wärme - beim **Kondensieren** kommt sie zurück.

Drinnen (Wohnung/Klassenzimmer)

Raumklima	Typische rF (%)	Bemerkung
Ideal für Wohlbefinden	40–60 %	Gute Balance für Haut, Schleimhäute, Atemwege
Zu trocken (z. B. im Winter mit Heizung)	<30 %	Kann Augen und Haut reizen
Zu feucht (Schimmelgefahr)	>65–70 %	Dauerhaft zu feucht = Problem

Luftdruck wird wird üblicherweise in Hektopascal (hPa) gemessen. Er hängt stark von Höhe, Wetterlage und geographischer Lage ab.

Hier eine Übersicht über übliche Luftdruckwerte in verschiedenen Situationen:

Normale Luftdruckbereiche auf Meereshöhe

Situation	Typischer Luftdruck (hPa)	Bedeutung bzw. Wetterlage
Normaldruck („Standardatmosphäre“) auf Meereshöhe	1013,25 hPa	Referenzwert (ISA: *International Standard Atmosphere*)
Hochdruckwetter (Hochdruckgebiet)	1020–1045 hPa	Meist sonnig, trocken, stabil
Tiefdruckwetter (Tiefdruckgebiet)	980–1005 hPa	Unbeständiges Wetter, Regen, Wind
Starkes Tief / Sturm	<980 hPa	Sturmgefahr, Unwetter
Orkan / Tropensturm (Auge)	<950 hPa	Extrem selten in Mitteleuropa

Luftdruck nach Höhe über dem Meer

Der Luftdruck nimmt mit der Höhe *exponentiell* ab. Das heißt: Erst verändert er sich nur wenig – aber je höher du steigst, desto schneller wird die Luft dünner.

Höhe über NN	Typischer Druck (hPa)	Vergleich
0 m	ca. 1013 hPa	Normaldruck
500 m	ca. 955 hPa	z. B. Harz, Mittelgebirge
1000 m	ca. 900 hPa	z. B. Alpenvorland
2000 m	ca. 800 hPa	z. B. Berghütten
3000 m	ca. 700 hPa	z. B. Zugspitze
8000 m	ca. 356 hPa	z. B. Mount Everest
11 000 m	ca. 226 hPa	Reiseflughöhe im Flugzeug

Luftdruck im Alltag – was ist „normal“?

In Deutschland schwankt der Luftdruck auf Meereshöhe meist zwischen: **980 hPa (Tief)** und **1040 hPa (Hoch)**.

Alles innerhalb dieses Bereichs ist ganz normal. Druckschwankungen werden z. B. bei Wetterumschwüngen, Föhn (ein warmer Wind aus den

Bergen, der plötzlich das Wetter ändert) oder Gewittern spürbar.

Über einen längeren Zeitraum messen

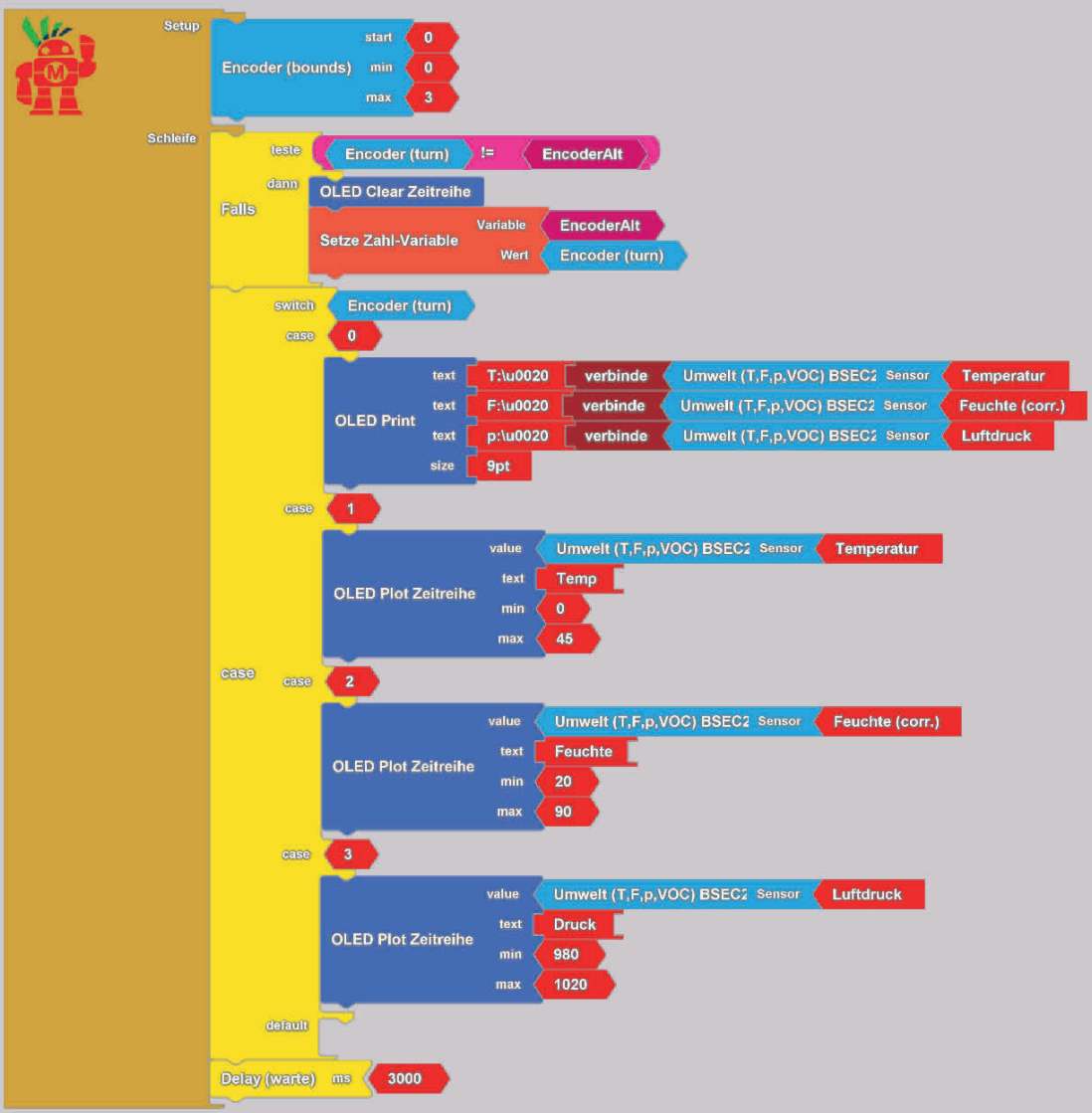
Wie wir noch sehen werden, kann dir ein einzelner Messwert schon viel über das aktuelle Wetter verraten. Richtig spannend wird es aber, wenn du die Werte über längere Zeit verfolgst und dir den Verlauf in einem Diagramm anzeigen lässt. Dann erkennst du, wie sich das Wetter verändert. Klingt gut? Genau das machen wir jetzt.

Für jeden unserer drei Werte können wir beispielsweise eine eigene Kurve auf dem Display anzeigen lassen – was du sehen willst, kannst du dann mit dem **Dreh-Drück-Encoder** auswählen. Unsere bisherige Übersicht über alle drei Werte können wir auch behalten.

Der Aufbau in der IoT-Werkstatt wird nun schon ein wenig umfangreicher, aber keine Sorge: Wir gehen alles Schritt für Schritt durch.

Den bisherigen Block, also **OLED Print** mit den Messwerten für Temperatur, Luftfeuchtigkeit und Luftdruck, kannst du nun aus dem Startblock aushängen. Dafür greifst du ihn und ziehst ihn von

das ganze Programm



Startblock ab und legst ihn auf einen freien Bereich - alle angehängten Blöcke bleiben dabei erhalten.

Das neue Programm teilt sich grob in zwei Bereiche:

- Der **Encoder** muss eingebunden werden. Makey:Lab soll erkennen, wenn man daran dreht und es muss bestimmt werden, was dann passieren soll.
- Wir brauchen verschiedene Ansichten, die unsere Werte zeigen. Hier wird auch der Umweltsensor verknüpft.

Der Encoder

Da wir nun das erste Mal den Encoder selbst in ein Sketch einbauen, wollen wir zunächst klären, was er alles kann und was wir damit vorhaben.

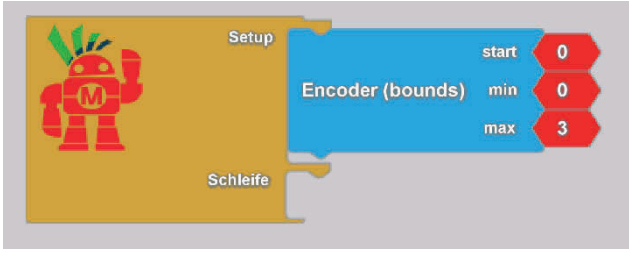
Bei dem *Dreh-Drück-Encoder* ist der Name Programm: man kann ihn endlos drehen und drücken. Wenn du ihn bewegst, dann wirst du feststellen, dass er gerastert ist. Das macht ihn besonders gut für unseren Einsatzzweck, da er genaue Werte ausgibt, die wir dann auslesen und verwenden können. Auf Deutsch nennt man einen Encoder auch Drehgeber oder noch einfacher Drehregler.

Damit Makey:Lab weiß, dass du den Encoder verwenden willst, musst du ihn in dein Programm einbinden. Außerdem teilen wir mit, wie viele seiner ausgegebenen Werte wir verwenden wollen.

Dafür suchen wir unter Sensoren den Block **Encoder (bounds)** und hängen ihn am **Startblock** unter Setup ein. Damit bereitest du ihn vor, um ihn in der Programm-Schleife nutzen zu können. Der Block hat drei Eingabefelder: **Startwert**, **Minimum** und **Maximum**. Den Startwert kannst du in der Regel bei **0** lassen – er bestimmt, mit welchem Wert der Encoder beim Programmstart beginnt. Jetzt musst du noch den Wertebereich einstellen, den er ausgeben soll. Dafür legst du einen minimalen und einen maximalen Wert fest; diese stehen aktuell noch auf **–100** und **100**.

Da wir nur vier Werte brauchen, für unsere vier Ansichten, gibst du hier 0 und 3 ein.

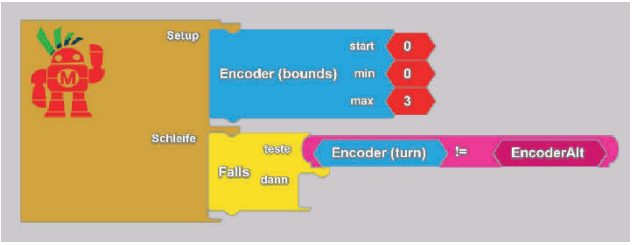
Erklärung: In der Programmierung fängt ein Zahlenbereich häufig nicht mit der 1, sondern mit der 0 an - in unserem Fall macht das also vier Möglichkeiten: 0, 1, 2 und 3.



Hier stellen wir den Wertebereich des Encoders ein.

Im nächsten Schritt bauen wir eine Überprüfung ein, die kontrolliert, ob sich der Encoder bewegt hat und was

dann passieren soll. Das kann der Block **Falls (teste / dann)**, den findest du unter *Kontrollstrukturen*.



Mit den Blöcken sind Vergleiche leicht möglich.

Damit soll verglichen werden, ob der aktuelle Encoder-Wert noch dem alten Wert entspricht. Dafür nimmst du den **vergleiche**-Block unter *Logische Vergleiche*, der in der Mitte ein *!=* hat. Hier wird verglichen, ob der erste Block einen anderen Wert als der zweite hat (also, ob er "ungleich" ist - dafür steht das *!=*). Die Werte die du dort einsetzt sind **Encoder (turn)**, also der aktuelle Wert, und eine eigene Variable *EncoderAlt*.

Encoder (turn) liegt wieder unter *Sensoren*. Den Baustein *EncoderAlt* gibt es so noch nicht, den musst du dir selbst bauen. Nimm aus dem Reiter *Variablen / Konstanten* den Block **Name Integer Variable**. Diesen Block kannst du ähnlich wie den Textblock umbenennen. Doch erst einmal schauen wir uns an, was Variablen überhaupt sind.

Über Variablen

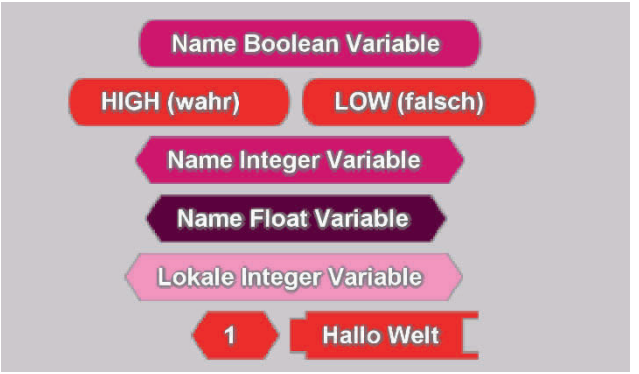
Eine Variable ist wie ein Schublade mit einem eindeutigen Namen. Du kannst dort einen Wert hinterlegen, später wieder auslesen oder ändern, also wie ein kleines Gedächtnis. Hier kannst du auch den alten Encoder-Wert zwischenspeichern für unseren Vergleich.

Es gibt verschiedene Variablen, je nach dem, was sie beinhalten. Hier eine kleine Übersicht:

Typ	Beschreibung	Beispiel
int	Ganze Zahl (ca. –32.000 bis +32.000)	0
float	Kommazahl	22.5
boolean	Wahr oder Falsch (1 oder 0)	true oder false
char	Einzelnes Zeichen	A
string	Text	Temperatur
byte	Ganze Zahl von 0 bis 255	128
unsigned int	Nur positive ganze Zahlen	1000

Die IoT-Werkstatt nimmt uns hier Arbeit ab: Wir müssen nicht selbst sagen, welchen Typ eine Variable hat. Stattdessen wird automatisch erkannt, was gemeint ist –

und das hängt davon ab, wie und welche Blöcke miteinander verbunden sind.



Variablen haben unterschiedliche Formen und Farben.

Ein Block mit dreieckiger Verbindung steht hier für eine Zahl. Er kann sowohl ganze Zahlen (Integer, z. B. 0, 1, 2, 3) als auch Kommazahlen (Float, z. B. 2.5, 3.3) enthalten.

Wichtig: Kommazahlen werden hier immer mit Punkt geschrieben – also 3.3 statt 3,3.

Andere Formen stehen für Texte oder Wahr/Falsch-Werte. So ein Wahr-Wert kommt übrigens auch aus dem Vergleichsblock (schau ihn dir mal genau an), der zusammen mit dem **Falls**-Block dann eine Aktion auslöst, die wir unter *dann* definieren. Dazu gleich mehr.

Grundsätzlich kannst du den Namen für eine Variable frei wählen, wichtig dabei ist aber, dass der Name **immer exakt gleich geschrieben** werden muss, wenn du ihn verwendest.

Darauf ist zu achten:

- Variablennamen dürfen nur Buchstaben, Ziffern und Unterstriche enthalten (a–z, A–Z, 0–9, _)
- Sie dürfen nicht mit einer Zahl beginnen (also *score1* ist ok, *1score* nicht)
- keine Leerzeichen zwischen den Wörtern
- Keine Sonderzeichen oder Umlaute (ä, ö, ß, etc. sind tabu)
- Groß- und Kleinschreibung zählt – *EncoderAlt* und *encoderAlt* sind zwei verschiedene Variablen

Wie genau man Variablen schreibt, hängt vom persönlichen Stil ab. Es haben sich allerdings ein paar Schreibweisen etabliert, beispielsweise:

- *camelCase* - Erste Silbe klein, jede weitere mit großem Anfangsbuchstaben. Beispiel: *drehWert*, *encoderPosition*, *anzahlDruecke*
- *PascalCase* - Alle Wörter groß
- *snake_case* - Alle Wörter klein, mit Unterstrich getrennt. Beispiel: *dreh_wert*, *encoder_position*, *anzahl_druecke*

Als Beispiel haben wir *EncoderAlt* gewählt. Der Name spricht für sich und sagt ganz klar: das ist der alte Encoder-Wert.

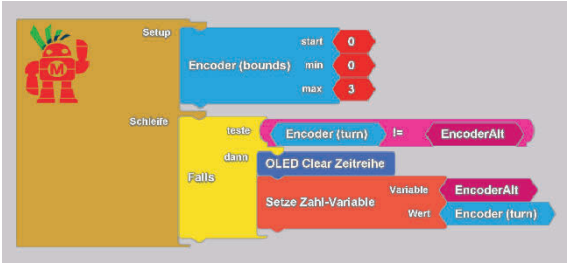
Und dann?

Du hast jetzt eine Überprüfung in dem **Falls**-Block aufgebaut. Diese bekommt eine positive Rückmeldung, wenn der aktuelle Encoder-Wert anders als der *EncoderAlt*-Wert ist. Dadurch wir die Aktion ausgelöst, die wir nun unter *dann* zusammensetzen.

Zunächst soll der Bildschirm bereinigt werden. Dies macht der Block **OLED Clear Zeitreihe** – er entfernt alte Zeitreihen aus dem Speicher. Da der Zeitreihen-Block immer denselben Speicherbereich nutzt, würden sich sonst mehrere Kurven gegenseitig überschreiben. Im nächsten Block wollen wir nämlich neue Zeitreihen anzeigen lassen.

Damit nach dem Drehen erneut eine Überprüfung gemacht werden kann, nutzt du zusätzlich den Block **Setze Zahl-Variable**.

Unter der Variable *EncoderAlt* soll der aktuelle *Encoder (turn)* Wert übernommen werden - dieser gibt die Drehbewegung aus. Klone also die Blöcke aus der Überprüfung und setze sie in den neuen Block ein. *EncoderAlt* unter **Variable** und *Encoder (turn)* unter **Wert**.



Unsere Programm-Logik auf einen Blick.

Kurz: Neuer und alter Wert werden verglichen, dann wird der neue Wert zum alten Wert - damit die Überprüfung beim nächsten Drehen auch wieder funktioniert.

Nun kümmern wir uns um die Anzeige und die Einbindung des Umweltsensors.

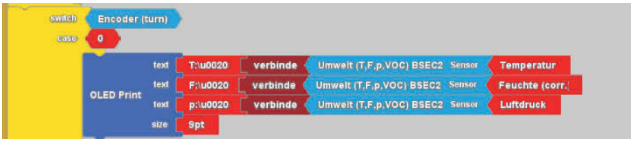
Daten anzeigen lassen

Jetzt braucht unser Programm noch eine Entscheidungshilfe: Es soll je nach Encoder-Stellung wissen, was es anzeigen soll. Dafür verwenden wir den **case**-Block. Den findest du unter dem Reiter Kontrollstrukturen.

Ein Case-Block (von englisch case = Fall) prüft, welcher von mehreren möglichen Fällen oder Werten gerade aktiv ist, und führt dann den passenden Programmteil aus.

Als Auslöser setzen wir oben am Block unter **switch** (Schalter) den *Encoder (turn)*, danach wird abgefragt, welcher Wert zutrifft: 0, 1, 2 oder 3? Wenn du den **case**-

Block auf deinen Arbeitsbereich legst, sind die Variablen bereits voreingestellt - du könntest sie natürlich ändern, aber in unserem Fall passt es so. Jede Zahl soll eine andere Display-Ansicht aufrufen. Falls du noch unseren bisherigen Wetterstation-Aufbau hast aus **OLED Print** und den drei Messwerten, dann kannst du ihn nun in den **case** mit der Variable 0 anlegen. Ansonsten baue ihn dir schnell neu.

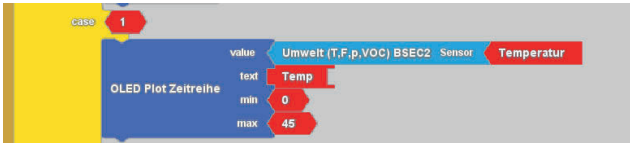


Das ist unsere erste Auswahlmöglichkeit (case).

Die drei weiteren Ansichten (**case 1**, **2** und **3**) bauen wir etwas anders auf. Hier sollen ja unsere Diagramme erscheinen, die den zeitlichen Verlauf unserer Werte anzeigen. Der richtige Block dafür ist die **OLED Plot Zeitreihe**, die du unter *Anzeigen / Aktoren* findest. Sie erstellt das Diagramm und wir müssen nur noch einstellen, welche Daten wir dort anzeigen lassen wollen.

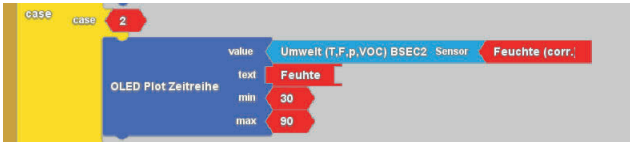
Wir fangen mit der der Temperatur als **case 1** an. Hole dir den Wert also wie gehabt mit dem **Umweltsensor**-Block und lege ihn an **Value** des **Zeitreihe**-Blocks. **value** steht für Wert - hier wird also bestimmt, was du anzeigen willst. Du kannst für die spätere Bildschirmausgabe noch eine Beschriftung wählen. Dafür liegt bereits ein roter Platzhalter an *Text*. Was dort steht, kannst du wieder frei

wählen. Da wir die Temperatur anzeigen wäre beispielsweise eine einfach Form wie *Temp* gut. Dann wird noch der minimale und maximale Wert eingestellt, den du anzeigen lassen willst - das kennst du bereits vom *Encoder (turn)* aus dem Setup. Die Werte 0 und 45 zeigen also einen Bereich von 0 bis 45 Grad Celsius an, das ist ein guter Startpunkt.



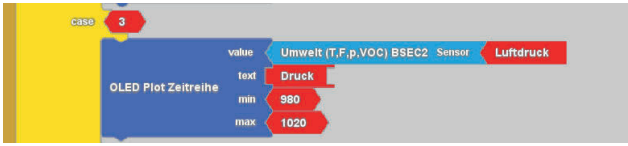
Mit min- und max-Werten bestimmst du, was angezeigt wird.

Unter **case 2** legst du alles genau gleich an, wählst aber *Feuchte (corr.)*. Die Beschriftung kannst du wieder abkürzen, vielleicht mit *Feuchte*. Der Wertebereich könnte zwischen 30 und 90 Grad liegen - hier kannst du aber gerne auch selbst einen anderen Bereich ausprobieren. Angezeigt wird die prozentuale Luftfeuchtigkeit.



Andere Sensorwerte, brauchen auch eigene Wertebereiche.

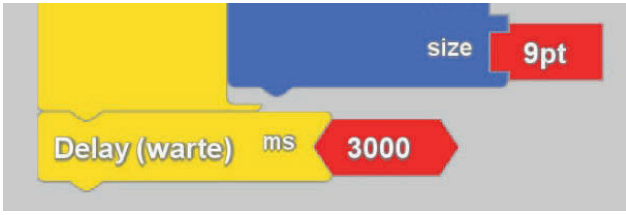
case 3 zeigt dann den Luftdruck (p) im Wertebereich von 980 bis 1020. Das sollte bereits einen guten Bereich abdecken. (Vergleiche das mal mit der Tabelle oben für den normalen Bereich.)



Hier lesen wir den Luftdruck aus.

Jetzt sind wir fast fertig: Am Ende hängen wir noch eine kleine Zeitverzögerung ein – mit dem **Delay (warte)**-Block – und stellen ihn auf *3000 ms* (Millisekunden, also 3 Sekunden). Diese Zeit beeinflusst, wie oft die Messung stattfindet und in den Zeitverlauf eingetragen wird. Genauer gesagt verlangsamt das Delay unsere Programm-Schleife, indem es eine kurze Pause einlegt. So können wir steuern, wann der nächste Durchlauf beginnt.

Würden wir diese Pause weglassen, würde die Schleife blitzschnell immer wieder durchlaufen – die Messdaten kämen viel zu schnell.



Die Verzögerung (Delay) wird in Millisekunden angegeben.

Mit diesem Aufbau kannst du auch über einen längeren Zeitraum messen. Übertrage dein Programm auf Makey:Lab und probiere es am besten gleich mal aus!

Später schauen wir uns noch an, wie du dein Makey:Lab ans Internet anbindest. Dann kannst du beispielsweise sehen, wie das Wetter bei dir zuhause ist, auch wenn du gerade unterwegs bist!

Bevor wir ein neues Projekt angehen, wollen wir noch einer spannender Frage nachgehen: **Wie wird das Wetter?**

Blick in die Wetter-Glaskugel

Makey:Lab kann dir auch helfen, eine Wettervorhersage zu treffen. Das ist vielleicht nicht so genau wie von einem Meteorologen - aber eine grobe Einschätzung anhand deiner Messdaten kannst du trotzdem bekommen. Wie das geht schauen wir uns jetzt an.

Öffne ein neues Sketch. Wenn du deine bisherige Wetterstation behalten willst, speichere sie vorher ab.

Wetterdaten vergleichen

Um einen Wettertrend zu erkennen werden wir jetzt zwei Sensordaten nutzen, die uns viel über Wetterveränderungen verraten: **Luftdruck** und **Luftfeuchtigkeit**.

Dann vergleichen wir die aktuellen Werte mit denen von vor 15 Minuten (oder einem längeren Zeitraum). Wenn sich etwas verändert hat - zum Beispiel der Druck stark

fällt - gibt Makey:Lab eine kleine Vorhersage auf dem Display aus.

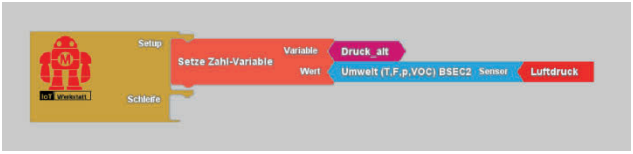
Fangen wir an!

Die aktuellen Werte von Luftdruck und Luftfeuchtigkeit bekommen wir direkt aus dem Sensor, unsere Vergleichszahlen müssen wir erst sammeln.

Wenn du Daten vergleichen willst, dann musst du sie in einer *Variable* speichern - damit Makey:Lab sie nicht wieder vergisst. (Denk an die Schubladen!)

Dafür setzt du zuerst die beiden Variablen *Druck_alt* und *Feuchte_alt*. Zieh dir dafür den Block **Setze Zahl-Variable** aus dem Reiter *Variablen / Konstanten* in den Arbeitsbereich und hänge ihn an den Startblock unter Setup an.

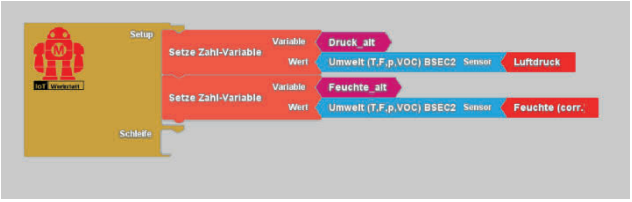
Der Block hat bereits zwei Platzhalter, die du nun ändern kannst. Damit bestimmst du, wie die Variable heißen soll (lila Block) und welchen Wert sie am Anfang hat (roter Block), wenn das Programm startet.



Hier bekommt eine Variable Daten vom Umweltsensor.

Vergib den Namen *Druck_alt*. Als Startwert nehmen wir keine feste Zahl, stattdessen soll hier direkt der passende Sensorwert eingefügt werden. Du löscht also den roten Zahlenblock (nach links auf die Blockliste ziehen) und holst dir aus dem Reiter *Sensoren* den Block: **Umwelt (T, F, p, Voc) BSEC2** und lässt ihn unter Wert einrasten. Im Parameter-Block, der bestimmt, welchen Wert dir der Umweltsensor ausgeben soll, stellst du dieses Mal Luftdruck ein.

Nun brauchst du noch die zweite Variable *Feuchte_alt*. Die legst du genauso an wie die vorherige, änderst den Namen und stellst als Sensorwert *Feuchte (corr.)* ein.



Eine zweite Variable wird gesetzt.

Dein Setup ist fertig. Das sind unsere alten Vergleichs-Werte mit denen wir in die Programmschleife starten. Sie werden mit den neuen Werten verglichen und dann überschrieben. So sind sie beim nächsten Durchlauf unsere "neuen alten Werte".

Unsere Schleife startet damit, dass wir uns auch noch die neuen Werte hinzu holen. Du legst also noch zwei Variablen an. Nenne sie einfach *Druck_neu* und

Feuchte_neu. Die beiden **Setze Zahl-Variable**-Blöcke hierfür legst du dieses Mal aber unter Schleife ab. Ansonsten werden sie gleich aufgebaut.

Wir legen Variablen für alte und neue Werte an.

Verschachtelte Bedingungen

In unserem Programm nutzen wir als Nächstes verschachtelte **Falls/sonst**-Blöcke, das heißt: eine Abfrage liegt innerhalb einer anderen. Warum machen wir das so? Ganz einfach: Wir wollen sicherstellen, dass immer nur eine Wettermeldung angezeigt wird – und zwar die, die am besten zur aktuellen Lage passt.

Statt viele einzelne Bedingungen nacheinander zu prüfen, fragen wir zuerst: Wird das Wetter schlechter? Wenn ja, zeigen wir das sofort an – und überspringen den Rest. Nur wenn das nicht zutrifft, prüfen wir, ob es besser wird. Wenn auch das nicht passt, bleibt nur noch **Wetter stabil**. So wird der Ablauf klar geregelt – und Makey:Lab trifft genau eine Entscheidung.

Lege unter die beiden Variablen-Blöcke einen **Falls/sonst**-Block ab, der befindet sich unter *Kontrollstrukturen*. Nun kümmern wir uns um die erste Bedingung.

Mit dem Falls/sonst-Block überprüfen wir die Werte.

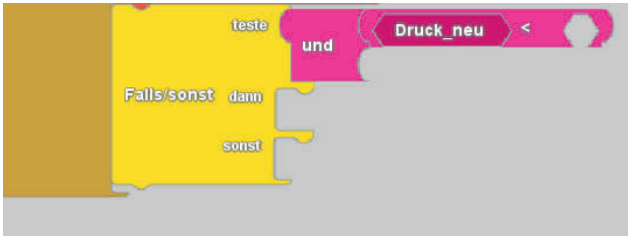
Das Wetter wird schlechter

Wir wollen einen schnellen Trend erkennen. Dafür nutzen wir ein paar Grenzwerte. Sind diese über- oder unterschritten, können wir eine Aktion auslösen.

Wenn der Druck fällt, dann deutet das daraufhin, dass das Wetter schlechter wird. Wir wollen aber auch schauen, ob die Luftfeuchtigkeit zunimmt. Ist das der Fall, dann könnte es regnen!

Hol dir aus *Logische Vergleiche* den Block **und**. Diesen schiebst du an den **Falls/sonst**-Block an *teste*. Unter dem gleichen Reiter findest du auch den Vergleiche-Block mit einem kleiner-als-Symbol < in der Mitte. Das wird unsere

erste Abfrage. Sie muss mit zwei Variablen gefüllt werden. In die erste legst du **Druck_neu**.



Mit "und" setzen wir weitere Bedinungen.

Tipp: Du kannst die Variable oben mit Rechtsklick und klonen verdoppeln.

Für die zweite Variable brauchen wir einen kleinen Aufbau, damit wir unseren Grenzwert einstellen können. Dieser soll der alte Wert minus 2 sein. Es wird also geprüft, ob der neue Wert um mehr als zwei gefallen ist. Das wäre ein Hinweis darauf, dass der Druck fällt und das Wetter schlechter wird.

Lege also den **Minus**-Block aus *Math. Operationen* in das zweit Variablen-Feld unseres Vergleichs. Hier kommt nun die Variable *Druck_alt* hinein und auf der anderen Seite die ganze Zahl 2. Den entsprechenden (roten) Zahlenblock findest du unter *Variablen / Konstanten*. Gib hier einfach 2 ein - da muss kein Minus davor, das macht schon der Block. Den gesamten logischen Vergleich legst du nun an den oberen runden Anschluss des **und**-Blocks an.

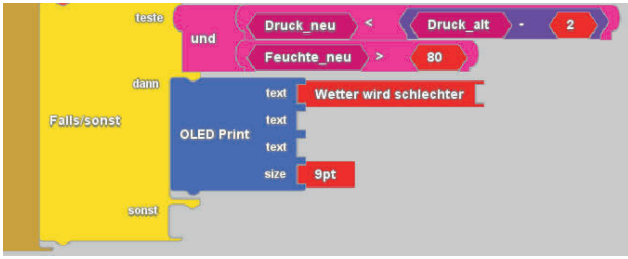
Eine kleine Rechnung hilft uns beim Vergleich.

Merke: Eine **und**-Bedingung brauchst du immer dann, wenn nicht nur eine Sache zutreffen soll – sondern mehrere gleichzeitig.

Jetzt brauchen wir noch die zweite Abfrage, diese befasst sich mit der Luftfeuchtigkeit. Ist sie über 80 Prozent, dann könnte das auf Regen hindeuten. Hier nimmst du dieses Mal den **größer-als**-Block >. In das erste Feld kommt die Variable *Feuchte_neu* und in das zweite Feld die Zahl 80.

Nun schauen wir noch auf die Feuchte.

Was passiert, wenn beide Bedingungen erfüllt sind, dass entscheiden wir jetzt. Der **OLED Print**-Block soll uns eine Text-Nachricht auf dem Display von Makey:Lab ausgeben. Hol dir also den Block aus *Anzeigen / Aktoren* und leg ihn am **Falls/sonst**-Block unter *dann* ab. Den Text-Platzhalter ersetzt du mit einer Nachricht deiner Wahl.
Beispielsweise: **Wetter wird schlechter**



Der Text wird direkt auf dem Display ausgegeben.

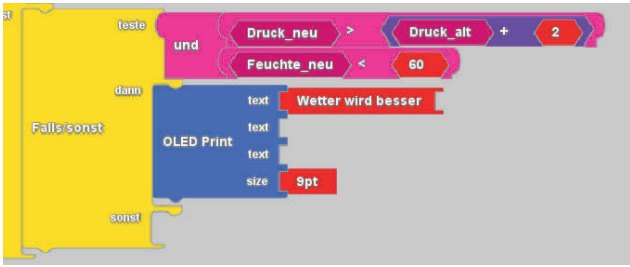
Jetzt wird's verschachtelt! Das bedeutet: Wir stecken einen **Falls/sonst**-Block in einen anderen hinein – so können wir mehrere Bedingungen der Reihe nach prüfen.

In unserem Fall: Falls das Wetter nicht schlechter wird, brauchen wir eine zweite Abfrage. Die platzierst du unter *sonst* im ersten Block.

Dort baust du wieder eine und-Bedingung: Dieses Mal prüfst du, ob der neue Druck mindestens zwei höher ist als der alte – und ob die Luftfeuchtigkeit unter 60 % liegt.

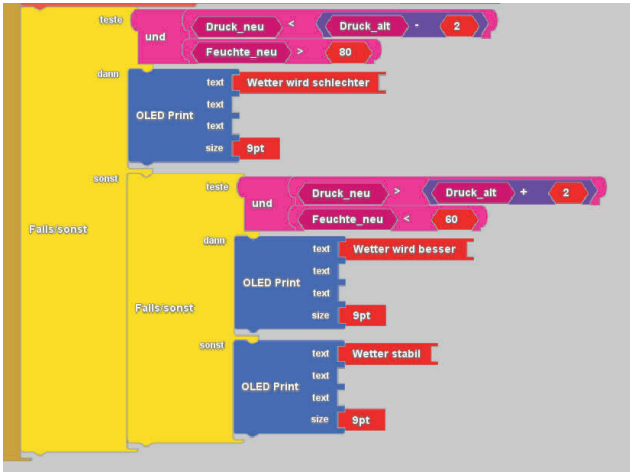
Nutze dafür die gleiche Blockstruktur wie beim ersten Mal, du kannst sie auch einfach klonen. Die Mathe-Blöcke kann man übrigens ganz einfach umwandeln. Dafür musst du nur auf das Symbol (zum Beispiel: <) klicken und es dann ändern.

Sollten die beiden Bedingungen zutreffen, kannst du dieses Mal im **OLED Print**-Block als Text **Wetter wird besser** einsetzen.



Nun wollen wir uns das gute Wetter anschauen.

Für den letzten Fall müssen wir keine Bedingungen mehr abfragen, denn er wird automatisch ausgewählt, wenn das Wetter nicht schlechter oder besser wird.



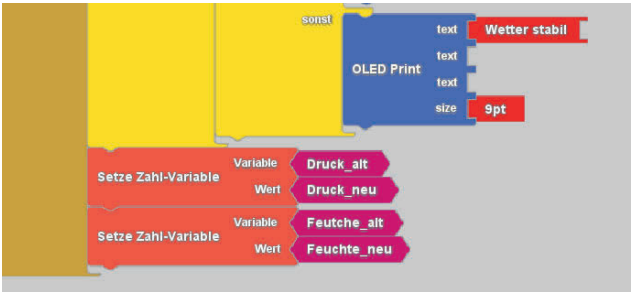
Nun ist das Programm schon recht umfangreich.

Unter das letzte *sonst* legst du also einfach einen dritten **OLED Print**-Block. Hier kannst du etwas schreiben wie: **Wetter stabil**

Jetzt bist du fast fertig. Es fehlt nur noch eine Kleinigkeit, damit unsere Schleife komplett ist. Du musst nach der Prüfung nun die alten Variablen mit den neuen Überschreiben, damit du sie beim nächsten Mal wieder vergleichen kannst.

Nutze dafür wieder jeweils den **Setze Zahl-Variable**-Block unter die anderen Blöcke. Die Variable *Druck_alt* erhält nun den Wert *Druck_neu.*

Ein zweiter **Setze Zahl-Variable**-Block überschreibt die *Feuchte_alt* mit *Feuchte_neu*.



Hier schreiben wir die neuen Werte über die alten.

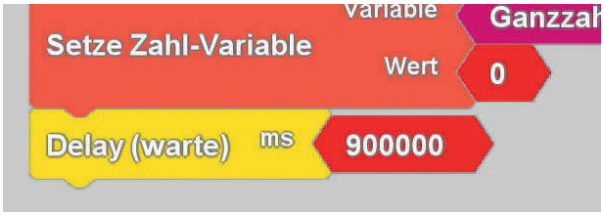
Wie oft wird gemessen?

Das Wetter wechselt ständig leicht. Uns interessiert aber nur, wenn es sich stark verändert. Dies geschieht meist über einen längeren Zeitraum. Anders als bei unserer bisherigen Wettermessstation ist es also sinnvoll, die Messabstände etwas länger zu setzen. Für den Anfang und damit du nicht zu lange warten musst, kannst du die

Messung alle 15 Minuten vornehmen. Später ist es aber sinnvoll den Wert auf beispielsweise 60 Minuten zu setzen. Das ist genauer, weil sich in dieser Zeit ein Trend deutlicher abzeichnet.

Um den Zeitabstand zwischen den Messungen einzustellen legst du abschließend als letzten Block in deine Programmschleife den **Delay (warte)**-Block. Diesen findest du unter *Kontrollstrukturen*. An ihm kannst du die Wartezeit in Millisekunden (ms) angeben.

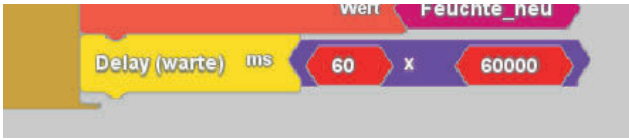
Merke: 1.000 Millisekunden sind 1 Sekunde. 15 Minuten wären 900.000 Millisekunden (15 × 60 × 1000). Eine Stunde sind dann schon 3.600.000 ms (60 × 60 × 1000).



Bei diesen hohen Zahlen muss man genau hinsehen beim Tippen.

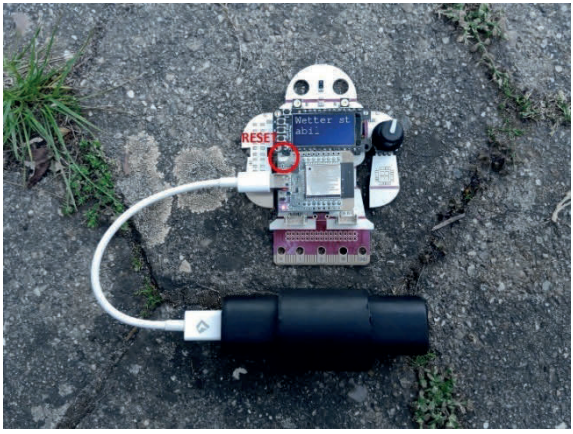
Du könntest diesen Wert einfach in deinem **Delay**-Block eintragen. Damit es aber etwas leichter zu handhaben ist und damit wir uns nicht verrechnen, können wir auch die IoT-Werkstatt für uns rechnen lassen. Statt einer Zahl legst du den Mathe-Block zum Multiplizieren (*x*) an. Ins zweite Feld kommt ein roter Zahlen-Block mit *60000*. Das ist genau eine Minute. In das erste Feld legst du jetzt einen weiteren Zahlen-Block. Hier kannst du jetzt einfach die

Anzahl an Minuten ein, die die Schleife warten soll. Also beispielsweise *15* oder *60*.



Auf diese Weise bestimmen wir, wie oft gemessen wird.

Jetzt ist dein **Wetter-Trend-Messer** fertig und du kannst dir vom Makey:Lab anzeigen lassen, wie das Wetter wird. Übertrage dein Programm und dann ab nach draußen! Du solltest aber darauf achten, dein Makey:Lab regengeschützt aufzustellen - Wasser mag es nicht.



Der kleine Reset-Knopf kann beim Ortswechsel sehr hilfreich sein.

Hinweis: Versorge das Makey:Lab erst mit Strom, wenn du draußen bist. Dann wird nicht versehentlich das Innen-Wetter gemessen. Oder: Du kannst die Messung jederzeit neu starten - dazu ziehst du das Makey:Lab kurz vom

Strom und steckst es wieder an oder - noch einfacher - du drückst den Reset-Knopf.

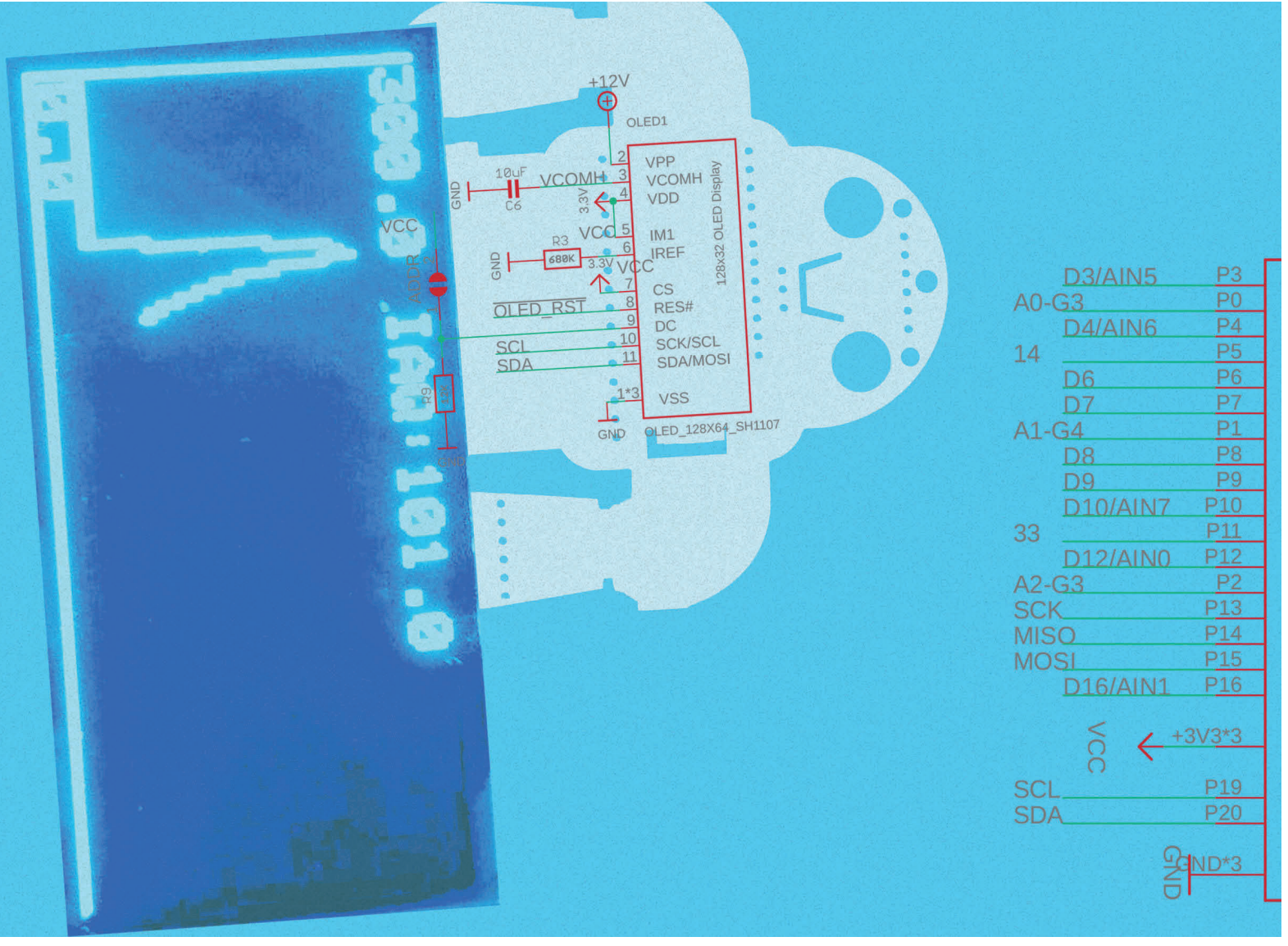
das wird richtig gut

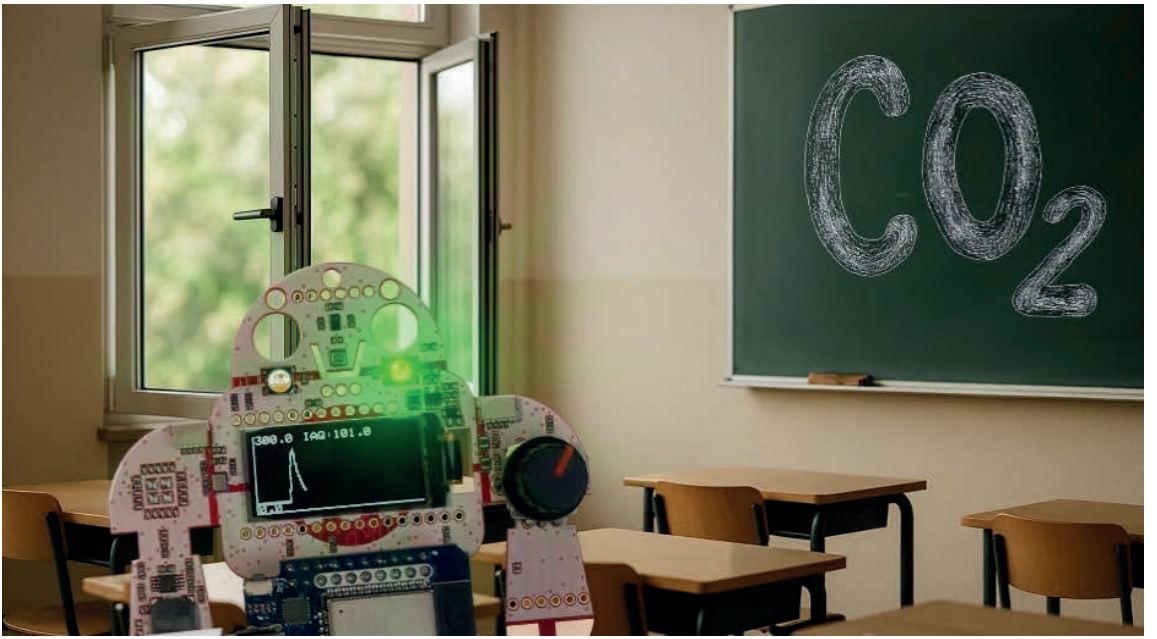
Du willst noch mehr wissen?

SCAN MICH

Hinter dem QR-Code findest du weitere Projekte vom Umwelt-Campus Birkenfeld rund um Wetter, Klima und Umwelt.

Forschende nutzen Wetterdaten, um Ökosysteme besser zu verstehen – und erforschen, warum das Wetter nicht nur das Eichhörnchen, sondern auch die Energiewende beeinflusst. Entdecke Ideen zum Wetter und informiere dich über den Studiengang **Erneuerbare Energien (B.Sc.)** am Umwelt-Campus Birkenfeld.


+12V
OLED1
VPP
VCOMH
VDD
IM1
IREF
CS
RES#
DC
SCK/SCL
SDA/MOSI
VSS
128x32 OLED Display
OLED_128X64_SH1107
10uF
C6
VCOMH
3.3V
VCC
R3
680K
OLED_RST
SCL
SDA
R9
0R
ADDR
GND
D3/AIN5 P3
A0-G3 P0
D4/AIN6 P4
14 P5
D6 P6
D7 P7
A1-G4 P1
D8 P8
D9 P9
D10/AIN7 P10
33 P11
D12/AIN0 P12
A2-G3 P2
SCK P13
MISO P14
MOSI P15
D16/AIN1 P16
VCC
+3V3*3
SCL P19
SDA P20
GND
GND*3



#02 Frische Luft fürs Gehirn

Damit merkst du, wann's Zeit zum Lüften ist

Kennst du das? Du sitzt im Unterricht oder arbeitest konzentriert am Schreibtisch – und plötzlich fühlst du dich irgendwie schlapp. Deine Gedanken schweifen ab, deine Augen werden schwer, und du fragst dich: „Warum bin ich so müde? Ich hab doch genug geschlafen …?!"

Die Antwort steckt (meistens) nicht in deinem Kopf – sondern in der Luft!

Wenn viele Menschen in einem Raum sind und die Fenster lange geschlossen bleiben, verbraucht sich die Luft. Genauer gesagt: Der Sauerstoff wird weniger – und das unsichtbare Gas CO_2 nimmt zu. Je mehr wir ausatmen, desto voller wird der Raum mit „gebrauchter Luft".

Irgendwann merkt das auch dein Gehirn: Es bekommt zu wenig frischen Nachschub.

Was passiert bei schlechter Luft?

Wenn der CO_2-Gehalt in der Luft steigt …

- werden wir schneller müde
- können wir uns schlechter konzentrieren
- bekommen manche sogar Kopfschmerzen

Aber: Wann ist der richtige Zeitpunkt zum Lüften? Und woher weiß man, wann es wirklich nötig ist?

Erinnerst du dich noch an die Corona-Zeit? Vielleicht hast du damals im Klassenzimmer mit Schal und Mütze gesessen, weil alle paar Minuten die Fenster aufgerissen wurden. Das war kein Spaß – aber wichtig. Denn frische Luft hilft nicht nur beim Denken, sondern auch beim Schutz vor Krankheiten.

Wenn viele Menschen zusammen in einem Raum sind, dann können sich winzige Tröpfchen in der Luft (sogenannte Aerosole) sammeln – und die enthalten manchmal Viren. Die gute Nachricht: Lüften pustet sie einfach raus!

Heute ist das zum Glück nicht mehr so dramatisch wie damals – aber: Gute Luft bleibt wichtig, ob mit oder ohne Pandemie.

Aber mit Makey:Lab kannst du sogar selbst messen, wann das Lüften wirklich nötig ist – und wann du ruhig mal das Fenster zulassen darfst.

Makey misst die Luftqualität

Du hast in den letzten Kapiteln schon mit dem BME680 gearbeitet – unserem Alleskönner für Temperatur, Feuchte, Druck ...

Aber dieser Sensor kann noch mehr!

Er hat einen kleinen Gassensor eingebaut, der flüchtige Stoffe in der Luft erkennt – sogenannte VOC (Volatile Organic Compounds). Das sind winzige Moleküle, die beim Atmen, Kochen, Duschen, Putzen oder auch beim Öffnen einer Tüte Chips in die Luft gelangen.

Wenn viele VOCs in der Luft sind, ist das ein Zeichen für „verbrauchte Luft“ – ähnlich wie bei CO_2. Der Sensor kann zwar kein CO_2 direkt messen, aber er kann anzeigen, wenn es stickig wird.

Der IAQ-Wert – eine Zahl für gute Luft

Vielleicht hast du schon mal eine CO_2-Ampel gesehen, die eine Zahl mit *ppm* anzeigt. Das heißt *parts per million* – also wie viele Moleküle pro Million gerade CO_2 sind. Frische Luft hat etwa 400 ppm, bei stickiger Luft im Klassenzimmer können es über 1500 ppm werden. Dann wird's höchste Zeit zu lüften.

CO_2-Wert (ppm)	**Bedeutung**
400–800	Frische Luft
800–1000	Noch okay
1000–1500	Wird stickig
über 1500	Lüften – jetzt!

Der IAQ-Wert funktioniert ganz ähnlich - auch wenn die Zahl anders aussieht. Aber auch hier gilt: **je höher der Wert, desto schlechter die Luft!**

IAQ-Wert	**Bedeutung**
0–50	Sehr gute Luft
51–100	Alles noch im grünen Bereich
101–150	Wird kritisch
151–200	Jetzt lüften!
über 200	**Alarm – Fenster auf!**

Kurz gesagt: Der IAQ-Wert ist wie ein „gefühlter CO_2-Wert“ – kein Laborwert, aber genau richtig, damit du weißt: Fenster auf oder zu?

Projekt: Bau dir deine eigene Lüftungsampel!

Alles, was du für unser Beispielprojekt brauchst, hast du mit Makey:Lab schon.

In unserem Programm liest du den IAQ-Wert aus und entscheidest mit einem **Falls/sonst**-Block, welche Farbe deine LED zeigen soll.

So weißt du (und deine Klasse), wann frische Luft nötig ist – ganz ohne Nase.

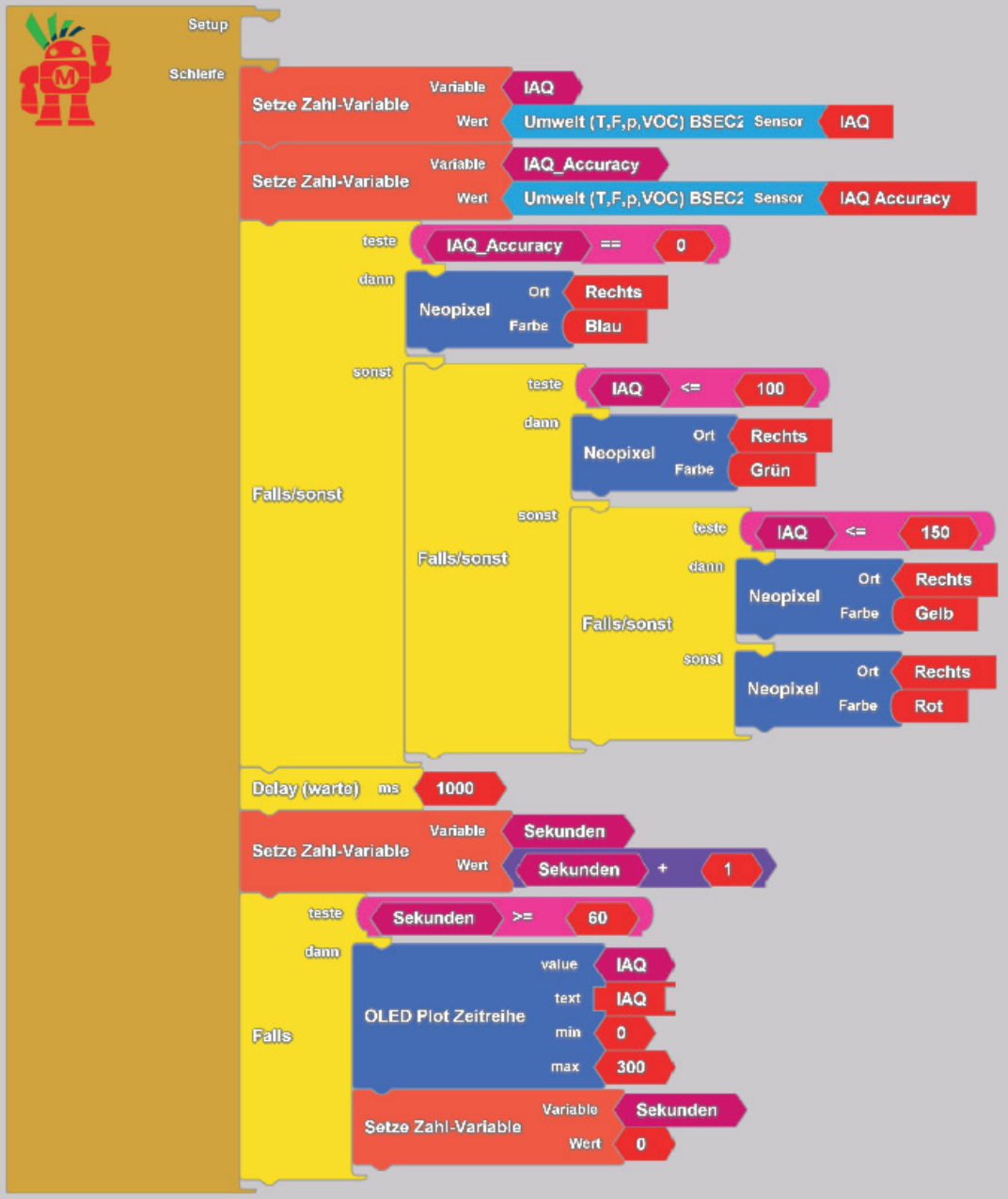
So funktioniert deine Luft-Ampel im Detail

Jetzt geht’s ans Eingemachte! Wir schauen uns an, wie genau dein Makey-Programm arbeitet, wie es die Luftqualität bewertet – und wie daraus schließlich eine bunte LED-Ampel entsteht.

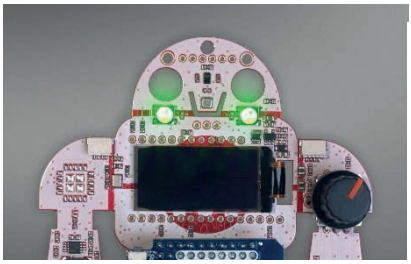
Der Sensor lernt erstmal dazu: Gleich zu Beginn startet dein Programm den BME680-Sensor und versetzt ihn in den sogenannten Trainingsmodus.

Warum das wichtig ist? Der Sensor braucht ein bisschen Zeit, um sich an seine Umgebung zu gewöhnen. Das kann auch mal ein paar Minuten dauern.

so sieht's fertig aus



In dieser „Lernphase“ ist der IAQ Accuracy-Wert (also die Genauigkeit des Luftwerts) noch 0. Solange das so ist, macht die Ampel erstmal nichts, außer blau zu leuchten.



Makey:Lab hat zwei RGBW-LEDs an Board.

Das bedeutet: "Ich bin noch nicht soweit." Du musst dann nur noch ein bisschen geduldig warten.

Was ist eine LED? Eine LED ist eine winzige Leuchtdiode, die besonders wenig Strom verbraucht und sehr langlebig ist. Man findet sie in Taschenlampen, Bildschirmen, Fahrradleuchten – und natürlich in Maker-Projekten!

Auf unserem Board steckt eine RGBW-LED. Sie kann Rot, Grün und Blau mischen – und hat zusätzlich eine echte weiße LED eingebaut. Dadurch wirken die Farben kräftiger, und Weiß sieht richtig sauber aus.

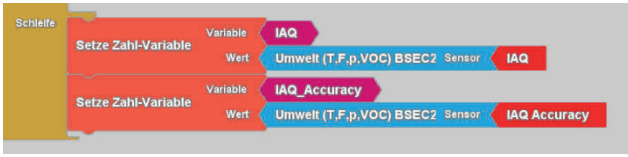
Wir steuern diese LED mit einem speziellen Programmblock namens NeoPixel (den findest du unter dem Reiter *Anzeigen / Aktoren*). „NeoPixel“ ist ein Standard, der von der Firma Adafruit bekannt gemacht wurde. Unsere LED ist NeoPixel-kompatibel – deshalb klappt das wunderbar!

Zwei Variablen für die Luft

Damit du mit dem IAQ-Wert weiterarbeiten kannst, brauchst du zwei Zahl-Variablen:

- *IAQ* – für den eigentlichen Luftwert
- *IAQ_Accuracy* – für die Genauigkeit des Werts

Ziehe zwei **Setze Zahl-Variable**-Blöcke in deinen Code und platziere sie ganz oben im Hauptprogramm (also unter *Schleife*).



Die beiden Variablen werden gesetzt.

Tipp: So etwas nennen wir eine Initialisierung – wir sagen dem Computer: "Hey, hier ist eine neue Zahl, merk dir den Wert!" (Ab in die Schublade!)

Die verschachtelte Luftprüfung

Jetzt wird's spannend: Du baust eine **Falls/sonst**-Abfrage, die prüft, ob *IAQ_Accuracy* noch 0 ist.

Wenn ja, bedeutet das: Der Sensor lernt noch. Dann wird einfach die LED auf Blau gesetzt.

Der IAQ Accuracy-Wert:

Wert	Bedeutung
0	Sensor lernt noch – die Werte sind ungenau.
1	Grobe Orientierung – kann man schon nutzen.
2	Schon recht zuverlässig – Werte sind brauchbar.
3	Voll kalibriert – Sensor kennt die Umgebung gut!

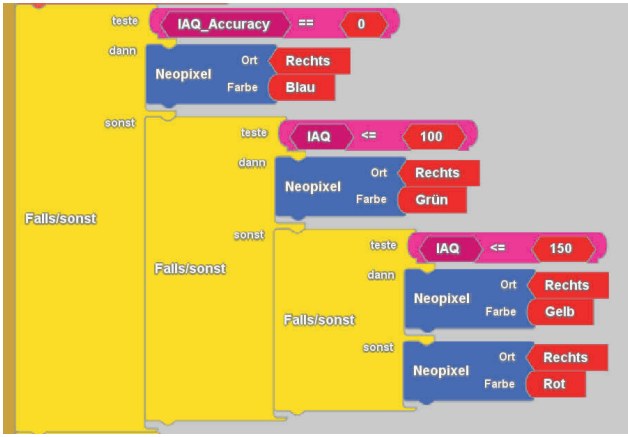
Erst wenn die Genauigkeit (*Accuracy*) mindestens 1 erreicht, läuft das Programm weiter und bewertet die Luftqualität anhand des IAQ-Werts.

Dafür verwenden wir eine verschachtelte Falls/Sonst-Struktur, die den IAQ in drei Stufen einteilt:

Falls der IAQ kleiner oder gleich 100 ist, zeigt die Ampel grün – die Luft ist gut.

Sonst, wenn der IAQ kleiner oder gleich 150 ist, wird die LED gelb – die Luft ist mittel.

In allen anderen Fällen wird die LED rot – die Luftqualität ist schlecht.

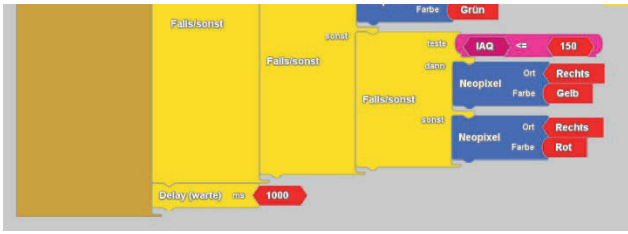


Unsere Ampel: die Blöcke sind verschachtelt aufgebaut.

So wird aus einem Zahlenwert eine automatische Frischluft-Ampel, die sofort zeigt, wann es Zeit zum Lüften ist.

Ein kurzer Stopp: delay()

Unten im Block findest du den Befehl **Delay (warte)** *1000 ms.*



Am Ende steht wieder ein Delay.

Das sorgt dafür, dass dein Programm nicht zu schnell läuft, sondern genau einmal pro Sekunde alles prüft.

Ohne diese kleine Pause würde Makey:Lab den Code so schnell durchrattern wie es kann - viel zu schnell! Das könnten locker Hunderte Male pro Sekunde sein.

Das wäre weder für den Sensor sinnvoll, noch für die Darstellung der Werte.

Ein Diagramm mit Verlauf

Jetzt machen wir noch etwas Schönes fürs Auge: Ein Verlaufsdiagramm auf dem Display!

Dazu nutzt du den Block: **OLED Plot Zeitreihe**

Mit diesen Werten zeigt das Display den richtigen Wertebereich.

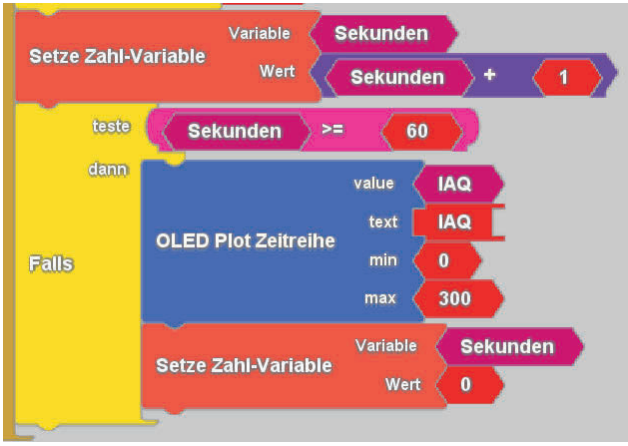
Als Wert gibst du einfach die Variable *IAQ* an.

Dazu definierst du den Zahlenbereich, den du anzeigen lassen möchtest – z. B. von 0 bis 300. Das sollte der alltäglichen Umgebung entsprechen. Ansonsten kannst du die Werte später noch anpassen.

Ein Timer für die Anzeige

Damit nicht bei jedem Durchlauf ein Punkt gesetzt wird, brauchst du eine kleine Stoppuhr – einen Timer:

- Erstelle eine neue Zahl-Variable namens *Sekunden*.
- Bei jedem Schleifendurchlauf (also jede Sekunde) wird *Sekunden* um +1 erhöht.
- Danach prüft dein Programm: Ist *Sekunden* größer als 60? **Ja**: Dann wird ein neuer Punkt im Diagramm gesetzt – und *Sekunden* wird wieder auf 0 zurückgesetzt.



Der fertige Timer. Am Ende wird die Zeit wieder auf 0 gesetzt.

Fertig, nun zeigt deine LED die Luftqualität – und das Diagramm auf dem Display den Verlauf über die Zeit. So wirst du zum echten Luft-Detektiv!

Es geht noch mehr! Wenn du magst, kannst du deine Lüftungsampel später noch erweitern:

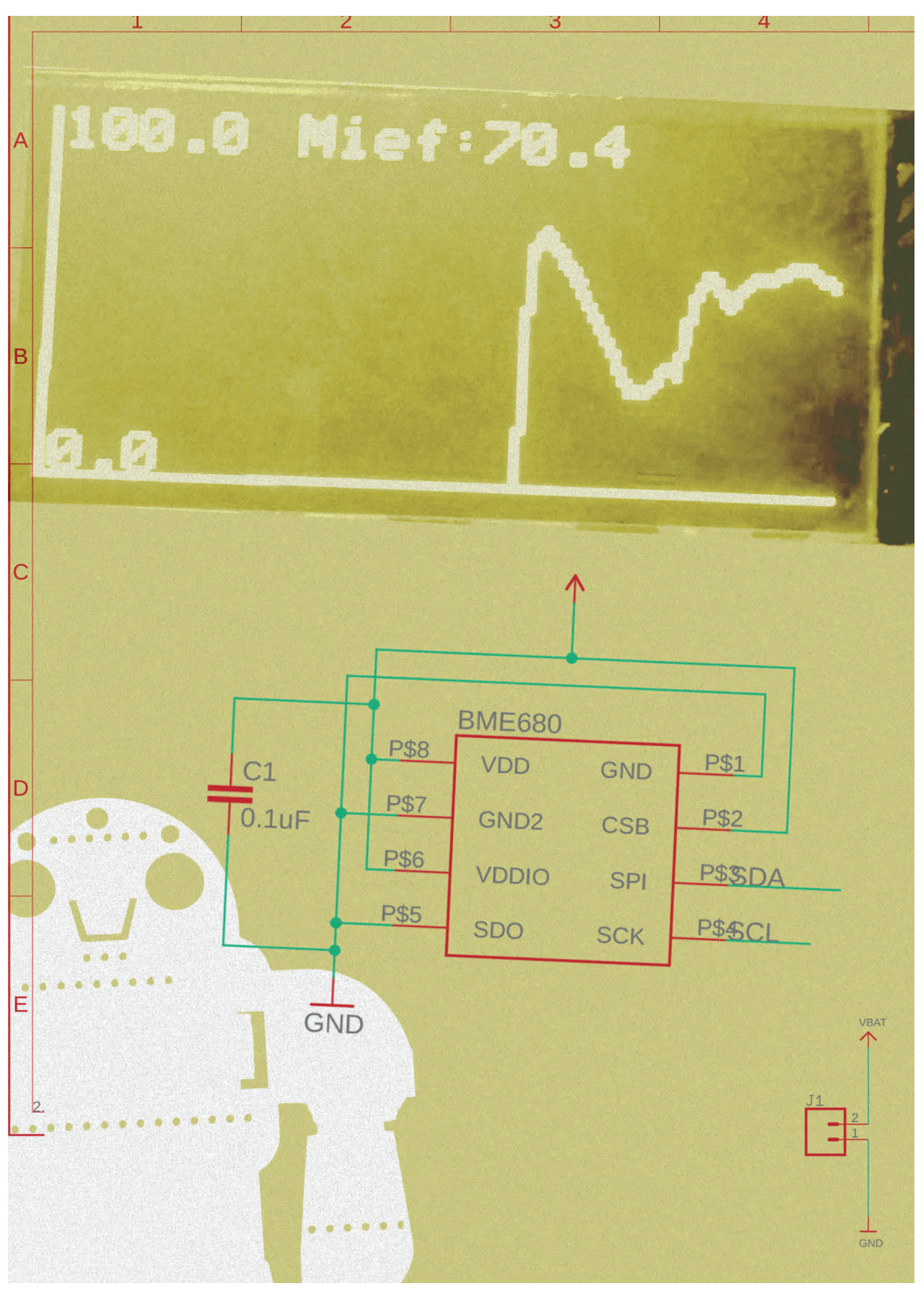
Im Kapitel *#10 Makey:Lab nutzt WLAN* lernst du, wie dein Makey:Lab eine eigene Webseite im WLAN anzeigen kann – ganz ohne Internet. Perfekt für Zuhause oder das Klassenzimmer, damit alle darauf zugreifen können.

Du willst noch mehr wissen?

SCAN MICH

Hinter dem QR-Code findest du weiterführende Infos vom Umwelt-Campus Birkenfeld – zu Luftqualität (IAQ), Aerosolen und Projekten wie Haustechnik im Hörsaal oder der Corona-Ampel.

-


100.0 Mief:70.4
0.0
BME680
VDD
GND
GND2
CSB
VDDIO
SPI
SDO
SCK
P$8
P$7
P$6
P$5
P$1
P$2
P$3
P$4
SDA
SCL
C1
0.1uF
GND
VBAT
J1
2
1
GND

#03 Makey:Lab schnuppert am Obstkorb

Wie reif ist die Banane?

Wusstest du? Dein Makey:Lab kann auch riechen. Nicht wie mit einer echten Nase – aber es kann Gase erkennen, die in der Luft schweben. Diese Gase nennt man VOCs, das steht für *Volatile Organic Compounds*. Dazu gehören z. B. Aromen, Ausdünstungen oder bestimmte Geruchsstoffe – und die entstehen überall in deinem Alltag:

- Wenn du ein frisches Brot aufschneidest.
- Wenn Nagellack trocknet.
- Oder wenn eine Banane langsam braun wird.

Gerade bei Obst wird es spannend: Wenn Früchte reifen, geben sie ganz bestimmte Gase ab – zum Beispiel Ethanol, also eine Form von Alkohol. Je reifer die Frucht,

desto stärker der Geruch. Das merkt auch deine Nase. Aber: Makey:Lab kann es sogar noch schneller bemerken.

Mit dem BME680-Sensor auf deinem Board kann Makey:Lab diese Gase erkennen – und damit „riechen", ob z. B. eine Banane noch unreif ist oder schon überreif.

Wie funktioniert das?

Der Sensor enthält ein winziges Heizelement, das sich im Inneren stark aufheizt – manchmal bis zu 450 °C. Aber keine Sorge: außen bleibt alles ungefährlich.

Je nachdem, wie heiß der Sensor ist, reagiert er unterschiedlich auf verschiedene Gase. Das ist ein bisschen wie bei einem Spürhund, der an mehreren Stellen schnüffelt, um Hinweise zu finden. So entsteht ein Geruchsprofil, das Rückschlüsse auf die Umgebung zulässt.

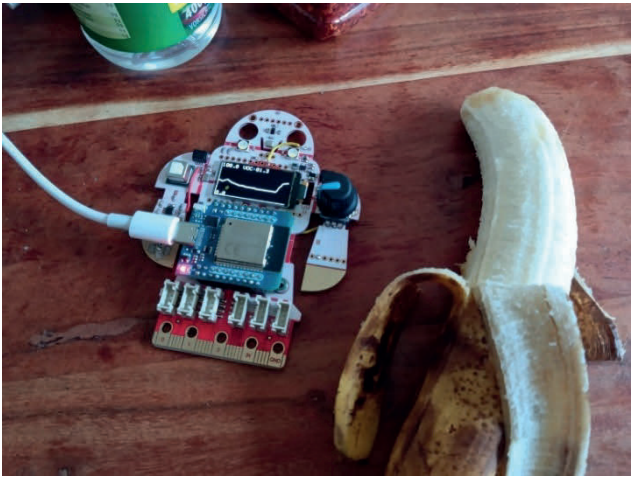
Vorbereitung

Bevor du dein Obst analysierst, solltest du sehen, wie Makey:Lab überhaupt auf Gerüche reagiert.

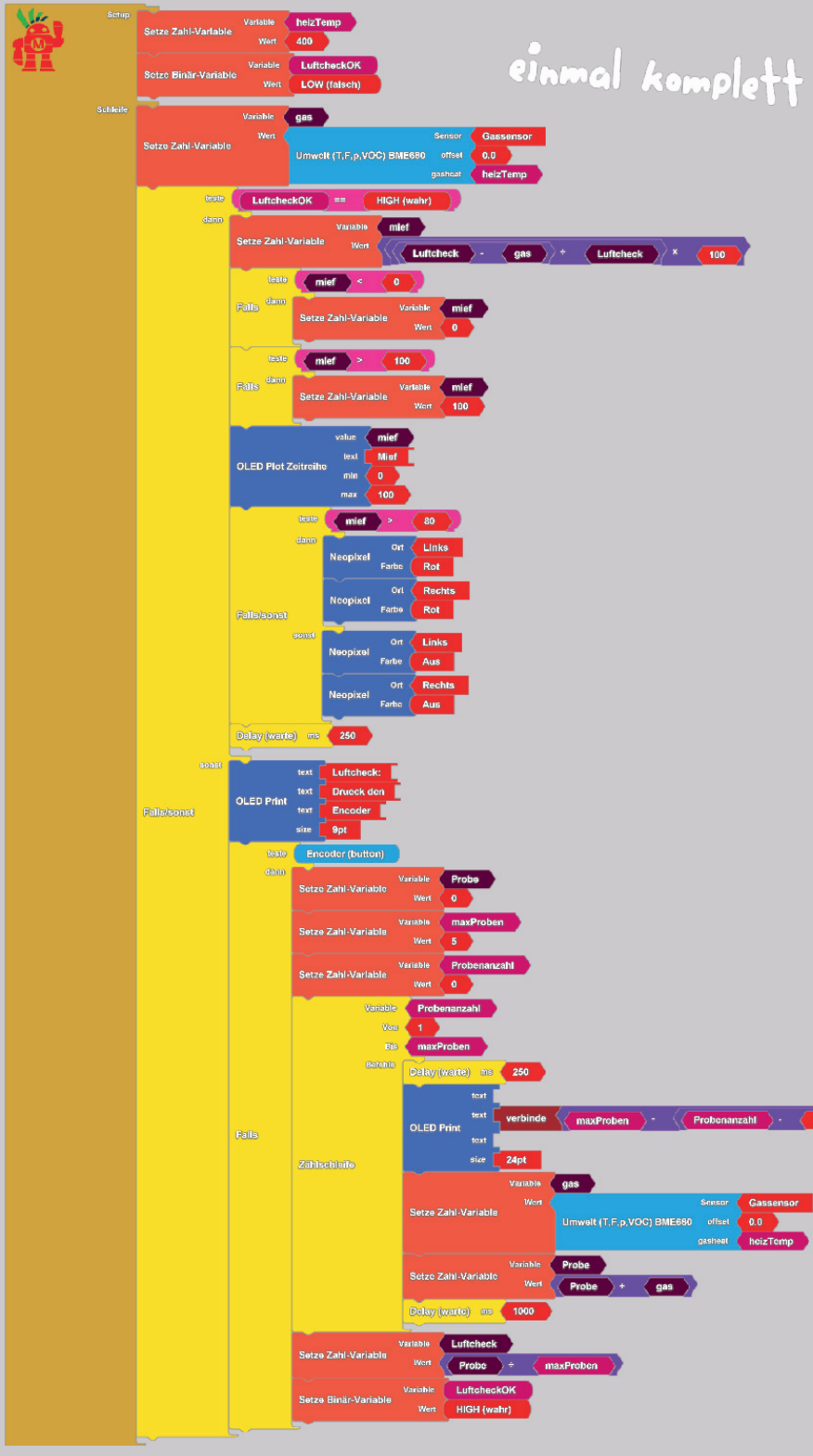
Erstelle ein einfaches Programm, das den VOC-Wert misst und auf dem OLED-Display anzeigt. So bekommst du ein Gefühl dafür, was „viel" oder „wenig" bedeutet. Anfangs reicht es, einfach die Zahl zu beobachten oder eine kleine Kurve anzuzeigen. Du kannst auch eine Ampel-Anzeige

programmieren: grün bei niedrigen Werten, gelb bei mittleren, rot bei hohen.

So siehst du sofort, wenn sich etwas in der Luft verändert – zum Beispiel wenn du den Deckel einer Gewürzdose öffnest oder etwas stark riechendes neben den Sensor legst.



Makey:Lab erkennt den Geruch dieser reifen Banane.


einmal komplett
Setup
Setze Zahl-Variable
Variable
heizTemp
Wert
400
Setze Binär-Variable
Variable
LuftcheckOK
Wert
LOW (falsch)
Schleife
Setze Zahl-Variable
Variable
gas
Wert
Umwelt (T,F,p,VOC) BME680
Sensor
Gassensor
offset
0.0
gasheat
heizTemp
teste
LuftcheckOK
==
HIGH (wahr)
dann
Setze Zahl-Variable
Variable
mief
Wert
Luftcheck
-
gas
÷
Luftcheck
x
100
Falls
teste
mief
<
0
dann
Setze Zahl-Variable
Variable
mief
Wert
0
Falls
teste
mief
>
100
dann
Setze Zahl-Variable
Variable
mief
Wert
100
OLED Plot Zeitreihe
value
mief
text
Mief
min
0
max
100
Falls/sonst
teste
mief
>
80
dann
Neopixel
Ort
Links
Farbe
Rot
Neopixel
Ort
Rechts
Farbe
Rot
sonst
Neopixel
Ort
Links
Farbe
Aus
Neopixel
Ort
Rechts
Farbe
Aus
Delay (warte)
ms
250
sonst
OLED Print
text
Luftcheck:
text
Drueck den
text
Encoder
size
9pt
Falls/sonst
teste
Encoder (button)
dann
Setze Zahl-Variable
Variable
Probe
Wert
0
Setze Zahl-Variable
Variable
maxProben
Wert
5
Setze Zahl-Variable
Variable
Probenanzahl
Wert
0
Zählschleife
Variable
Probenanzahl
Von
1
Bis
maxProben
Betreibe
Delay (warte)
ms
250
OLED Print
text
text
verbinde
maxProben
-
Probenanzahl
-
1
text
size
24pt
Setze Zahl-Variable
Variable
gas
Wert
Umwelt (T,F,p,VOC) BME680
Sensor
Gassensor
offset
0.0
gasheat
heizTemp
Setze Zahl-Variable
Variable
Probe
Wert
Probe
+
gas
Delay (warte)
ms
1000
Falls
Setze Zahl-Variable
Variable
Luftcheck
Wert
Probe
÷
maxProben
Setze Binär-Variable
Variable
LuftcheckOK
Wert
HIGH (wahr)

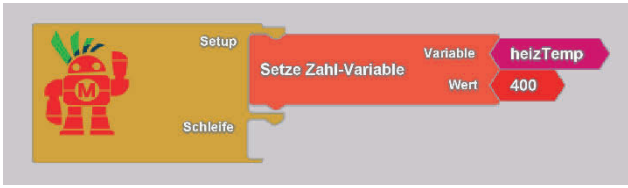
Das Riech-Programm

Jetzt wird Makey:Lab zur elektronischen Spürnase!

Wir bauen zusammen ein Programm, das zeigt, wie stark bestimmte Gerüche in der Luft sind – als Kurve auf dem Display, mit einer eigenen Miefskala. Und wenn es zu stark riecht, warnt dich sogar eine rote LED.

Zuerst legen wir fest, wie heiß der Sensor werden soll. Denn je nach Temperatur reagiert er unterschiedlich empfindlich – mal stärker auf bestimmte Gase, mal schwächer. Mit dieser Einstellung bestimmst du also, worauf Makey:Lab besonders anspringt.

Zieh dir dafür den Block **Setze Zahl-Variable** in ein neues Programm und platziere ihn unter Setup. Gib der Variable einen passenden Namen, zum Beispiel *heizTemp*.



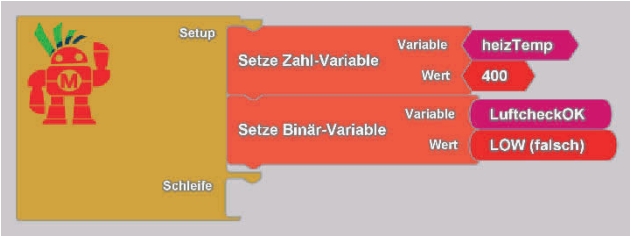
Der Sensor akzeptiert Heiztemperaturen zwischen 200 °C und 450 °C. Für den Anfang kannst du 400 °C wählen – damit reagiert er empfindlich auf viele verschiedene Gase und ist ideal für unseren Test. Später kannst du mit anderen Werten experimentieren und beobachten, wie sich das Messergebnis verändert.

LuftcheckOK

Wir werden gleich noch eine kleine Kalibrierung einbauen. Sie sorgt dafür, dass das Programm weiß, welche Gaskonzentration im Raum als normal gilt. Damit dieser Schritt nur einmal ausgeführt wird und sich sauber von der eigentlichen Messschleife trennen lässt, brauchen wir eine zusätzliche Variable.

Diese Variable merkt sich, ob die Kalibrierung bereits abgeschlossen ist. Sie kennt also nur zwei Zustände: falsch oder wahr.

Nimm dafür den Block **Setze Binär-Variable** – er ist genau für solche Ja/Nein-Entscheidungen gedacht.



Benenne die Variable in *LuftcheckOK* um und setze den Startwert auf LOW (falsch).

Jetzt baust du die eigentliche Programmschleife auf.

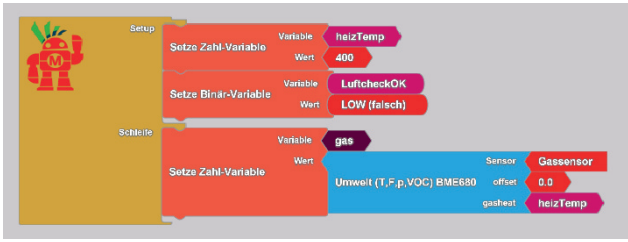
Beginne mit einem **Setze Zahl-Variable**-Block. Damit richtest du die Variable *gas* ein.

Wichtig: Dieses Mal brauchst du eine Float-Variable, weil der Sensor sehr genaue Werte liefert – also auch mit Nachkommastellen.

Wirf also die normale (lila) Standardvariable raus und hol dir unter dem Reiter *Variablen / Konstanten* die dunkellila Float-Variable.

Den Wert bekommt diese Variable aus dem erweiterten **Umwelt-Sensor (T, F, p, VOC) BME680**-Block.

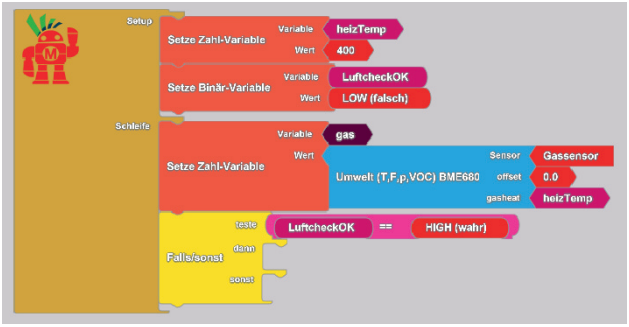
Dort wählst du den *Gassensor* aus, lässt den Offset bei 0.0 und legst bei gasheat deine Variable *heizTemp* ab – du kannst sie einfach klonen und hier einfügen.



So holt sich das Programm bei jedem Schleifendurchlauf den aktuellen Gaswert und speichert ihn in der Variable *gas*.

Jetzt, wo das Programm den aktuellen Gaswert ausliest, können wir prüfen, ob die Kalibrierung schon abgeschlossen ist.

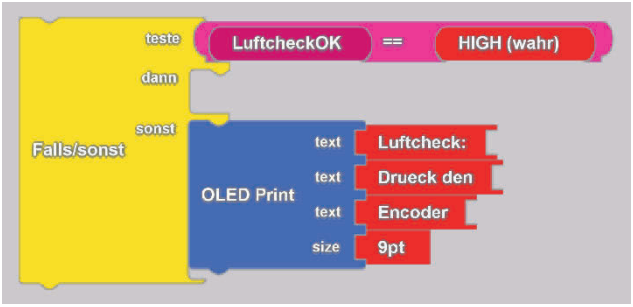
Dafür legst du einen **Falls/sonst**-Block an.



Ist die Kalibrierung abgeschlossen, erscheint die Miefskala auf dem Display. Wenn nicht, zeigt Makey:Lab stattdessen den Kalibrierungsmodus an.

Für die Überprüfung legst du unter teste einen logischen Vergleich ab:
LuftcheckOK == HIGH (wahr)

Nun kümmern wir uns zuerst um die Kalibrierung. Zieh dazu einen **OLED Print**-Block unter sonst.



Dort trägst du etwas Eindeutiges in die drei Textzeilen ein, die zur Verfügung stehen – zum Beispiel:

Luftcheck:
Drueck den
Encoder

Dieser Hinweis erscheint als Erstes auf dem Display, wenn dein Programm startet.

Damit Makey:Lab darauf reagieren kann, legst du nun einen weiteren **Falls**-Block unter den **OLED**-Block.

Unter *teste* platzierst du den **Encoder (button)**-Block – du findest ihn im Reiter *Sensoren*.

Geruchsproben nehmen

Unter *dann* baust du jetzt die eigentliche Kalibrierung auf:

Makey:Lab soll fünf Proben (also Messungen) in einem Abstand von jeweils einer Sekunde nehmen.

Am Ende wird daraus der Durchschnitt berechnet. So erhalten wir einen stabilen Grundwert – denn der Sensor „flattert“ zu Beginn oft noch ein wenig.

Dafür sind noch ein paar zusätzliche Variablen nötig.

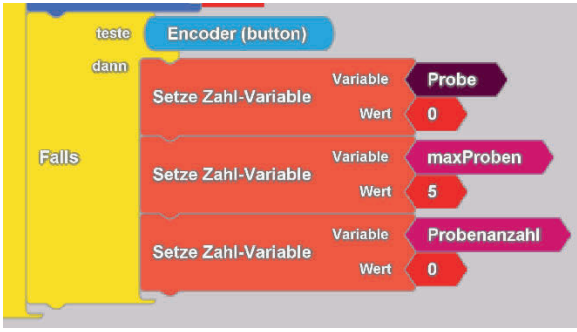
Den aktuellen Messwert aus dem Sensor haben wir ja schon in *gas* gespeichert. Jetzt brauchst du drei weitere Variablen:

Probe – steht für eine einzelne der fünf Messungen. Diese Variable muss wieder float sein, weil sie Nachkommastellen braucht. Setze sie anfangs auf 0.

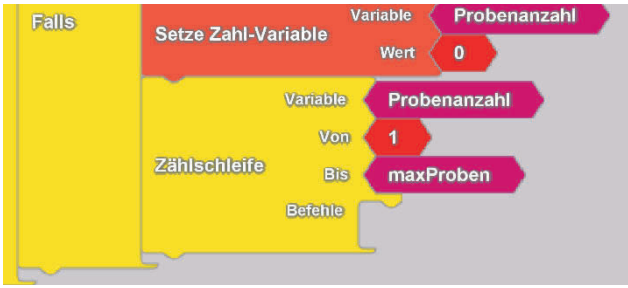
maxProben – das ist die Anzahl der Messungen, die Makey:Lab nehmen soll. Hier reicht eine ganzzahlige (integer) Variable. Setze den Wert auf 5.

Probenanzahl – damit erkennt das Programm, bei der wievielten Probe es gerade ist. Auch das ist eine integer-Variable, anfangs mit dem Wert 0.

Lege alle drei Variablen mit einem **Setze Zahl-Variable**-Block an und ordne sie untereinander unter dann an.



Damit Makey:Lab die Messung automatisch mehrmals wiederholt, holst du dir jetzt unter *Kontrollstrukturen* den **Zählschleife**-Block.



Damit lässt du einen Programmteil mehrmals hintereinander ausführen – in unserem Beispiel genau fünfmal für unsere fünf Messungen. Du kannst dir das vorstellen wie eine kleine Mini-Schleife innerhalb deiner großen Programmschleife.

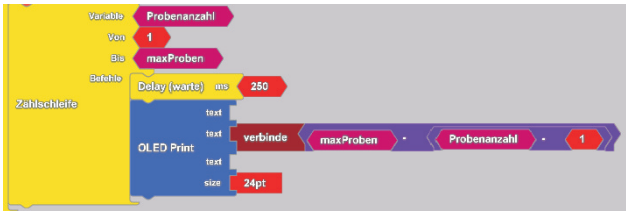
Am Zählschleife-Block tauscht du die Standard-Variable *count* gegen deine *Probenanzahl*-Variable aus.

Bei *von* lässt du die 1 stehen und bei *bis* setzt du deine Variable *maxProben* ein.

So kannst du später ganz einfach die Anzahl der Messungen ändern – zum Beispiel statt fünf sogar zehn Proben nehmen.

Jetzt legst du fest, was bei jedem Schleifendurchlauf passieren soll. Beginne mit einem kleinen **Delay** von *250 ms*. Dadurch hat der Sensor zu Beginn ein bisschen Zeit, sich zu beruhigen. Eigentlich könnten wir danach direkt die Messwerte sammeln – aber damit du siehst, was gerade passiert, bauen wir zuerst noch eine kleine

Countdown-Anzeige ein. So erkennst du immer, wie viele Proben noch fehlen.



Zieh dir jetzt einen **OLED Print**-Block unter das **Delay**.

In die mittlere Zeile kommt ein **verbinde**-Block. Daran hängen wir eine kleine Rechnung mit zwei **minus**-Blöcken an – du findest sie unter *Mathematische Operationen*.

So rechnen wir:
maximale Probenzahl – (aktuelle Probe – 1)

Diese Formel sorgt dafür, dass dein Countdown bei 5 startet und nach jeder Probe herunterzählt.

Wenn du nämlich einfach nur *maximale Probenzahl – aktuelle Probe* rechnen würdest, ginge es mit 4 los – wir wollen aber mit 5 beginnen.

Beispiel:
5 – (1 – 1) = 5
5 – (2 – 1) = 4
… und so weiter.

Damit der Countdown gut lesbar ist, stellst du beim **OLED Print**-Block die Schriftgröße auf *24 pt*. Das sieht auf dem Display richtig schön groß aus!

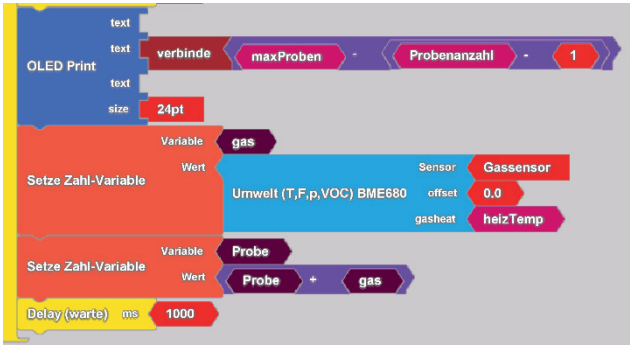
Jetzt nimmst du die eigentliche Probe mit einem **Setze Zahl-Variable**-Block. Oben setzt du unsere Float-Variable *gas* ein. Unter *Wert* kommt der **Gassensor**-Block, genau wie oben.

Bei *gasheat* legst du wieder die Variable *heizTemp* ab.

Damit wir mehrere Proben erfassen können, addieren wir sie einfach in der Variable *Probe*.

Das machst du mit einem weiteren **Setze Zahl-Variable**-Block: Lege hier die Float-Variable *Probe* ab, und unter *Wert* nimmst du einen **addieren**-Block.

Darin rechnest du:
Probe + *gas*



So wird jeder neue Messwert zum bisherigen Wert in *Probe* hinzugefügt.

Am Ende enthält die Variable also die Summe aller Messungen der Zählschleife.

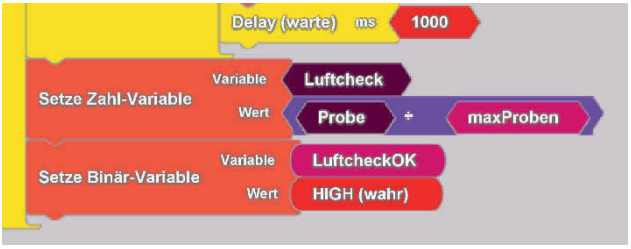
Um den Abstand zwischen den Messungen festzulegen, legst du am Ende ein **Delay** von *1000 ms* ein.

So wird jede Sekunde eine neue Probe entnommen – insgesamt dauert die Zählschleife also etwa 5 Sekunden.

Damit ist die Zählschleife abgeschlossen.

Am Ende berechnen wir aus der Summe aller Proben unseren eigentlichen Kalibrierungswert, also den *Luftcheck*.

Dafür legst du einen weiteren **Setze Zahl-Variable**-Block unter die **Zählschleife**.



Hier rechnest du mit einem divisions-Block:
Probe ÷ maxProben – also die Summe aller Messwerte geteilt durch die Anzahl der Proben.

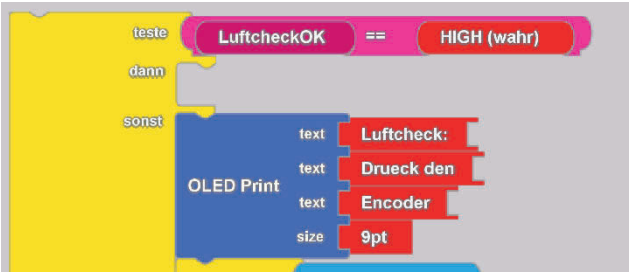
So erhältst du den Durchschnitt der Messungen aus den fünf Sekunden. Das ist dein stabiler Frischluft-Wert, mit dem wir später alle weiteren Messungen vergleichen.

Da der *Luftcheck* nun OK, also abgeschlossen ist, setzt du am Ende der Kalibrierung noch deine Binär-Variable *LuftcheckOK* auf *HIGH (wahr)*.

Von Messwerten zur Miefskala

Die Prüfung *LuftcheckOK* war erfolgreich – die Kalibrierung ist abgeschlossen. Jetzt geht es um den Programmteil, der dir den aktuellen Geruchswert fortlaufend anzeigt.

Wir machen also weiter im **Falls/sonst**-Block unter *dann*.



Der Sensor liefert Zahlen, die man so noch nicht direkt versteht.

Damit du auf dem Display sofort siehst, wie stark es gerade riecht, rechnen wir diesen Wert um – und erstellen unsere eigene **Miefskala**.

Dazu vergleichen wir die aktuelle Luft mit dem *Luftcheck*-Wert, den wir eben beim Kalibrieren gemessen haben.

Die Formel sieht so aus:
(Luftcheck – gas) / Luftcheck × 100

Das bedeutet: Wir schauen, wie viel kleiner der aktuelle Messwert (*gas*) im Vergleich zur Frischluft ist.

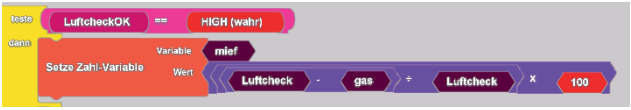
Je größer dieser Unterschied, desto stärker riecht es.

Wenn die Luft frisch ist, kommt also ein kleiner Wert heraus (wenig Mief).

Wenn der Sensor viele Gase erkennt, steigt der Prozentwert – und die Miefskala klettert nach oben.

So bekommst du eine übersichtliche Skala: 0 % steht für frische Luft, 100 % bedeutet: hier riecht's ordentlich!

Die ganze Rechnung findet wieder in einem **Setze Zahl-Variable**-Block statt. Als Float-Variable verwendest du *mief*.



Diese bekommt als Wert unsere Formel. Dafür verschachtelst du drei **Mathe**-Blöcke:

Zuerst einen **Multiplikations**-Block (× *100* für den Prozentwert).

Links darin setzt du einen **Divisions**-Block, der durch *Luftcheck* teilt.

Und ganz links legst du einen **Subtraktions**-Block ab und rechnest *Luftcheck – gas*.

Werte eingrenzen

Damit der Miefwert nicht aus dem Rahmen fällt und immer zwischen 0 % und 100 % bleibt, bauen wir zwei kleine Schutzprüfungen ein.

Wenn der Sensor weniger Gas misst als bei der Kalibrierung, setzen wir den Mief-Wert auf 0 %.

Und falls der Messwert durch Schwankungen einmal über 100 % springt, kappen wir ihn bei 100 %.

So bleibt die Anzeige übersichtlich – und unsere Nase weiß genau, wann's wirklich stinkt.

Das bekommst du mit zwei **Falls**-Blöcken hin:

Prüfe zuerst, ob die Variable mief kleiner als *0* ist. Nimm dafür einen **logischen Vergleich**-Block.

Wenn die Bedingung erfüllt ist, soll der Wert bei *0* bleiben.

Lege dazu einen **Setze Zahl-Variable**-Block unter dann und setze *mief = 0*.

Den zweiten Block baust du genauso auf:

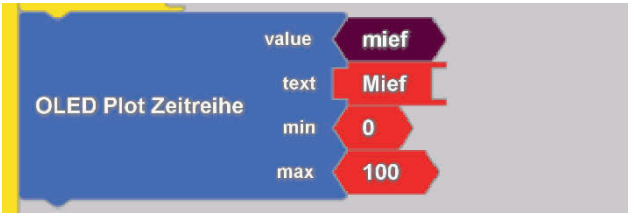
Wenn *mief* größer als *100* ist, dann setze *mief = 100*.

Mief sichtbar machen

Jetzt soll Makey:Lab den Geruch sichtbar machen. Zieh dir den Block **OLED Plot Zeitreihe** unter die Begrenzung.

Dieser Block zeigt, wie sich der Miefwert über die Zeit verändert.

Unter *text* gibst du der Kurve einen Namen – du könntest sie *VOC* nennen, weil wir flüchtige organische Verbindungen messen. Aber wir wollen ja unsere Miefskala, also nenn sie ruhig *mief*.



Als Wertebereich trägst du *0* bis *100* ein – das entspricht später unserer Prozentanzeige.

Und unter *value* legst du die Float-Variable *mief* ab.

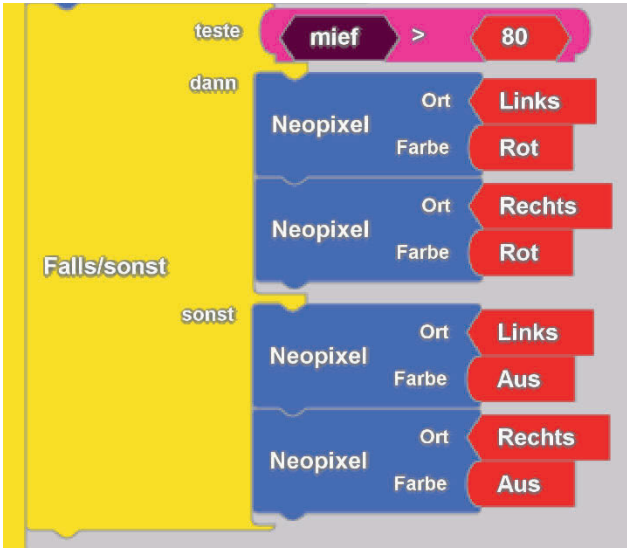
Mief-Warnung

Zusätzlich wollen wir noch eine kleine LED-Warnung einbauen – sie soll leuchten, wenn es zu stinkig wird.

Dafür hängst du unter den **Zeitreihe**-Block einfach einen **Falls/sonst**-Block.

Teste darin mit einem *logischen Vergleich*, ob der Wert *mief größer als 80 ist*.

Diesen Beispielwert kannst du natürlich beliebig anpassen.



Wenn die Bedingung erfüllt ist, sollen die **Neopixel** rot leuchten.

Lege dafür unter dann zwei **Neopixel**-Blöcke an und stell Position und Farbe ein – einer links, einer rechts.

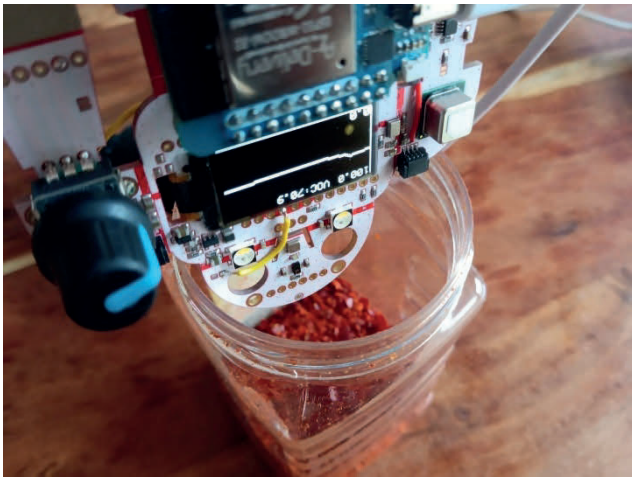
Unter *sonst* kommen wieder zwei **Neopixel**-Blöcke, diesmal stellst du sie auf *aus*.

So gehen die LEDs automatisch wieder aus, wenn die Bedingung nicht mehr erfüllt ist.

Zum Schluss fügst du noch ein **Delay** von *250 ms* ein.

Das ist nicht unbedingt notwendig, weil das Plotten unseres Miefwerts ohnehin etwas verzögert läuft – aber es ist sauberer programmiert. Der Sensor braucht nämlich für jede Messung ein bisschen Zeit, damit der Wert zuverlässig bleibt.

Keine Sorge: Übrigens, nur weil der Sensor etwas anzeigt, heißt das nicht, dass es bei dir unangenehm riecht. Der BME680 ist einfach extrem empfindlich – schon normale Raumluft mit ein bisschen Atemluft oder Küchendunst reicht, damit er reagiert. Das ist ganz normal – und zeigt dir, wie feinfühlig deine künstliche Nase ist! Ziemlich cool, oder?



Makey:Lab schnüffelt an der Chili-Dose. Es braucht etwas, bis man es an der Kurve sieht.

Auf dem richtigen Riecher

Jetzt läuft dein Makey:Lab als elektronische Nase – Zeit für die ersten Experimente!

Bevor du eine ganze Testreihe startest, kannst du mit ein paar einfachen Gegenständen ausprobieren, wie dein Sensor auf verschiedene Gerüche reagiert.

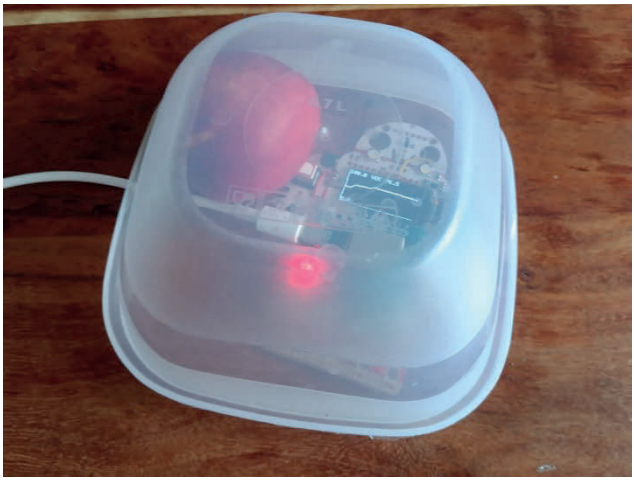
Wenn du das Programm startest, zeichnet dein Makey:Lab eine Kurve auf dem Display. Sie zeigt dir, wie stark es gerade riecht – und ob sich der Geruch im Laufe der Zeit verändert. Probiere aus, was passiert, wenn du z. B. eine reife Banane oder eine Gewürzdose danebenlegst.

Experimentiere mit unterschiedlichen Dingen und untersuche ihre Ausdünstungen. Das geht mit Feststoffen, aber auch mit Flüssigkeiten.

Achtung: Halte Makey:Lab mit dem Sensor nur in die Nähe – niemals direkt in die Flüssigkeit! Der Sensor ist sehr empfindlich und hat eine kleine Öffnung, dort darf niemals ein Fremdstoff eindringen. Deswegen fass ihn am besten nicht an – selbst Fingerabdrücke oder Creme können den Sensor beschädigen.

Du wirst vermutlich nur sehr geringe oder vielleicht sogar gar keine Veränderungen auf dem Display sehen. Das liegt daran, dass sich die Gerüche in der Luft schnell verflüchtigen. Wenn du Makey:Lab und dein Testobjekt in ein geschlossenes Gefäß legst, dann bekommst du deutlich zuverlässigere Ergebnisse.

Du brauchst dafür kein Labor – schon eine Salatdose oder ein Gefrierbeutel reichen völlig, dann sammeln sich die Gase darin an – und der Sensor hat mehr Zeit, sie zu erkennen.



Unter einer Dose lassen sich manche Proben besser erfassen.

(Über)Reife Früchte erkennen

Wusstest du schon? Auch in der Lebensmittelindustrie wird dieses Prinzip genutzt! Dort kommen spezielle Sensoren zum Einsatz, die – genau wie dein Makey:Lab – auf bestimmte Gase reagieren. So können zum Beispiel Erdbeeren automatisch nach Farbe und Geruch sortiert werden. Die Maschinen erkennen, ob eine Frucht reif genug ist – oder noch zu grün. Technik hilft hier also, frisches Obst besser zu verarbeiten und Verschwendung zu vermeiden. Ziemlich clever, oder?

Deine eigene Testreihe

Wie bei einem kleinen Forschungsprojekt kannst auch du nun verschiedene Proben vergleichen. Vielleicht kannst du auch gemeinsam mit anderen eine kleine Forschergruppe bilden: Wer findet die geruchsintensivste Frucht?

Am besten startest du mit einem Referenzwert – zum Beispiel mit leerer Glocke oder einem Stück Papier.

Dabei solltest du beachten: Die Umgebung hat einen großen Einfluss auf die Messung. Schon ein leichter Luftzug oder eine offene Tür kann die Werte verändern – ebenso wie die Temperatur oder ob du gerade danebenstehst. Deshalb empfehlen wir dir, möglichst immer unter den gleichen Bedingungen zu messen – zum Beispiel in einem ruhigen Raum, ohne Durchzug. Wir geben dir bewusst keine festen Werte vor, denn jeder Versuchsaufbau ist etwas anders: Deine Frucht, deine Raumluft, sogar das verwendete Gefäß – all das beeinflusst, was der Sensor misst.

Probiere nacheinander verschiedene Dinge aus:

- Frisches Obst vs. altes Obst
- Gewürze, Teebeutel, Kräuter
- Zahnpasta, Seife, Joghurtdeckel

Notiere dir die gemessenen Werte oder bewerte sie als niedrig / mittel / hoch. Du kannst sogar kleine Tabellen anlegen, Farben vergeben oder eine Kurve beobachten.

So könnte deine Tabelle aussehen (Beispielwerte):

Probe	Mief-Wert	Eindruck
Referenz (leer)	0	neutral
Apfel frisch	10	eher frisch
Banane alt	50	sehr reif
Chili	75	würzig

Hinweis: Der Sensor braucht manchmal ein bisschen, um neue Gerüche richtig zu erfassen – nicht weil er langsam ist, sondern weil sich die Duftstoffe erst in der Luft verteilen müssen.

Sobald sie ankommen, reagiert der Sensor schnell – der Wert steigt steil an.

Beim Absinken ist er dagegen deutlich langsamer – das liegt am Sensorprinzip und ist völlig normal.

Also: Lass dir Zeit und beobachte, wie sich der Wert langsam wieder einpendelt – genau das macht gute Forschung aus!

Was du daraus lernen kannst

Du siehst: Auch das, was wir sonst nur riechen, lässt sich mit ein bisschen Technik sichtbar machen. Dein Makey:Lab zeigt, dass Gerüche messbar sind – und dass Daten Geschichten erzählen können, etwa über Reife oder Sauberkeit.

So lernst du, wie Sensoren arbeiten und wie aus Zahlen Beobachtungen werden. Genau so funktioniert Forschung: messen, vergleichen, verstehen. Dein kleines Experiment ist damit ein echtes Stück Wissenschaft im Mini-Format.

Du willst noch mehr wissen?

SCAN MICH

Hinter dem QR-Code findest du spannende Projekte vom Umwelt-Campus Birkenfeld – dort erfährst du, wie Forscher mit Sensoren und KI verschiedene Getränke klassifizieren.

Im KI-Labor forschen Studierende an „künstlichen Nasen“, die in der Landwirtschaft, im Weinbau oder sogar in der Kunststoffindustrie eingesetzt werden.
Mit einer guten Idee kannst du später vielleicht sogar deine eigene Firma gründen.

Unterwegs mit Makey:Lab

Immer dabei - bereit für Experimente

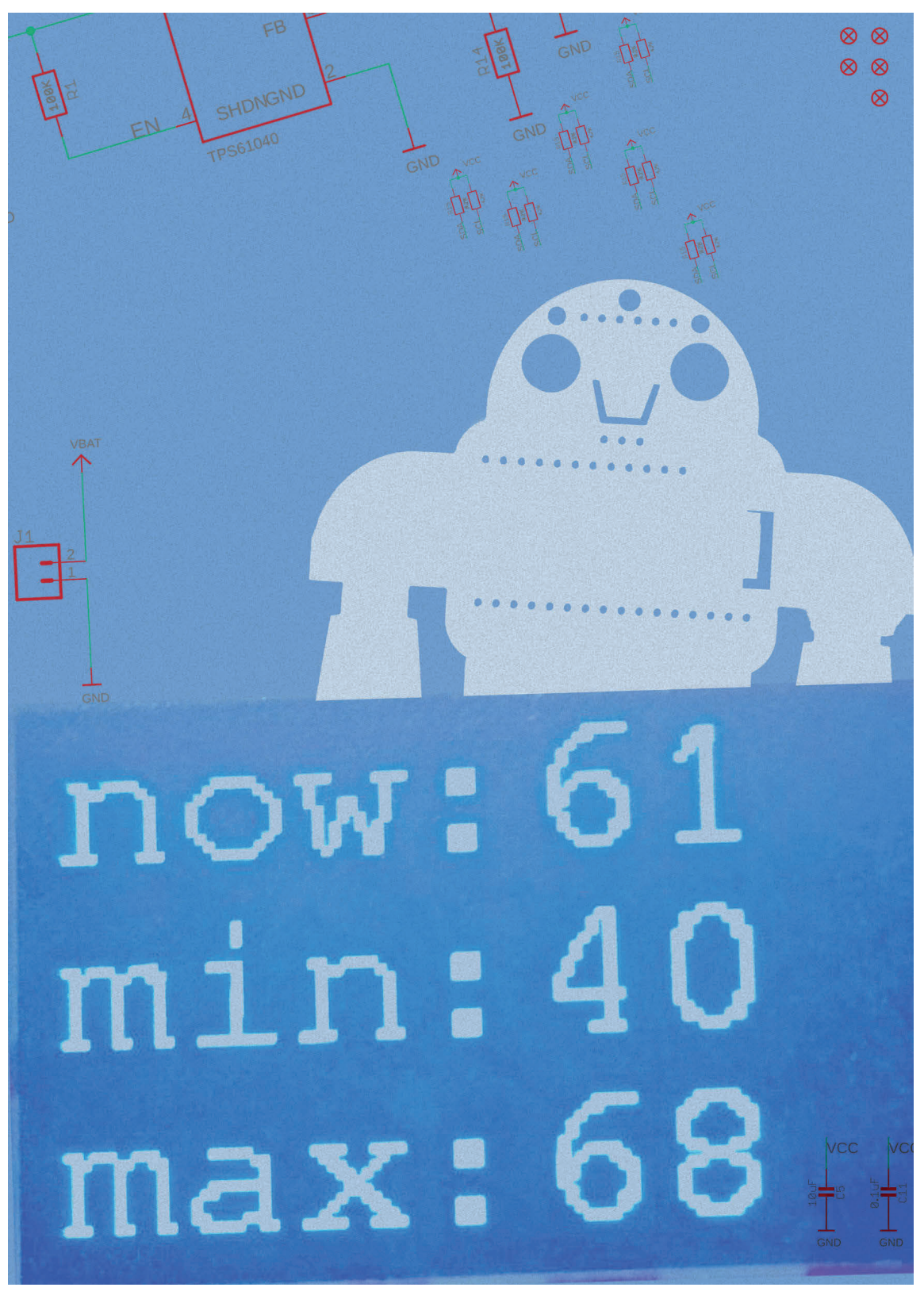
Dein Makey:Lab passt perfekt in die Hosentasche – aber warum verstecken?

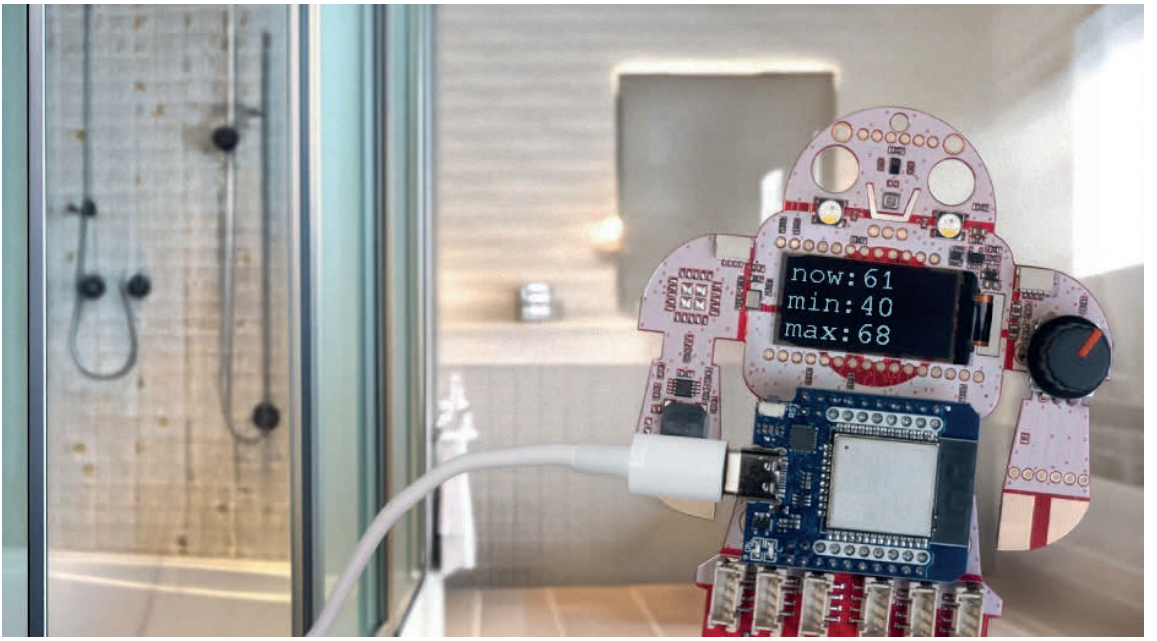
Durch die mittlere Öse kannst du einfach ein Schlüsselband anbringen, am besten natürlich eines von der Maker Faire!

So hängt dein Makey:Lab sicher um den Hals oder am Rucksack, immer griffbereit für spontane Ideen.

Egal ob Workshop, Messe oder Naturexperiment – dein Makey ist dabei. Und dabei kannst du allen zeigen, dass du ein echter Maker bist!

cool, oder?


FB
SHDNGND
EN
TPS61040
R1
100k
GND
R14
100k
VCC
SDA
SCL
VBAT
J1
now:61
min:40
max:68
10uF
C15
0.1uF
C11



#04 Die Duschpetze

Damit misst du die Feuchtigkeit in deinem Bad

Kennst du das? Du willst schnell ins Bad – aber jemand duscht schon ewig. Wahrscheinlich ist der Spiegel schon komplett beschlagen, und die Luft ist drückend feucht.

Klar: Duschen ist schön – manchmal nötig, manchmal einfach entspannend.
Aber wer zu lange duscht, verschwendet nicht nur Zeit, sondern auch eine Menge Wasser.

Eine Minute Duschen verbraucht im Schnitt rund 12 Liter Wasser. Wer also 10 Minuten unter der Dusche steht, lässt etwa 120 Liter den Abfluss hinunterrauschen – und das ist nur das Wasser. Hinzu kommt die Energie fürs warme Wasser – denn mal ehrlich: Wer duscht schon gern kalt?

Gerade in Zeiten von Klimakrise und steigenden Preisen ist das ein echtes Thema.

Anstatt sich über zu lange Duschen zu streiten, kannst du mit Makey:Lab einen neutrale Schiedsrichter ins Bad schicken.

Wir bauen eine Duschpetze

Die Idee: Ein kleines Programm, das erkennt, wie lange im Badezimmer geduscht wurde – und eine freundliche Erinnerung anzeigt, wenn's langsam reicht.

Denn unser Sensor kann nicht nur das Wetter messen, sondern ist auch in unseren vier Wänden eine echte Hilfe, wenn es um Luftfeuchtigkeit geht. Die steigt beim Duschen nämlich deutlich an.

Jedes Bad ist anders!

Aber halt – nicht so schnell:
Nicht jede Dusche produziert gleich viel Dampf. Einige Badezimmer haben ein Fenster, andere eine Lüftung. Und je nachdem, wo dein Makey:Lab steht, bekommt er ganz andere Messwerte.

Um unsere Duschpetze zu bauen, brauchen wir mehrere Funktionen:

Scout

Wir starten erstmal mit einem kleinen Forschungsmodus – unserem **Scout**. Damit beobachtest du, wie sich die

Luftfeuchtigkeit beim Duschen verändert, und findest eigene Werte, die zu deinem Badezimmer passen.

Danach entscheidest du, was „zu lang“ ist – und programmierst deine Duschpetze nach deinen Regeln.

Duschpetze

Erkennt, wenn der Wert über deiner Schwelle liegt, und zeigt eine Warnung auf dem Display sowie über eine LED an, wenn die eingestellte Duschzeit überschritten ist. Zusätzlich kannst du auch einen Alarm über den kleinen Buzzer ausgeben.

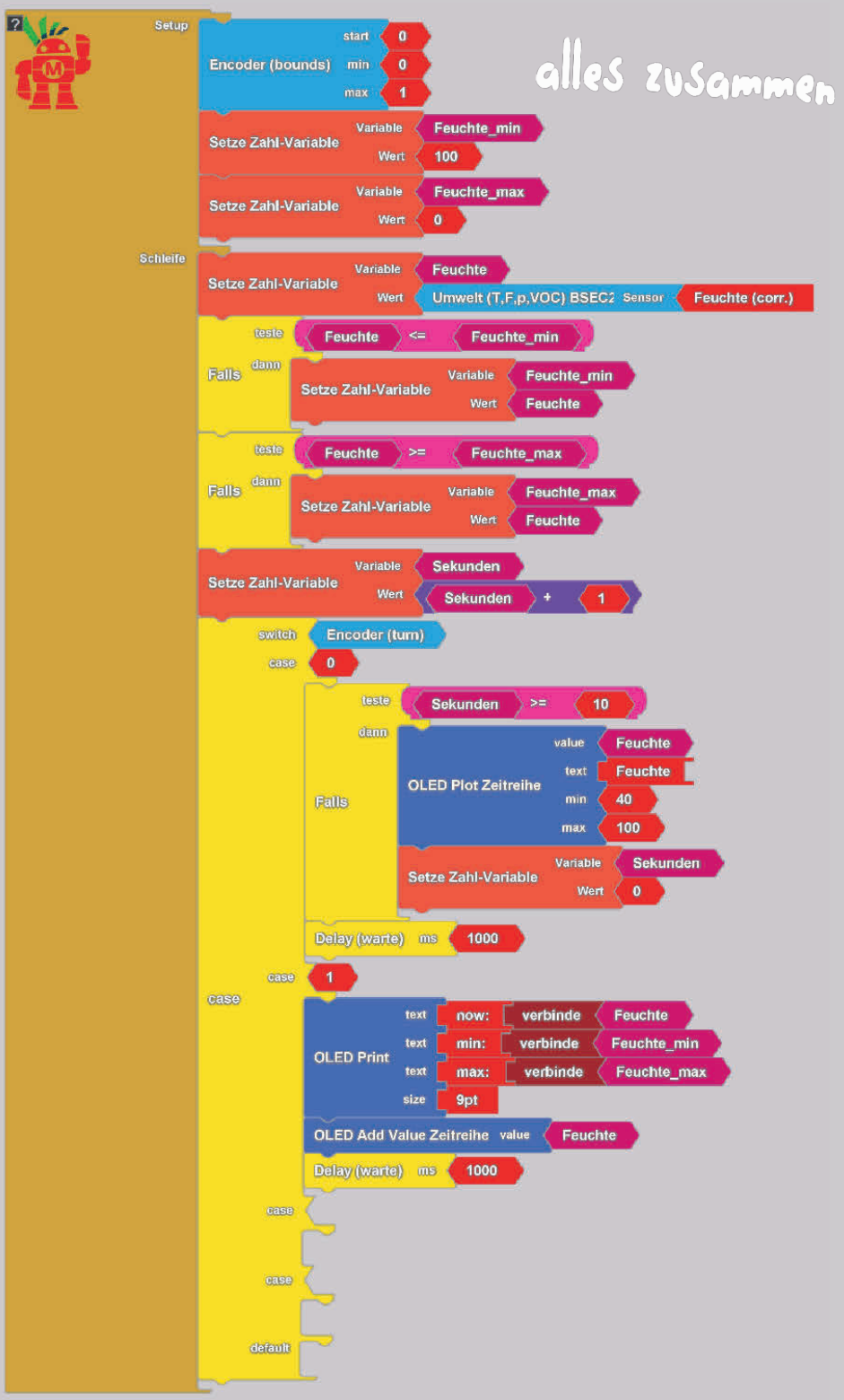
Statistik

Zeigt dir die Dauer deiner letzten Duschsession – perfekt für alle, die Fortschritte sehen (oder einfach angeben) wollen. Diese Mini-Statistik kannst du ganz einfach in die Duschpetze integrieren.

Bereit? Du brauchst nur eine Variable für die Feuchtigkeit, ein bisschen Mathe – und ein ruhiges Plätzchen im Bad für dein Makey:Lab. (**Achtung**: Nicht **in** die Dusche! Direkte Feuchtigkeit und Elektronik vertragen sich nicht.)

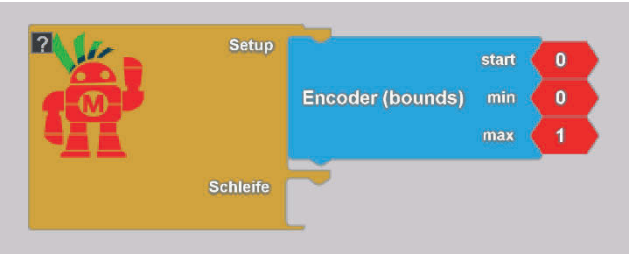
Lerne dein Badezimmer kennen

Zunächst baust du dir deinen Scout-Modus. Dieser zeigt die Luftfeuchtigkeit im Verlauf an und ermittelt die Minimal- und Maximalwerte. So kannst du später die Duschpetze viel genauer einstellen.


alles zusammen
Setup
Encoder (bounds)
start
0
min
0
max
1
Setze Zahl-Variable
Variable
Feuchte_min
Wert
100
Setze Zahl-Variable
Variable
Feuchte_max
Wert
0
Schleife
Setze Zahl-Variable
Variable
Feuchte
Wert
Umwelt (T,F,p,VOC) BSEC2
Sensor
Feuchte (corr.)
Falls
teste
Feuchte
<=
Feuchte_min
dann
Setze Zahl-Variable
Variable
Feuchte_min
Wert
Feuchte
Falls
teste
Feuchte
>=
Feuchte_max
dann
Setze Zahl-Variable
Variable
Feuchte_max
Wert
Feuchte
Setze Zahl-Variable
Variable
Sekunden
Wert
Sekunden
+
1
switch
Encoder (turn)
case
0
Falls
teste
Sekunden
>=
10
dann
OLED Plot Zeitreihe
value
Feuchte
text
Feuchte
min
40
max
100
Setze Zahl-Variable
Variable
Sekunden
Wert
0
Delay (warte)
ms
1000
case
1
case
OLED Print
text
now:
verbinde
Feuchte
text
min:
verbinde
Feuchte_min
text
max:
verbinde
Feuchte_max
size
9pt
OLED Add Value Zeitreihe
value
Feuchte
Delay (warte)
ms
1000
case
case
default

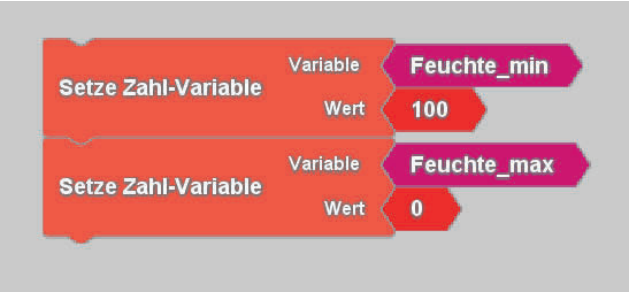
Vorbereitung: Werte und Umschalter festlegen

Bevor es in die Schleife geht, bereiten wir im Setup ein paar Dinge vor:



1. Encoder-Bereich definieren

Unter dem Reiter *Sensoren* findest du den Block **Encoder (Bounds)**. Hier legst du fest, zwischen wie vielen Bildschirmansichten du mit dem Drehregler umschalten kannst. Gib als Minimum *0* und als Maximum *1* ein. Du hast damit zwei Ansichten: grafischer Verlauf und Zahlenwerte – perfekt zum Start. Mit dem Wert *start 0* stellst du ein, welche Anzeige beim Programmstart aktiviert ist.



2. Startwerte setzen

Mit zwei **Setze Zahl-Variable**-Blöcken legst du die Anfangswerte für *Feuchte_min* und *Feuchte_max* fest. Damit du bei der ersten Messung direkt eine Veränderung siehst, trickst du ein bisschen:

– *Feuchte_min* bekommt erstmal den höchsten möglichen Wert, z. B. 100.
– *Feuchte_max* bekommt den niedrigsten möglichen Wert, z. B. 0.

So wird schon der erste echte Messwert automatisch übernommen, da er auf jeden Fall einen Wert dazwischen liefern wird.

Hauptschleife: Messen, speichern, vergleichen

In der Programmschleife holst du dir den aktuellen Feuchtigkeitswert:

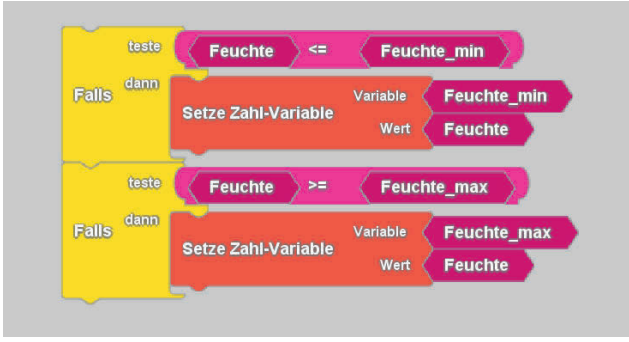
Feuchte messen

Mit **Setze Zahl-Variable** liest du den Feuchtigkeitswert vom Sensor ein, z. B. als *Feuchte*. Wähle in der Quelle des Blocks *Feuchte (corr.)*.

Min/Max ermitteln

Jetzt vergleichst du, ob der neue Wert kleiner oder größer ist als die bisherigen Messungen:

– Ist der aktuelle Feuchtewert kleiner oder gleich dem gespeicherten *Feuchte_min*, dann wird *Feuchte_min* durch den neuen Wert ersetzt.
– Ist der aktuelle Wert größer oder gleich dem bisherigen *Feuchte_max*, dann wird *Feuchte_max* aktualisiert.

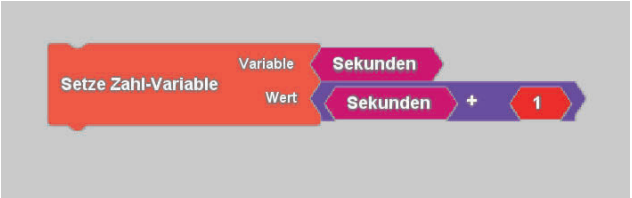


Das erledigst du mit jeweils einem **Falls**-Block und einem **logischen Vergleichsblock** wie <= oder >=.

Zeitschaltuhr bauen

Damit das Programm nicht jede Sekunde neue Punkte in den Verlauf einträgt, bauen wir einen kleinen Timer:

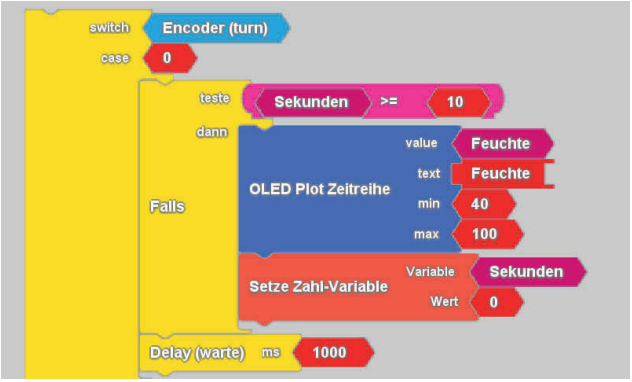
Erstelle hierzu eine **Zahlenvariable** *Sekunden*, die du bei jedem Schleifendurchlauf durch eine **mathematische Operation** um 1 erhöhst – also *Sekunden* + *1*.



Damit kannst du prüfen, wann der Wert 10 erreicht – also 10 Sekunden verstrichen sind. Erst dann soll ein neuer Wert in die Zeitreihe gezeichnet werden.

Bildschirme definieren: Case 0 & 1

Jetzt kommt der spannende Teil: die verschiedenen Bildschirmansichten. Die baust du mit einem **Case-**Block auf (unter *Kontrollstrukturen*). Je nach Stellung des Encoders wird ein anderer Programmabschnitt ausgeführt.



Case 0: Zeitreihe anzeigen

Hier prüfst du mit einem *Falls*-Block, ob der Sekundenwert über 10 liegt. Dafür legst du einen **logischen Vergleich** unter *teste* ab, setzt die Variable

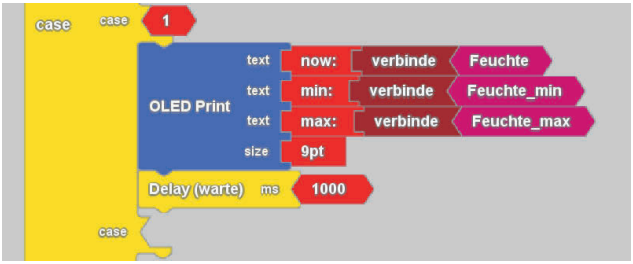
Sekunden ein, wählst als Vergleich >= (größer gleich) und legst eine **Zahlenvariable** 10 auf die andere Seite.
Nun bestimmst du, was passiert, wenn der Test erfolgreich war: Es soll ein neuer Feuchtewert in eine **OLED Plot Zeitreihe** eingetragen werden (Tipp: Das nennt man auch „plotten“). Hol dir also den Block aus dem Reiter *Anzeigen / Aktoren* und lege ihn unter *dann* des **Falls**-Blocks ab.

Damit der den richtigen Wert anzeigt, setzt du bei *value* deine Feuchtevariable ein. Bei *min* und *max* stellst du den angezeigten Wertebereich z. B. auf 40 und 100 – das sind typische Werte für Luftfeuchte in Innenräumen.
Nach der Zeitreihe setzt du die Variable *Sekunden* wieder auf 0, damit der Timer von vorne läuft. Für die richtige Taktung setzt du am Ende ein **Delay** von 1000 ms – also eine Sekunde.

Was jetzt passiert: Wenn der Encoder den *Case 0* ausgewählt hat, startet die gesamte Programmschleife, die du bisher aufgebaut hast, nach dem **Delay** von vorne. Es wird also wieder eine neue Sekunde gezählt und geprüft, ob der Wert bereits bei 10 Sekunden liegt. Wenn ja, wird ein neuer Wert in die Zeitreihe eingetragen.
So zeigt dir die Zeitreihe an, wie die Feuchtigkeit über die Zeit ansteigt oder fällt.

Hinweis: Wenn du Makey:Lab startest, dauert es so allerdings auch 10 Sekunden, bevor die erste Anzeige

erscheint. Nicht wundern – der Bildschirm bleibt anfangs kurz leer.



Case 1: Min/Max anzeigen

Auf dem zweiten Bildschirm soll eine kleine Statistik angezeigt werden. Verwende dafür einen **OLED Print**-Block, um *Feuchte_min* und *Feuchte_max* anzuzeigen. Ergänze auch hier ein **Delay** von 1000 ms, damit das Timing stabil bleibt – wir wollen ja weiterhin die Sekunden zählen!

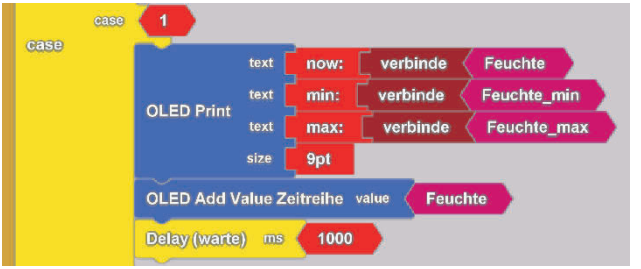
Hast du eine Idee, wie du dir hier noch als dritten Wert die aktuelle Feuchtigkeit anzeigen lassen kannst? Probiere es ruhig mal aus.

Kleiner Trick: Plotten im Hintergrund

Ein Problem gibt es noch: Sobald du auf *Case 1* umschaltest, stoppt die Zeitreihe – weil der **OLED Plot Zeitreihe**-Block nur in *Case 0* aufgerufen wird.

Was tun? Ganz einfach: Dafür gibt es den Block **OLED Add Value Zeitreihe**, den findest du unter dem Reiter *Anzeigen / Aktoren*. Leg ihn vor dem Delay ab – dann wird

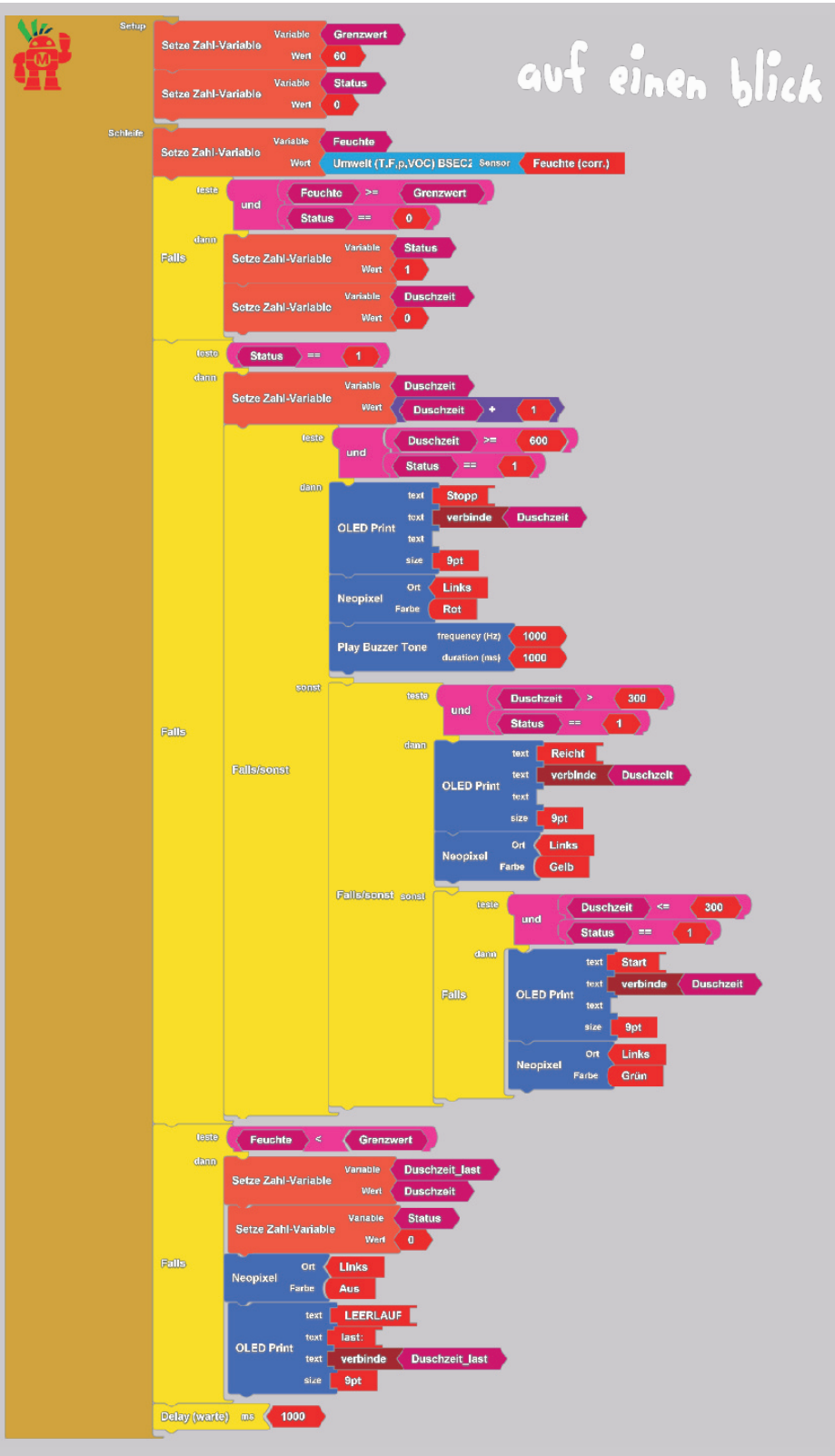
der Speicher der Zeitreihe auch weiterhin mit Daten gefüllt.



Leg dich auf die Lauer!

Jetzt hast du dein Scout-Programm fertig. Am besten legst du Makey:Lab beim nächsten Duschen einfach mal in die Nähe der Duschkabine. Nun werden die Daten erfasst, die du für deine Duschpetze brauchst.
Du musst zunächst herausfinden, wie die Luftfeuchtigkeit im Bad ist, wenn gerade keiner duscht. Außerdem interessiert dich, wie die Werte ansteigen, wenn jemand die Dusche nutzt.

Schreib dir dafür am besten den Minimalwert auf – das sollte der „Ruhezustand“ sein. Der Maximalwert zeigt dir, bis zu welchem Wert die Feuchtigkeit beim Duschen ansteigt. Auch wirst du im Verlauf genau sehen, wann das Wasser angestellt worden ist.


auf einen blick
Setup
Setze Zahl-Variable
Variable
Grenzwert
Wert
60
Setze Zahl-Variable
Variable
Status
Wert
0
Schleife
Setze Zahl-Variable
Variable
Feuchte
Wert
Umwelt (T,F,p,VOC) BSEC2
Sensor
Feuchte (corr.)
Falls
teste
und
Feuchte
>=
Grenzwert
Status
==
0
dann
Setze Zahl-Variable
Variable
Status
Wert
1
Setze Zahl-Variable
Variable
Duschzeit
Wert
0
Falls
teste
Status
==
1
dann
Setze Zahl-Variable
Variable
Duschzeit
Wert
Duschzeit
+
1
Falls/sonst
teste
und
Duschzeit
>=
600
Status
==
1
dann
OLED Print
text
Stopp
text
verbinde
Duschzeit
text
size
9pt
Neopixel
Ort
Links
Farbe
Rot
Play Buzzer Tone
frequency (Hz)
1000
duration (ms)
1000
sonst
Falls/sonst
teste
und
Duschzeit
>
300
Status
==
1
dann
OLED Print
text
Reicht
text
verbinde
Duschzeit
text
size
9pt
Neopixel
Ort
Links
Farbe
Gelb
sonst
Falls
teste
und
Duschzeit
<=
300
Status
==
1
dann
OLED Print
text
Start
text
verbinde
Duschzeit
text
size
9pt
Neopixel
Ort
Links
Farbe
Grün
Falls
teste
Feuchte
<
Grenzwert
dann
Setze Zahl-Variable
Variable
Duschzeit_last
Wert
Duschzeit
Setze Zahl-Variable
Variable
Status
Wert
0
Neopixel
Ort
Links
Farbe
Aus
OLED Print
text
LEERLAUF
text
last:
text
verbinde
Duschzeit_last
size
9pt
Delay (warte)
ms
1000

Achtung, jetzt wird gepetzt!

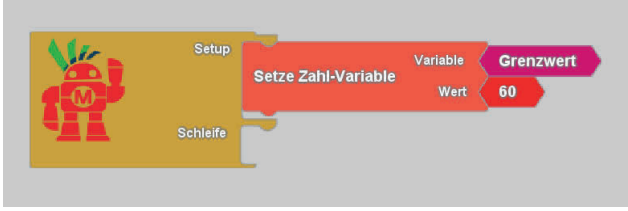
Für die eigentliche Duschpetze legst du ein neues Programm an.

Grundsätzlich soll hier die aktuelle Feuchte gemessen und mit deinem gewählten Grenzwert verglichen werden. Wird dieser überschritten, geht das Programm davon aus, dass geduscht wird.

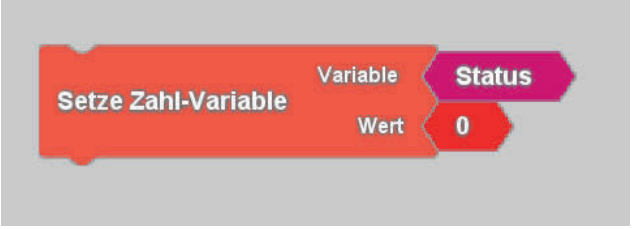
Sobald dies der Fall ist, startet auch ein Duschtimer. Nach bestimmten Zeitabständen werden dann verschiedene Aktionen ausgelöst. Und am Ende gibt es noch eine kleine Statistik, wie lang die Duschzeit war.

Der Duschpetze-Modus mit Statistik

Zu Beginn musst du deinem Programm im Setup sagen, wie hoch der Grenzwert ist. Dieser sollte leicht über dem *Feuchte_min*-Wert liegen, den du im Scout-Modus gemessen hast. Wenn dein Bad ohne Dusche etwa 50 % Luftfeuchtigkeit hat, versuch es mit 60 % als Startwert. Lege dafür einen **Setze Zahl-Variable**-Block an und nenne ihn zum Beispiel *Grenzwert*.



Dein Programm wird verschiedene Zustände erkennen – also Leerlauf und Duschen. Dafür brauchst du noch eine weitere Variable. Die kannst du beispielsweise *Status* nennen. Gib ihr den Wert 0.



Nun geht es an die Schleife. Den wichtigsten Wert holst du dir über die Variable *Feuchte*, die vom Umweltsensor gespeist wird. Nimm auch hier einen **Setze Zahl-Variable**-Block.

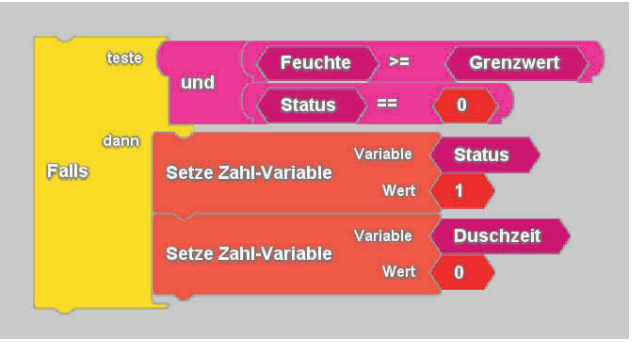
Danach beginnst du mit einem Vergleich. Nutze dafür einen **Falls**-Block und teste zwei Bedingungen:

1. Liegt die aktuelle Feuchte über deinem Grenzwert?
2. Ist der Status aktuell auf 0?

Dafür legst du den **und**-Block aus dem Reiter *Logische Vergleiche* unter *teste* an. Dieser legt fest, dass beide Bedingungen erfüllt sein müssen, damit etwas passiert.

Füge zwei logische Vergleiche hinzu und stell die Bedingungen ein:

1. Bedingung: *Feuchte >= Grenzwert*
2. Bedingung: *Status == 0*

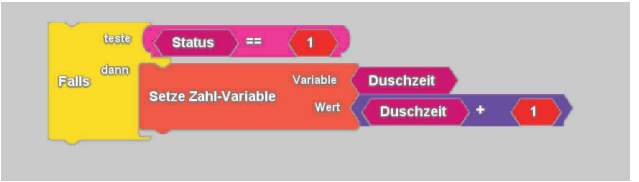


Unter *dann* legst du jetzt fest, was genau passieren soll. Mit **Setze Zahl-Variable** veränderst du die Variable *Status*. Die wird jetzt auf 1 gesetzt – das Duschen hat begonnen! Und du führst eine neue Variable ein, die *Duschzeit*. Diese setzt du hier zunächst auch auf 0, damit das Programm sauber starten kann.

Duschzeit messen

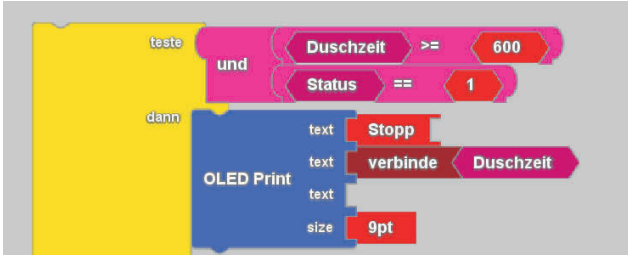
Jetzt legst du fest, was passiert, wenn das Programm in den Status 1 springt. Das geht mit einem neuen **Falls**-Block. Hier erfasst du die Duschzeit mit einem Timer. Es müssen also wieder Sekunden gezählt werden, und am Ende der Schleife taktet ein **Delay** auf 1 Sekunde – das kommt später.

Zunächst legst du einen logischen Vergleich unter *teste* an. Prüfe, ob *Status == 1* ist. Unter *dann* legst du einen **Setze Zahl-Variable**-Block. Dieser soll einen neuen Wert in die Variable *Duschzeit* eintragen.
Als neuen Wert setzt du die mathematische Operation *Duschzeit + 1*. Zusammen mit dem **Delay** am Ende werden so wieder die Sekunden gezählt.



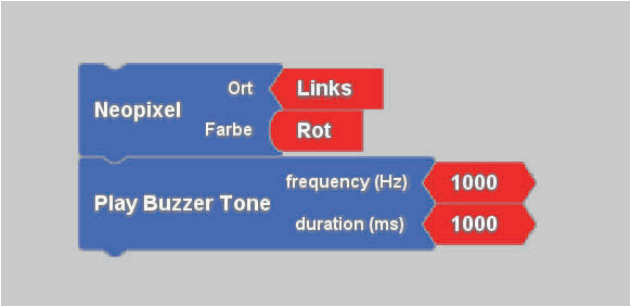
Nach dem Timer prüfst du, ob eine bestimmte Anzahl an Sekunden vergangen ist. Nimm also einen **Falls/sonst**-Block und lege ihn unter den Timer. Wir zäumen jetzt das Pferd von hinten auf und beginnen mit dem höchsten Wert: Teste mit einem **und**-Vergleich, ob die Duschzeit über 600 Sekunden (also 10 Minuten) liegt und ob der *Status 1* ist (also noch geduscht wird).

Also:
Duschzeit >= 600
Status == 1

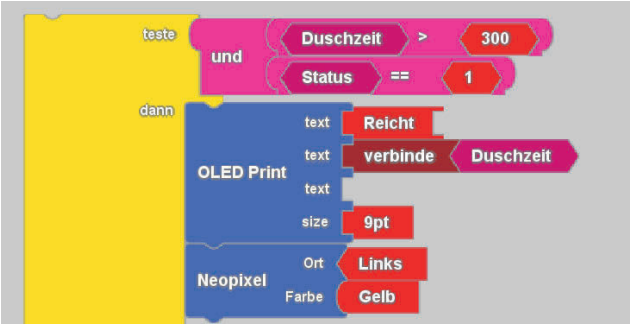


Wenn beide Bedingungen zutreffen, dann soll auf dem Bildschirm der Text *Stopp* angezeigt werden; darunter wird die Duschzeit angezeigt. Nimm dafür den **OLED Print**-Block. Lege in die erste Zeile deine Textmeldung („Stopp“) und in die zweite Zeile den **verbinde**-Adapter und lege die Variable *Duschzeit* an.

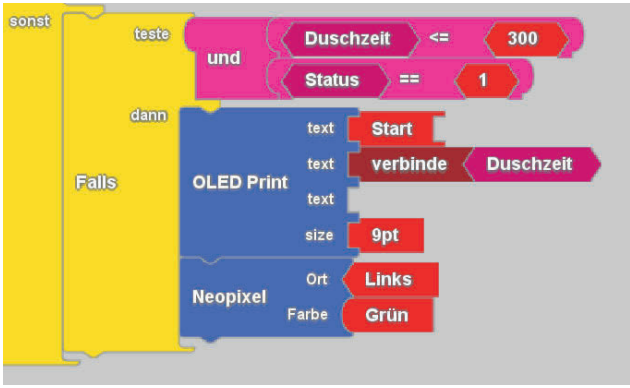
Damit es noch eindrucksvoller wird, hängst du unter den **Display**-Block einen **Neopixel**-Block (unter *Anzeigen / Aktoren*), der die LED rot färbt. Und weil es dann richtig dramatisch wird, kannst du auch noch einen *Play Buzzer Tone*-Block anlegen (den findest du unter dem Reiter *IoT: BT / Audio*). Setze die Frequenz (Hz) beispielsweise auf 1000 und die *duration* (Dauer) auf 1000 ms. Nun gibt es ein klares Signal, wenn 600 Sekunden überschritten sind.



Nun kommt unter *sonst* eine Verschachtelung. Lege einen weiteren ***Falls/sonst***-Block an, damit du Aktionen für die anderen Duschzeitwerte bestimmen kannst.

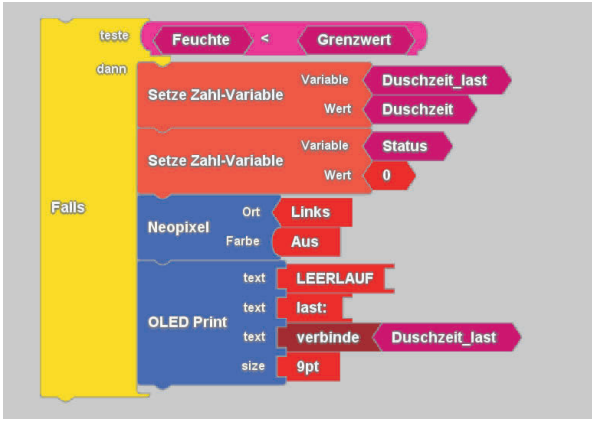


Du testest wieder mit einem **und**-Vergleich, ob die Duschzeit über 300 liegt (und *Status 1* ist). Dann gibt es über einen weiteren *OLED*-Block die Mitteilung „reicht“ und die LED färbt sich gelb.
Probiere das mal alleine aus oder schau auf das Bild der fertigen Schaltung – der Aufbau ist gleich, nur die Werte ändern sich.



Die letzte Möglichkeit legst du unter *sonst* ab. Hier prüfst du, ob die Duschzeit noch unter 300 liegt (bei *Status 1*). Ist dies der Fall, dann zeigt das Display *Start* an und die LED bleibt grün. Das ist der erste Zustand, der eintritt, wenn der *Status* auf 1 springt, das Duschen also beginnt.

Nun sollte auch eine kleine Statistik eingebaut werden. Diese folgt als weiterer **Falls**-Block, diesmal aber nicht verschachtelt, sondern unter dem bisherigen Programm.



Dort prüfst du, ob die Feuchte unter dem Grenzwert liegt. Wenn dieser Zustand eintritt, dann wird mit **Setze Zahl-Variable** die *Duschzeit* in die Variable *Duschzeit_last* geschrieben – das ist die alte Duschzeit.
Außerdem wird der *Status* wieder auf 0 gesetzt, die LED ausgeschaltet und auf dem OLED der Text „Leerlauf“ angezeigt, darunter die *Duschzeit_last*.

Damit setzt du das Programm zurück. Die letzte Duschzeit wurde erfasst, und die Schleife ist bereit für die nächste Messung, wenn die Feuchte wieder über den Grenzwert steigt.
Vergiss am Ende den **Delay**-Block nicht – er liefert den Takt für den Timer. Daher setzen wir ihn auf 1000 ms.

Nun ist deine Duschpetze einsatzbereit!
Teste sie gemeinsam mit deiner Familie und finde heraus, wie lang so manch eine Duschsession wirklich dauert – vielleicht gibt’s ja Überraschungen.

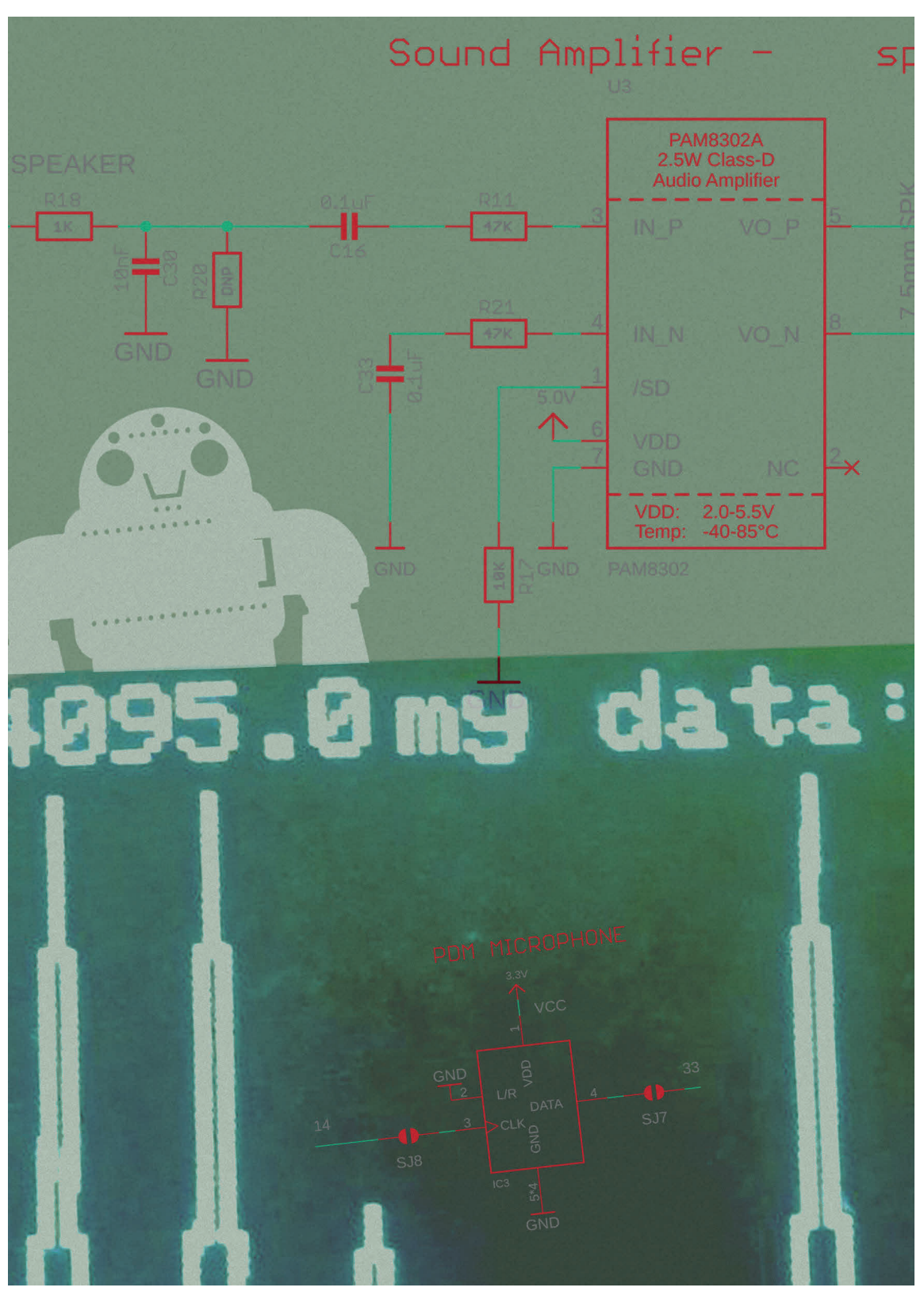
Tipp:
Automatisch einschaltende Lüfter oder geöffnete Fenster können Makey:Lab leicht verwirren, weil sie die Feuchtigkeit plötzlich verändern.
Am besten stellst du dein Gerät an einen Ort, der die natürliche Raumfeuchte gut mitbekommt, aber nicht direkt im Luftzug steht.

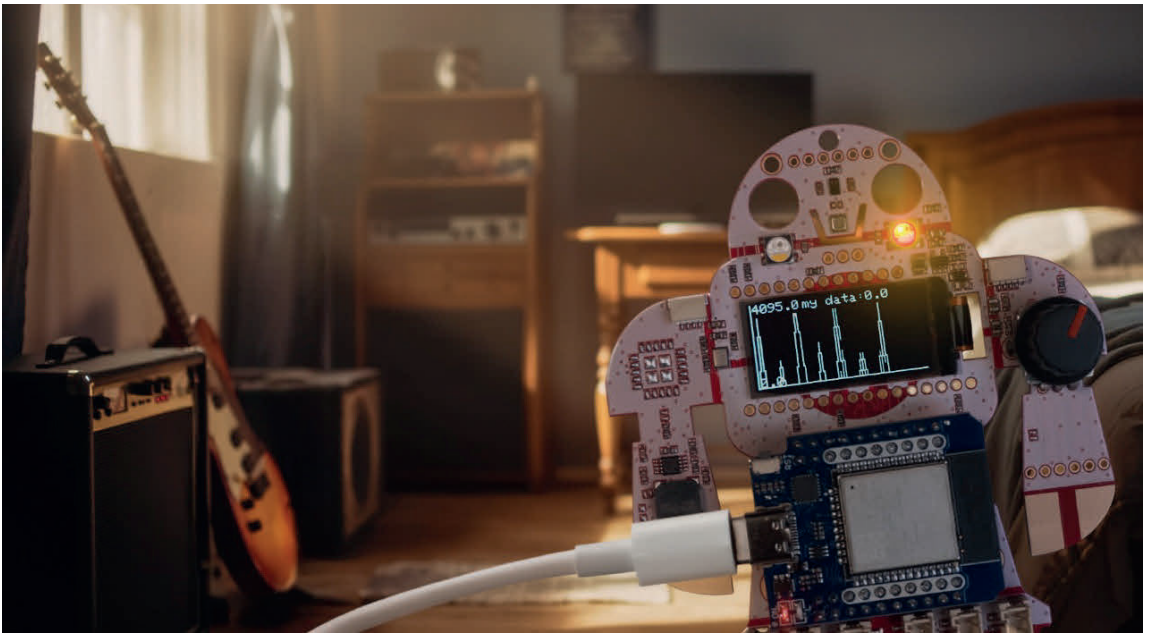
Du willst noch mehr wissen?

SCAN MICH

Hinter dem QR-Code findest du spannende Infos vom Umwelt-Campus Birkenfeld – über Luftfeuchtigkeit beim 3D-Druck und eine smarte Klimaüberwachung für Filament.

Weitere Infos gibt es im **Maschinenbau**-Studiengang und den Laboren für Additive Fertigung und Robotik am Umwelt-Campus Birkenfeld.


Sound Amplifier -
U3
PAM8302A
2.5W Class-D
Audio Amplifier
SPEAKER
R18
1K
0.1uF
C16
R11
47K
IN_P
VO_P
10nF
C30
R20
DNP
GND
GND
R21
47K
IN_N
VO_N
C33
0.1uF
/SD
5.0V
VDD
GND
NC
VDD: 2.0-5.5V
Temp: -40-85°C
GND
R17
10K
GND
PAM8302
4095.0 my data:
PDM MICROPHONE
3.3V
VCC
GND
L/R
VDD
DATA
CLK
GND
33
SJ7
14
SJ8
IC3
GND



#05 Die Lärmampel

So erkennst du, wann’s zu laut wird

In einem vollen Klassenraum kann es ganz schön laut werden – manchmal so laut, dass man die eigene Stimme kaum noch hört. Viele merken gar nicht, wie hoch der Lärmpegel wirklich ist. Aber zu viel Lärm ist nicht nur anstrengend – er kann sogar richtig ungesund sein!

Lärm bedeutet Stress – für unsere Ohren und für unser Gehirn. Schon ab etwa 85 Dezibel (das ist ungefähr so laut wie eine stark befahrene Straße) empfehlen Experten, dass man sich nur eine begrenzte Zeit in dieser Umgebung aufhält. In Büros, Schulen oder Werkstätten gibt es deshalb oft klare Regeln, wie laut es

sein darf – und wie lange. Im Berufsleben überwachen manchmal sogar spezielle Lärmmesser die Lautstärke.

Auch wir können den Lärmpegel überwachen. Denn dein Makey:Lab kann hören! Na ja, nicht wie ein Mensch – aber das eingebaute Mikrofon erkennt, wie stark die Luft um ihn herum vibriert. Mit ein paar einfachen Blöcken programmierst du deine eigene Lärmampel, die anzeigt, ob es gerade ruhig, etwas lauter oder schon zu laut ist.

Und das Beste: Du kannst dir die Geräusche sogar auf dem Display anzeigen lassen – als Kurve, die den Lärmverlauf sichtbar macht. So wird aus einem unsichtbaren Phänomen wie Lärm plötzlich etwas, das du messen, anzeigen und verändern kannst.

Vielleicht willst du deine Lärmampel in der Klasse einsetzen? Oder beim Zocken mit Freunden? Oder wenn ihr einfach mal testen wollt, wer die lauteste Musikbox hat.

Was ist eigentlich Dezibel?

Geräusche misst man in Dezibel – abgekürzt: dB. Je höher der Wert, desto lauter wird es. Unser Ohr kann riesige Unterschiede wahrnehmen – vom leisesten Flüstern bis zum dröhnenden Konzert.

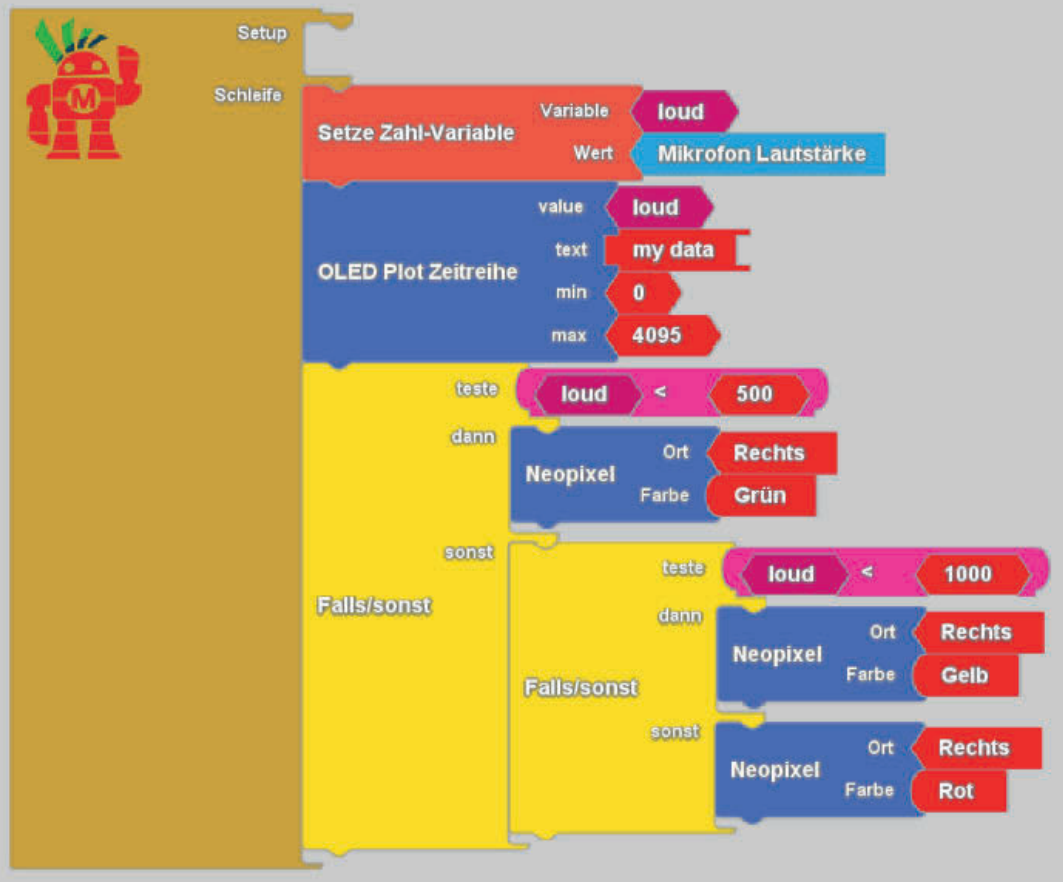
Hier ein paar Beispiele für Geräusche und ihre Lautstärke:

Geräusch	Lautstärke (ca.)
Atemgeräusch	10 dB
Flüstern	30 dB
Normales Gespräch	60 dB
Staubsauger	70 dB
Straßenverkehr (laut)	85 dB
Rockkonzert	100–110 dB
Presslufthammer	120 dB
Flugzeugstart (nah)	130 dB
Schmerzgrenze	ab 120 dB
Explosion	140 dB und mehr

Wichtig: Schon ab etwa 85 dB kann Lärm auf Dauer das Gehör schädigen. Deshalb ist Gehörschutz z. B. auf Baustellen Pflicht!

Schon gewusst? +10 Dezibel fühlt sich für unsere Ohren etwa doppelt so laut an – obwohl das Messgerät nur ein bisschen mehr anzeigt! Das liegt daran, dass unser Ohr laute Geräusche anders wahrnimmt als leise.

bau es einfach so



Makey:Lab überwacht den Lärm

In diesem Projekt wollen wir Makey:Lab dazu nutzen, den Lärmpegel zu überwachen und darauf zu reagieren. Lass uns mal schauen, welche Blöcke wir dazu brauchen.

Der Aufbau kommt dir vielleicht bekannt vor: Wir messen wieder einen Sensorwert – diesmal vom Mikrofon – und zeigen ihn auf dem Display als Kurve an. Zusätzlich steuern wir eine LED-Ampel, die bei Lärm grün, gelb oder rot leuchtet – je nachdem, wie laut es gerade ist.

Schritt für Schritt

Zunächst müssen wir unseren Sensorwert einlesen. Dafür speichern wir den Wert in einer Variable. In unserem Beispiel nennen wir sie *loud* - als Abkürzung für *loudness*, also Lautstärke.

Wir setzen die Variable loud.

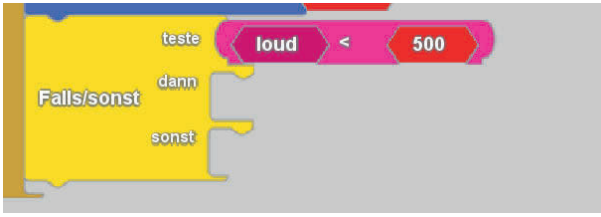
Nutze dafür wieder den **Setze Zahl-Variable** Block. Dort änderst du den Variablen-Namen und holst dir aus dem Reiter Sensoren den Block Mikrofon Lautstärke. Diesen kannst du als Wert deiner Variable einhängen.

Als nächstes nehmen wir den **OLED Plot Zeitreihe Block**. Er soll nun den Wert aus der Variable *loud* anzeigen, dafür musst du sie unter *value* ablegen. Du kannst die loud-Variable entweder mit Rechtsklick klonen oder du ziehst dir den Block Name Integer Variable in dein Programm und benennst ihn entsprechend um.

Unter Text kannst du festlegen, wie unsere Kurve auf dem Display beschriftet werden soll. Als *min* und *max* nimmst du 0 und 4095. Das entspricht dem Wertebereich, den das Mikrofon messen kann. Je höher der Wert, desto lauter.

Den Verlauf der Lautstärke zeigst du so in einer Zeitreihe.

Nun zeigt unser Programm bereits einen Lautstärkeverlauf auf dem Display an. Um allerdings auf verschiedene Lärmbereiche zu reagieren, müssen wir diese noch festlegen. Dafür brauchen wir zwei verschachtelte **Falls/sonst**-Blöcke, die verschiedene Bedingungen prüfen – das ist uns Luft-Detektiven bestimmt noch aus dem vorherigen Kapitel bekannt, sonst schlag nochmal nach.



Es kommt wieder ein Falls/sonst-Block zum Einsatz.

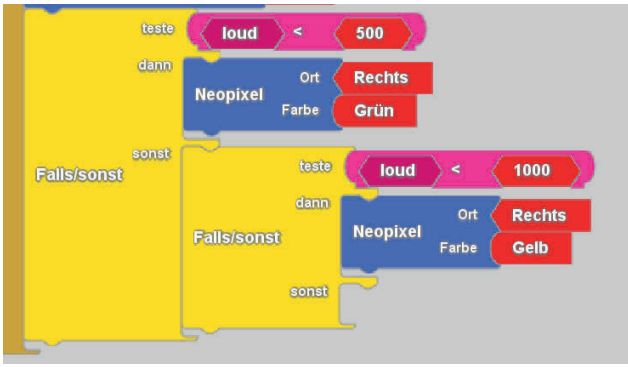
Den ersten legen wir unten an unser Programm an. Er soll prüfen, ob die Lautstärke unter dem Wert 500 ist. Ist das der Fall, dann soll die LED grün leuchten.

Für die Prüfung nimmst du den passenden Block (rund) aus *logische Vergleiche* und legst ihn unter teste ab. Als erste Variable wählst du *loud* und legst sie im **Vergleichs**-Block ab. An zweite Stelle kommt eine einfache Zahlenvariable. Hier trägst du *500* ein.

Je nach Wert soll eine LED angehen.

Als Aktion legst du unter *dann* einen **Neopixel**-Block ab. Hier kannst du unter Ort festlegen, welche der beiden

LEDs leuchten soll und in welcher Farbe. Da dies unser leisester Wertebereich ist, soll hier die Farbe *Grün* erscheinen.



Nun haben wir schon zwei Lautstärke-Bereiche definiert.

Nach Prüfung und Aktion müssen wir dem Programm unter sonst sagen, was passieren soll, wenn der Wert über *500* liegt.

Wir legen hier einen zweiten **Falls/sonst**-Block ab. Dieser prüft dann, ob die Lautstärke unter dem Wert *1000* liegt. Ist die Bedingung erfüllt, soll die LED *gelb* leuchten. Trifft es nicht zu, liegt die Lautstärke also über dem Wert *1000*, dann soll die Farbe *Rot* angezeigt werden.

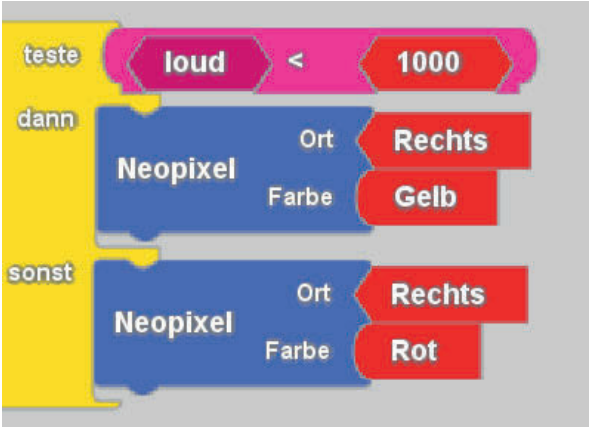
Nun ist auch unsere Abfrage fertig:

- Wenn es leise ist, bleibt die LED grün – das bedeutet: alles in Ordnung.
- Wird es lauter, springt die LED auf gelb – eine Warnung: „Jetzt aufpassen!“
- Und wenn’s richtig laut wird, leuchtet die LED rot – Jetzt wird’s zu viel!

Die Ampel zeigt dir auf einen Blick, wie laut es gerade ist. Das ist sinnvoll, da sich das Gehör an eine laute Umgebung gewöhnen kann. Dann bekommst du quasi gar nicht mehr mit, dass deine Ohren gerade belastet werden. Wenn man zu lange lauten Geräuschen ausgesetzt ist, dann kann das Gehör sogar dauerhaft Schaden nehmen. Makey:Lab hilft dir, rechtzeitig zu merken, wenn es zu laut wird – so kannst du dein Gehör besser schützen.

Über deine Lärmkurve kannst du auch einen zeitlichen Verlauf sehen. Sobald du das Programm auf Makey:Lab übertragen hast, kannst du ja mal klatschen und beobachten, was auf dem Display passiert.

läuft... irgendwie



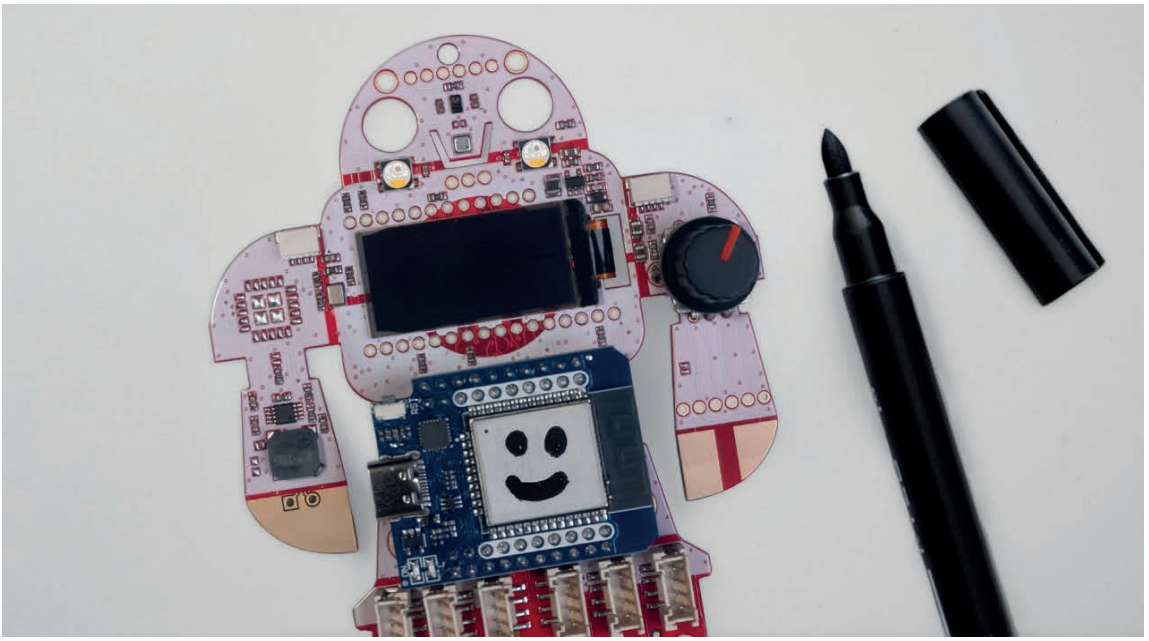
Wenn die Bedingungen nicht zutreffen, leuchtet es rot.

Deine Ideen sind gefragt: Was könnte Makey:Lab noch machen, wenn es zu laut wird?

Du willst noch mehr wissen?

SCAN MICH

Hinter dem QR-Code findest du ein spannendes Video aus dem Akustiklabor des Umwelt-Campus Birkenfeld – mit Einblicken in Schallmessung, Lärmbewertung und den Studiengang **Umweltplanung (B.Sc.)**.



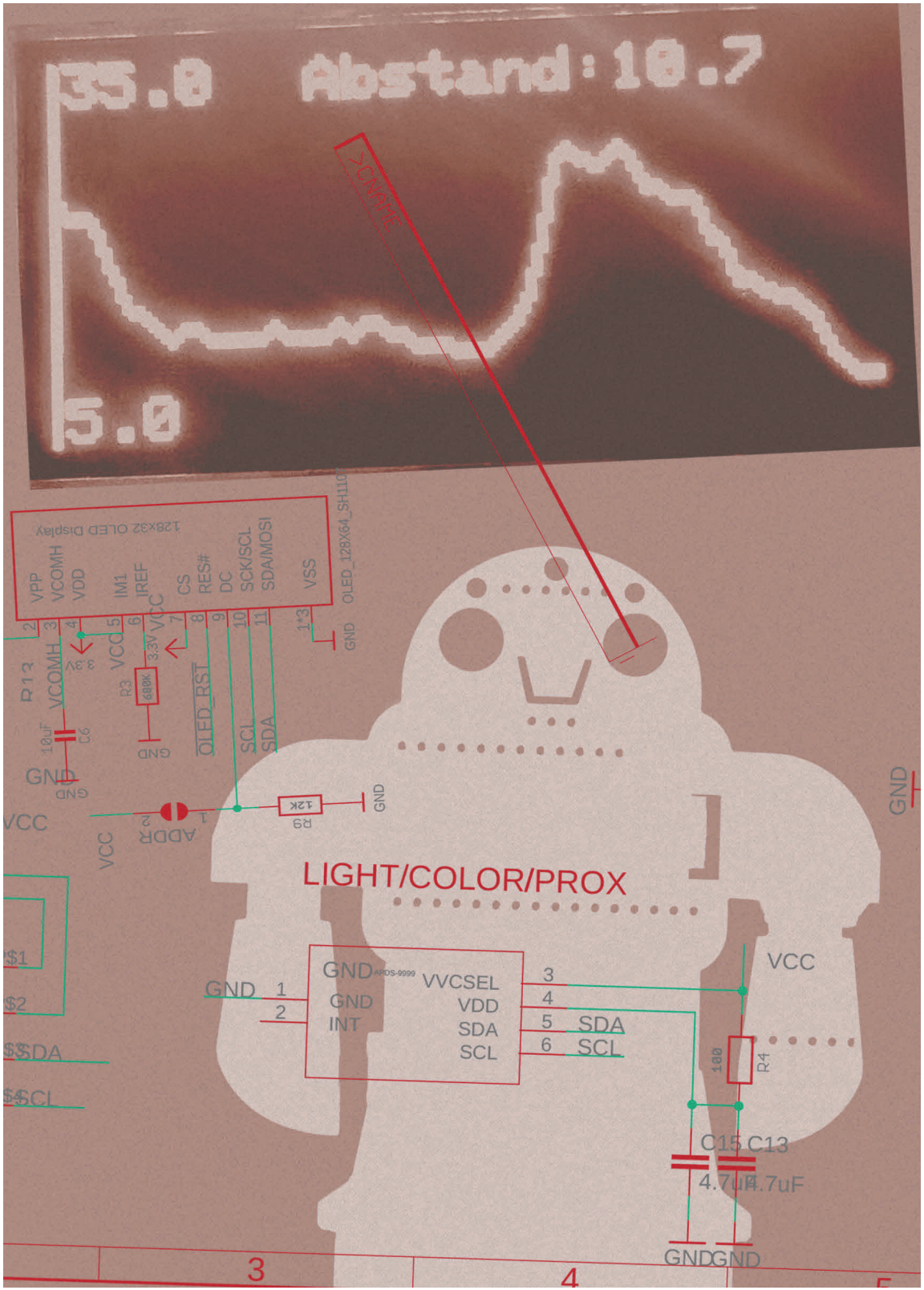
Zwischenruf: Jetzt wird’s persönlich

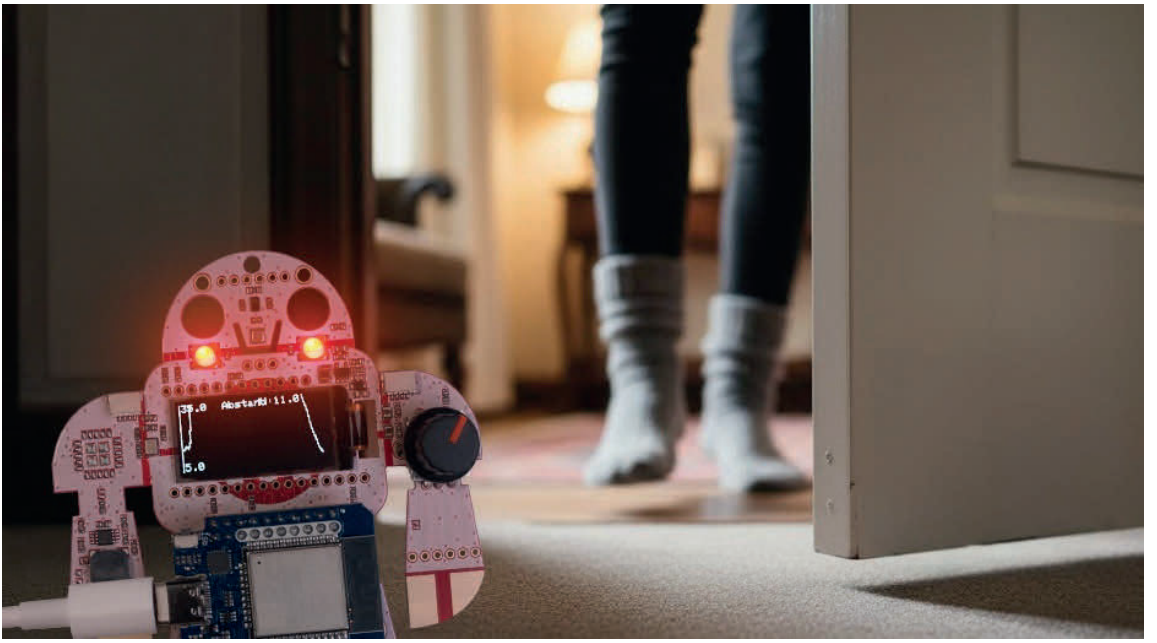
So bekommt dein Makey:Lab Charakter

Dein Makey:Lab ist dein persönlicher Laborroboter. Und der darf ruhig ein bisschen Charakter zeigen! Nimm dir einen Permanentmarker und verpass ihm ein Gesicht, ein Symbol oder einfach ein paar lustige Striche. Die große silberne Fläche auf dem Modul – das ist der Aluminiumdeckel des ESP-Chips – eignet sich perfekt dafür.

Ob Smiley, Rakete oder Monogramm – mach’s so, dass du dein Makey:Lab unter tausend anderen wiedererkennst. Denn das hier ist dein Labor – und das sollen ruhig alle wissen! Technik darf Spaß machen.

Wichtig: Bitte nur auf der silbernen Fläche malen, nicht auf Pins oder Bauteile. Alles, was leitfähig ist, muss frei bleiben.


35.0
Abstand: 10.7
5.0
>NAME
128X32 OLED Display
VPP
VCOMH
VDD
IM1
IREF
CS
RES#
DC
SCK/SCL
SDA/MOSI
VSS
OLED_128X64_SH1106
GND
VCC
3.3V
R3
680K
OLED_RST
SCL
SDA
10uF
C6
R9
12K
ADDR
LIGHT/COLOR/PROX
APDS-9999
GND
INT
VVCSEL
VDD
SDA
SCL
R4
100
C15
C13
4.7uF
4.7uF
3
4



#06 Zimmer-Alarmanlage

Jetzt merkst du, wenn jemand in dein Zimmer kommt

Makey:Lab hat mit dem **APDS-9999** einen besonders spannenden Sensor an Bord. Er kann nicht nur Farben erkennen, sondern auch Abstände messen – und das eröffnet jede Menge Möglichkeiten.

Mit so einer Abstandsmessung lässt sich zum Beispiel ganz einfach eine kleine Alarmanlage bauen. So kannst du dein Zimmer vor ungebetenen Gästen schützen: Sobald sich die Tür öffnet, schlägt dein Makey:Lab Alarm und vertreibt Eindringlinge!

Wie das funktioniert, schauen wir uns jetzt mit dem nächsten Programm an.

so läuft's

Setup
Schleife
OLED Plot Zeitreihe
value
Farbe, Abstand, Licht APDS9999 Sensor Abstand (cm)
text
Abstand
min
5
max
35
Falls/sonst
teste
Farbe, Abstand, Licht APDS9999 Sensor Abstand (cm) < 10
dann
Neopixel
Ort
Rechts
Farbe
Rot
Neopixel
Ort
Links
Farbe
Rot
Play Buzzer Tone
frequency (Hz)
1000
duration (ms)
5000
sonst
Neopixel
Ort
Rechts
Farbe
Aus
Neopixel
Ort
Links
Farbe
Aus
Delay (warte)
ms
20

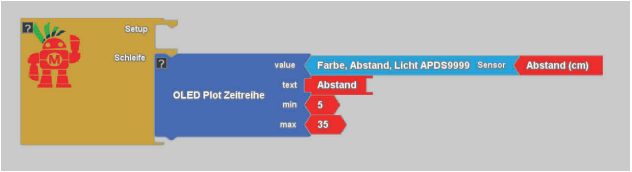
Abstand erkennen

Damit du besser verstehst, wie der Sensor arbeitet, zeichnen wir zuerst den gemessenen Abstand als laufende Kurve auf dem Display auf.

Anschließend legst du verschiedene Abstandsbereiche fest – und sorgst dafür, dass die LEDs passend leuchten und bei Annäherung ein kleiner Alarm ertönt.

Der Aufbau ist ziemlich einfach: Das Programm muss sich keine Werte merken und braucht deshalb auch keine Variablen. Es zeigt einfach nur den aktuellen Messwert – und reagiert sofort darauf.

Damit du den Sensorwert auf dem Display siehst, nutzt du wieder den **OLED Plot Zeitreihe**-Block.



Diesen verbindest du unter *value* mit dem **Farbe, Abstand, Licht APDS9999**-Block - den findest du unter dem Reiter *Sensoren*. Dort kannst du direkt als Ausgabe *Abstand (cm)* auswählen.

Damit man sofort erkennt, was die Zeitreihe zeigt, schreibst du im **Zeitreihen**-Block unter *text* die

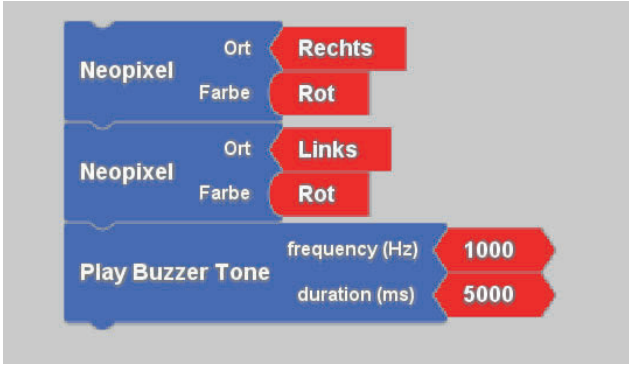
Beschriftung „Abstand“. Den Wertebereich kannst du zwischen 5 und 35 eingrenzen – das entspricht dem Messbereich in Zentimetern.

Jetzt beginnt die eigentliche Prüfung:
Wir unterscheiden zwei Zustände – entweder liegt die Entfernung über 10 Zentimeter oder sie liegt darunter.

Lege unter den **Zeitreihen**-Block einen **Falls/sonst**-Block. Unter *teste* prüfst du mit einem logischen Vergleich, ob der Sensorwert Abstand unter 10 Zentimeter liegt.

Also: *Abstand (cm) < 10*

Ist das der Fall, sollen die LEDs rot leuchten. Füge dafür zwei **Neopixel**-Blöcke (die sind zu finden bei *Anzeigen / Aktoren*) unter *dann* ein – einen für *Rechts*, einen für *Links* – und wähle bei beiden die Farbe *Rot*. So erkennst du sofort, wenn der Abstand unterschritten wird.



Damit wir einen schönen Alarm bekommen, fügst du unter den **Neopixel**-Blöcken noch einen **Play Buzzer Tone**-Block ein. Du findest ihn unter dem Reiter *IoT: BT / Audio*.

Stelle die Frequenz auf *1000 Hz* und die Dauer auf *5000 ms* (also fünf Sekunden) – das sollte lang genug sein, um wirklich jeden zu alarmieren!

Nun legst du fest, was passiert, wenn der Abstand über 10 Zentimeter liegt – also wenn niemand im Raum erkannt wird. Dann bleibt der Alarm still: Lichter aus! Lege dafür unter *sonst* zwei **Neopixel**-Blöcke an. Stelle bei beiden die Farbe auf *Aus* und wähle erneut *Rechts* und *Links* als Position.

Damit ist dein Abstands-Alarm fast fertig.

Zum Schluss fügst du noch ein **Delay**-Block ein – damit entschleunigst du die Messung ein wenig. Probiere für

den Anfang 20 ms. Ohne diese Pause würde dein Makey:Lab im Dauerlauf messen, und das braucht unsere Alarmanlage nicht. Mit dem **Delay** läuft alles ruhiger und du sparst sogar etwas Strom.

Teste ruhig ein wenig, um den passenden Wert für dein Setup zu finden. Je nach Raumgröße oder Aufbau kann es helfen, das Delay leicht zu verändern, bis die Messung stabil reagiert.

Alarm platzieren

Jetzt geht's an die Aufstellung: Wo erkennt dein Alarm Eindringlinge am besten? Probier's einfach aus!

Der **APDS-9999** sitzt auf der Vorderseite deines Makey:Lab – der kleine schwarze Kasten zwischen den Augen. Die kleine schwarze Öffnung ist seine "Linse". Richte sie auf den Bereich, den du überwachen möchtest.

Am einfachsten klappt es, wenn du dein Makey:Lab beispielsweise an einen Stapel Bücher lehnst und direkt neben die Tür stellst. Sobald jemand den Raum betritt, huscht die Türkante am Sensor vorbei – und der Alarm wird ausgelöst.

Wenn du das Gerät leicht schräg aufstellst, erfasst es die Bewegung beim Öffnen besonders zuverlässig. Ziemlich

einfach, oder? Pass nur auf, dass die Tür dein Makey:Lab nicht vom Boden fegt.

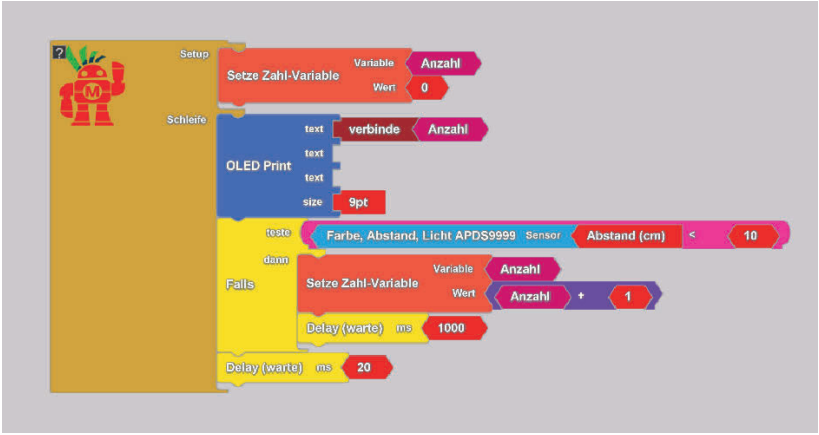
Tipp: Der Sensor erkennt helle, matte Oberflächen am zuverlässigsten.
Ist deine Tür sehr dunkel, hilft schon ein kleines Stück weißes Papier auf Höhe deines Makey:Labs. Das kannst du mit etwas Klebeband anbringen.

Von außen fällt das kaum auf – aber dein Sensor erkennt den Abstand damit viel präziser.

Weitere Ideen
Jetzt kannst du schon unerwünschte Eindringlinge vertreiben – aber natürlich lässt sich deine Alarmanlage noch weiter ausbauen!

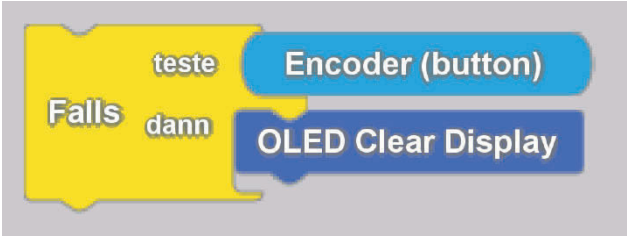
Wie wäre es zum Beispiel mit einem Stealth-Modus? Dabei lässt du einfach die Zeitreihe und die LEDs weg – so bleibt dein Alarm völlig unauffällig, bis er plötzlich losgeht.

Oder du programmierst einen stillen Alarm, der mitzählt, wie oft jemand dein Zimmer betreten hat.
Dafür legst du eine Variable an, die bei jedem Alarm um 1 erhöht wird. Damit das Zählen zuverlässig klappt, solltest du den zeitlichen Abstand zwischen deinen Messungen (**Delay**) etwas vergrößern, sodass wirklich nur ein Ereignis erfasst wird, wenn sich die Tür öffnet.



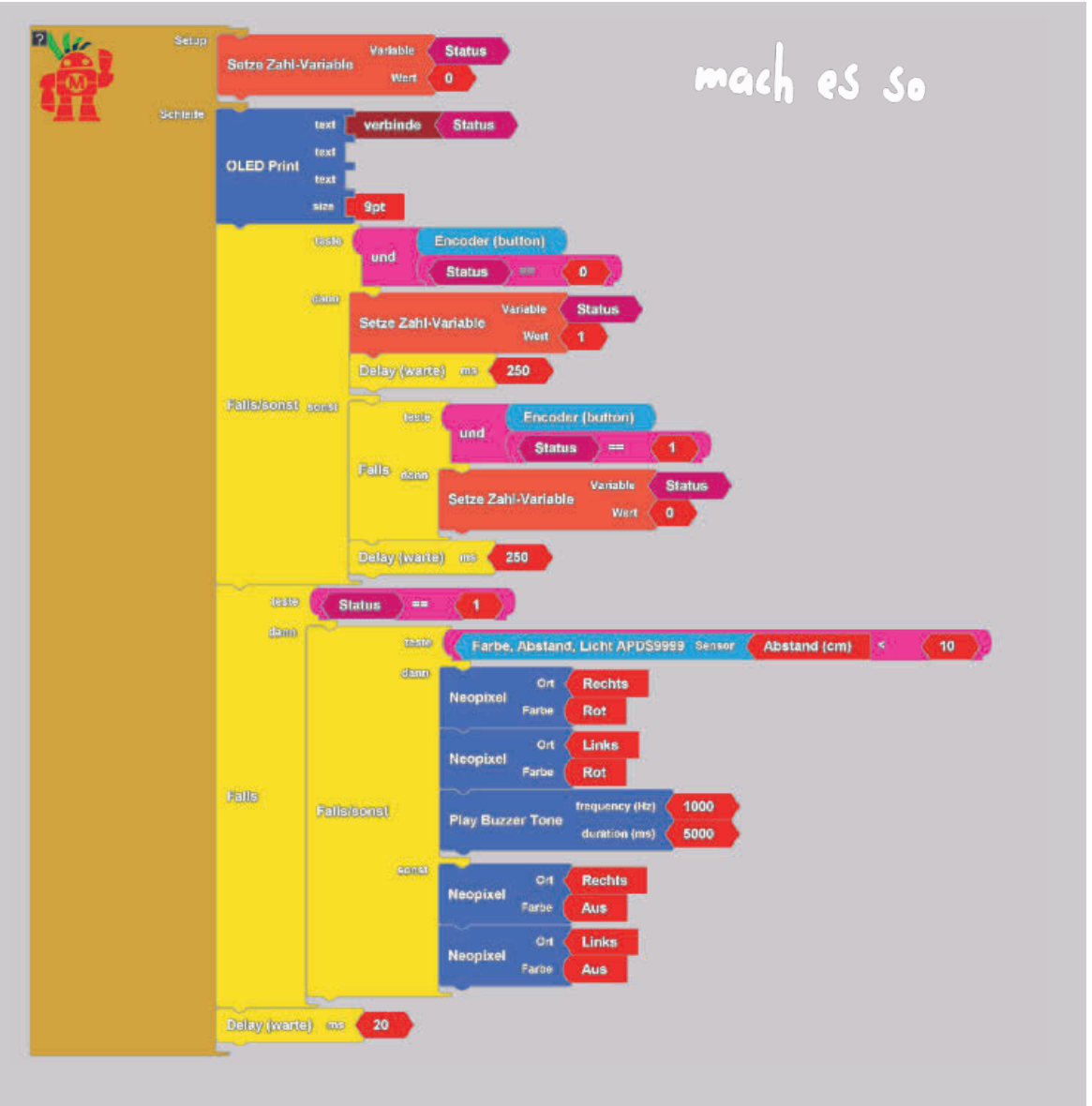
Tipp: Wenn du möchtest, dass du einen stillen Alarm hast – das Display also leer bleibt, dann kannst du den **OLED Print**-Block einfach weglassen. Das IoT-Logo nach Programmstart verschwindet von alleine. Es gibt auch die Möglichkeit das Display im Programm gezielt zu leeren. Dafür kannst du den **OLED Clear Display**-Block nutzen – du findest ihn unter dem Reiter *Externe Interfaces*.

Hier aktivieren wir den Block durch einen Druck auf den Encoder:



Alarm scharf schalten

Du kannst auch einen Status einbauen, mit dem du deine Alarmanlage manuell aktivierst. So bleibt sie beim Aufbau zunächst stumm und lässt sich erst durch einen Druck auf den Encoder scharf schalten. Das ist besonders praktisch,



wenn du Makey:Lab gerade positionierst – sonst piept dir der Alarm schon um die Ohren, bevor du fertig bist.

Erstelle dafür eine neue **Zahlenvariable**, zum Beispiel *Status*, und setze sie auf 0 (aus).
Wenn der Encoder gedrückt wird, änderst du den Wert auf 1 (scharf).
Im Programm prüfst du dann zusätzlich, ob Status = 1 ist, bevor der Alarm ausgelöst wird.

So kannst du dein Makey:Lab entspannt aufstellen – und erst danach die Falle zuschnappen lassen.

Flexibler Abstand

Du kannst das Programm auch so anpassen, dass keine feste Entfernung im Code steht, sondern du den Auslöseabstand erst beim Aufstellen festlegst.

Dafür legst du eine Variable an, in der du den aktuellen Messwert speicherst – zum Beispiel durch einen Druck auf den Encoder.

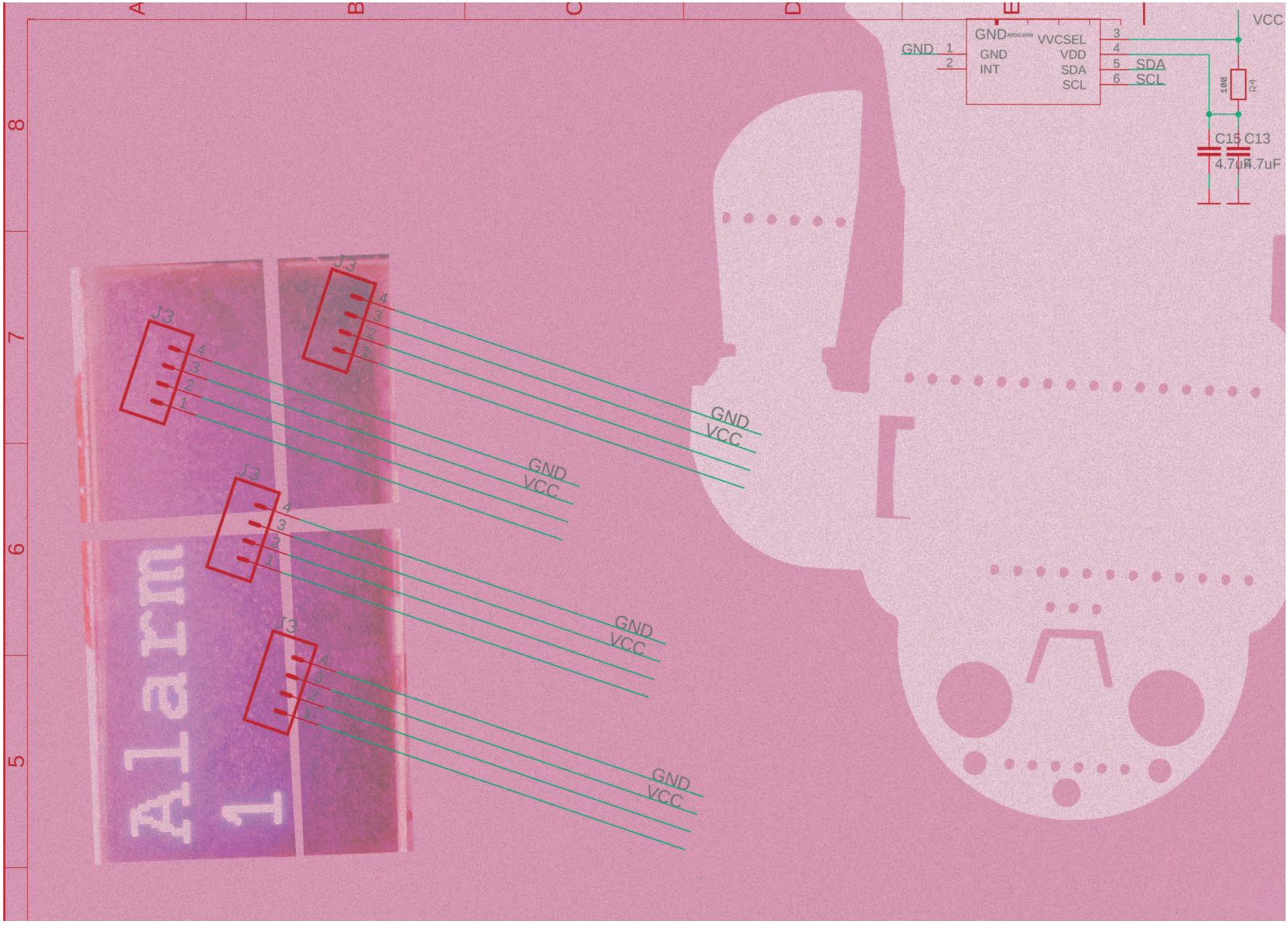
In der Schleife vergleichst du dann deinen *Abstand_manuell* mit dem aktuellen Abstand. Ist der aktuelle Wert beispielsweise um 2 cm kleiner, wird der Alarm ausgelöst.

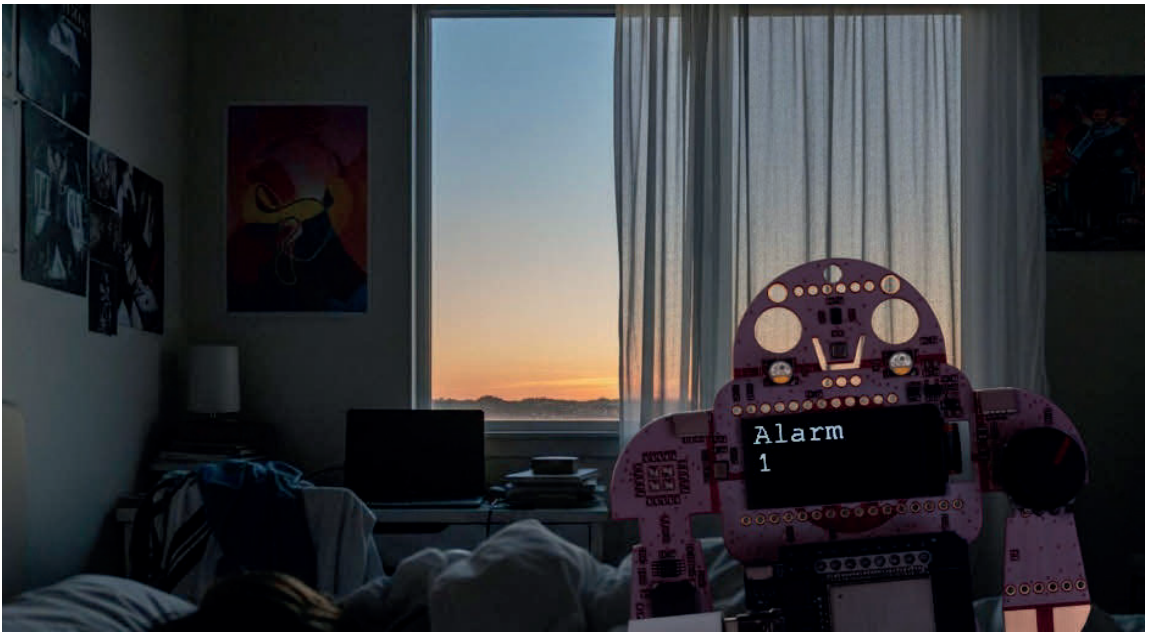
Das hat den Vorteil, dass du beim Aufbau freier bist und etwas experimentieren kannst. Diese Variante aktiviert sich auch erst, wenn du den Referenzabstand einmal festgelegt hast – das Scharfstellen ist also direkt integriert.

Du willst noch mehr wissen?

SCAN MICH

Hinter dem QR-Code findest du Projekte vom Umwelt-Campus Birkenfeld – zu Ultraschall-Abstandsmessung, Robotik und selbst gebauten Starkregenpegeln. Mehr dazu im Studienbereich **Maschinenbau und Informatik**.


A
B
C
D
E
8
7
6
5
GND
GND
INT
VVCSEL
VDD
SDA
SCL
3
4
5
6
SDA
SCL
VCC
R1
100
C15
C13
4.7uF
4.7uF
J3
J3
J3
J3
GND
VCC
GND
VCC
GND
VCC
GND
VCC
Alarm
1



#07 Tageslicht-Wecker

Aufstehen? Erst, wenn's hell wird

Ein lärmender Wecker am Morgen – das kann ganz schön störend sein. Vor allem, wenn er uns viel zu früh aus dem Schlaf reißt.

Rein biologisch richtet sich der Mensch eigentlich nach dem **Tageslicht**: Wird es draußen heller, werden wir langsam wach.

Sollten wir dann nicht auch einen Wecker haben, der sich nach dem Licht richtet? Einen, der uns erst dann weckt, **wenn es wirklich Tag wird** und die Sonne aufgeht?

Das kann auch *Makey:Lab*!
Der **APDS-9999-Sensor** auf deinem Board kann nämlichnicht nur Entfernungen messen – er ist auch ein

prima **Lichtmesser**, fast wie ein kleines elektronisches Auge.

Und genau das nutzen wir jetzt:
Wir bauen einen **Tageslicht-Wecker**, der erst dann lospiept, wenn es hell genug ist.
Zuerst entsteht eine einfache Version, und danach – wenn du Lust auf mehr hast – kommt die **Pro-Variante**, die die Uhrzeit aus dem Internet holt, um sie zu vergleichen.

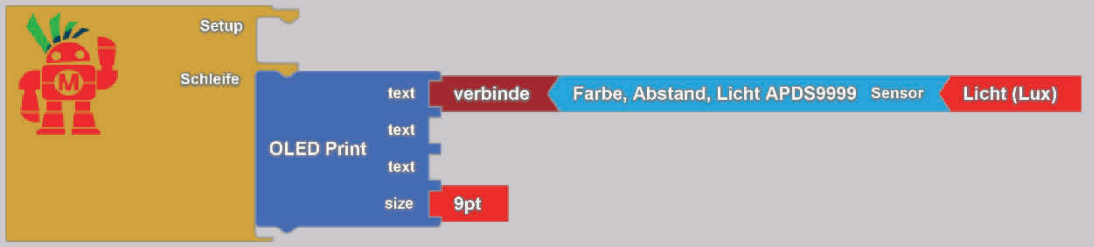
Erste Versuche

Damit du den Sensor besser kennenlernen kannst, bauen wir zunächst ein paar einfache Testprogramme.

Für den Anfang schauen wir uns den Helligkeitswert einmal an, den uns der Sensor liefert.

Ziehe hierfür zunächst einen **OLED Print**-Block in deine Programmschleife.

Um die Helligkeit auslesen zu können, braucht es nicht viel. Du kannst dafür wieder den Sensorblock (**Farbe, Abstand, Licht APDS9999**) nehmen, mit dem wir bereits den Abstand gemessen haben. Nur wählst du hier als Ausgabe dieses Mal *Licht (Lux)* aus. Dann legst du ihn mit einem **verbinde**-Block an deine erste Textzeile des **OLED**-Blocks an.



Nun kannst du das Programm starten und siehst auf dem Bildschirm den Helligkeitswert in Lux. Was genau das für ein Wert ist, schauen wir uns jetzt an.

Was bedeutet eigentlich „Lux“?
Lux ist die Einheit für **Beleuchtungsstärke** – sie beschreibt, wie viel Licht auf eine Fläche fällt: Je heller es ist, desto höher ist der Lux-Wert.

Vielleicht hast du auch schon einmal von *Candela* gehört – einer anderen physikalischen Größe, wenn es um Licht geht. Sie gibt an, wie stark eine Lichtquelle leuchtet, also wie viel Licht sie aussendet. Lux dagegen zeigt, wie viel davon tatsächlich auf einer Fläche ankommt.

Unser Sensor macht genau das: Er misst, wie viel Licht ihn trifft.
Damit können wir erkennen, ob es Tag oder Nacht ist – oder ob du gerade mit der Taschenlampe draufhältst.

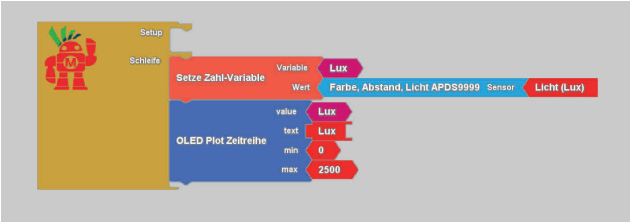
Typische Lux-Werte

Umgebung / Lichtquelle	Typischer Lux-Wert
Vollständige Dunkelheit (abgedeckt)	< 1 Lux
Dämmerung im Zimmer	5 – 50 Lux
Innenraum mit normaler Beleuchtung	300 – 500 Lux
Schreibtischlampe direkt auf Sensor	800 – 1500 Lux
LED-Taschenlampe / Handy-LED nah dran	2000 – 3000 Lux
Bewölkter Tag im Freien	10 000 Lux
Sonniger Sommertag (direkt in der Sonne)	60 000 – 100 000 Lux

Das kannst du mit deinem Testprogramm gleich mal ausprobieren. Welchen Wert siehst du jetzt gerade, was ist wenn du ein Licht anschaltest oder mit Makey:Lab ans Fenster gehst? Du kannst auch mal mit deiner Handy-Lampe auf den Sensor leuchten. Merk dir den hellsten Wert, den du misst.

Helligkeit im Verlauf

Nun schauen wir uns den Lichtwert in einem Zeitverlauf an. Dafür kommt erneut die **OLED Plot Zeitreihe** zum Einsatz. Damit können wir einen schönen Verlauf sehen - das ist noch leichter zu verstehen, als ein reiner Zahlenwert.



Ziehe den Block in deine Programmschleife. Als *value* soll hier der Sensorwert einfließen. Als *text* kannst du beispielsweise "*Lux*" schreiben. Bei den Grenzwerten müssen wir ein bisschen ausprobieren.

Wie du der Tabelle oben entnehmen kannst, sind die Unterschiede zwischen dunkel und hell sehr hoch. Damit deine Messung in der Zeitreihe korrekt wiedergegeben wird, müssen wir den Wertebereich etwas einschränken. Wenn du dich gerade im Haus befindest, dann wird sich der Lux-Bereich vermutlich zwischen 0 und 2500 befinden. Der hellste Wert ist beispielsweise, wenn du mit künstlichem Licht direkt auf den Sensor leuchtest. Du hast dir ja deinen Maximalwert gemerkt, den kannst du hier eintragen.

Wenn du das Programm nun startest, dann wird dir auf dem Display der Helligkeitsverlauf angezeigt - von ganz dunkel (halte einfach mal einen Finger auf den Sensor) bis zum normalen Umgebungslicht (wie vermutlich jetzt) und ganz hell (leuchte mit dem Handy drauf). Spannend ist auch der Verlauf, wenn du eine dimmbare Lampe nutzt oder wenn du die Vorhänge vor dem Fenster zuziehst oder

öffnest. Dann kannst du den Helligkeitsverlauf wunderbar ablesen.

Flexibel Lichtmesser

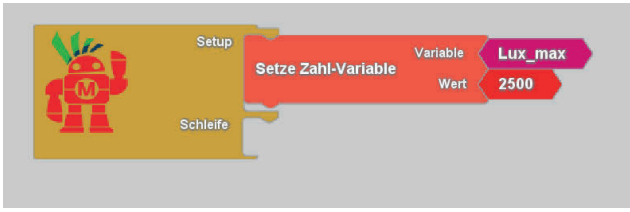
Du kannst dein Programm noch etwas flexibler machen – so, dass du den hellsten Lichtwert jederzeit direkt am Makey:Lab festlegen kannst.

Drückst du den Encoder-Button, merkt sich dein Makey:Lab den aktuellen Lichtwert als neuen Maximalwert.

Das nennt man Kalibrieren.

Du „kalibrierst“ dein Gerät also auf die tatsächlichen Lichtverhältnisse, ohne dafür den Code anpassen zu müssen. So passt sich das Makey:Lab automatisch an seine Umgebung an – egal, ob du in heller oder dunkler Umgebung arbeitest.

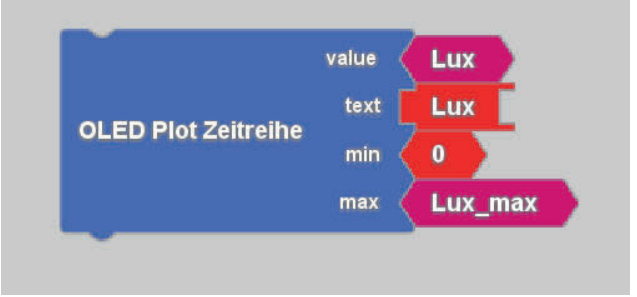
Dafür musst du zunächst mit dem **Setze Zahl-Variable**-Block eine neue Variable definieren. Nimm hierfür beispielsweise *Lux_max* und setze einen Standardwert, mit dem Makey:Lab startet. Das könnte vielleicht 2500 sein. Lege den Block in dein Setup. Diese Variable setzen wir später in unsere Zeitreihe ein.



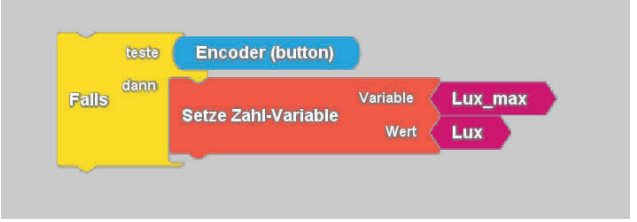
Damit du auch den aktuellen Helligkeitswert als Variable bekommst, lege in die Schleife nun einen weiteren **Setze-Zahl-Variable**-Block. Nenne die Variable *Lux* und als Wert nimmst du den Sensor-Ausgang *Licht (Lux)*.

Danach folgt der **OLED Plot Zeitreihe**-Block. Als *value* kannst du dieses Mal deine Variable *Lux* einsetzen. Als *text* nimmst du auch wieder *Lux* – das lässt du dir ja anzeigen.

Als Minimalwert lässt du 0 und als Maximalwert klonst du dir von oben deine *Lux_max* Variable und setzt sie ein.



Wenn das Programm nun startet, dann ist der Wert noch beim Standardwert 2500. Den wollen wir jetzt allerdings noch ändern, sobald der **Encoder** gedrückt wird.



Lege dafür einen **Falls**-Block ans Ende deines Programm. Unter *teste* legst du den **Encoder (Button)** ab. Als Aktion schreibst du einen neuen *Max*-Wert. Dafür brauchst du wieder einen **Setze Zahl-Variable**-Block. Als Variable klonst du dir *Lux_max* und unter *Wert* liest du den aktuellen Helligkeitswert ein. Dafür kannst du deine *Lux* Variable nehmen.

Nun funktioniert es so: Sobald du den Button drückst, ändert sich im **Zeitreihe**-Block der Maximalwert auf den

aktuellen Helligkeitswert. Zum Kalibrieren leuchtest du mit dem hellsten Licht in deiner Umgebung auf den Sensor und drückst den Button.

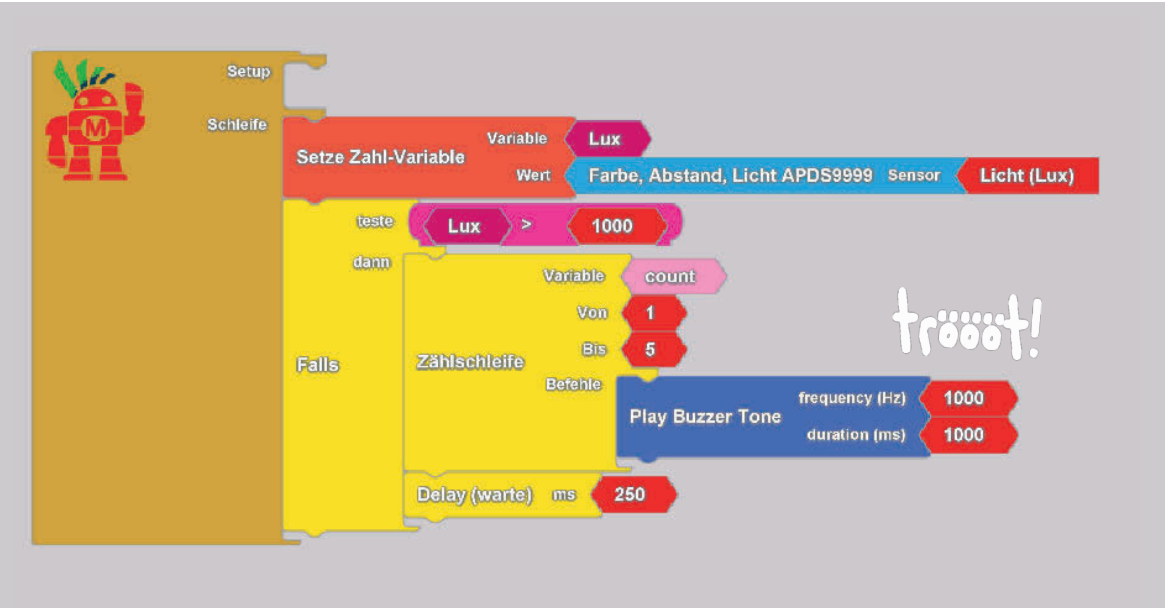
Nun zeigt dir die Zeitreihe die Werte von dunkel bis zu deinem neuen Maximalwert. Das hat den Vorteil, dass du deinen Wertebereich viel feiner siehst und Veränderungen besser erkennst.

Du schneidest quasi den gesamten oberen Bereich, den beispielsweise Tageslicht unter freiem Himmel produziert, ab.

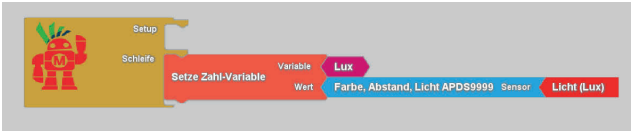
Damit hast du jetzt schon einen richtig coolen Lichtmesser für verschiedene Lichtsituationen.

Makey:Lab macht Alarm, wenn es hell wird

Nun kannst du dir deinen Tageslicht-Wecker bauen. Der Alarm soll losschrillen, sobald ein bestimmter Lux-Wert überschritten wird. Nach den ersten Testprogrammen ist das jetzt ganz einfach.

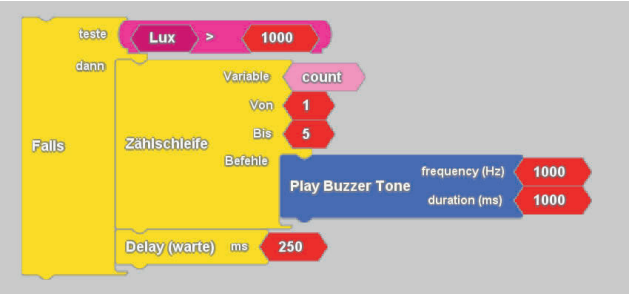


Erstelle ein neues Programm. In die Schleife legst du den **Setze Zahl-Variable**-Block. Darin definierst du die Variable *Lux* und speist sie wieder mit dem Sensorwert.



Dann holst du dir einen **Falls**-Block. Unter *teste* setzt du einen **logischen Vergleich** größer als (>), dort legst du zwei Werte ab: Links *Lux* und rechts eine Zahlvariable, die du zum Testen auf 1000 festlegen kannst. Passe den Wert so an, dass er größer ist als dein normales Umgebungslicht.

Also beispielsweise:
Lux > 1000



Als Aktion soll nun dein Alarm erklingen. Damit es etwas schöner ist, nutzen wir hierfür dieses Mal den **Zählschleife**-Block (unter *Kontrollstrukturen*). Darin kannst du festlegen, wie oft ein Befehl ausgeführt werden soll. Standardmäßig steht die hierfür vorgesehene Variable *count* bereits auf den Werten 1 bis 5 - also fünfmal. Unter Befehl legst du einfach deinen **Play Buzzer Tone**-Block ab. Stell die Frequenz auf 1000 Hz und die Dauer auf 1000 ms ein.

Damit zwischen den fünf Wiederholungen ein bisschen Zeit vergeht, folgt der **Zählschleife** ein **Delay** von beispielsweise ebenfalls 1000 ms.

Geht dein Alarm los, erklingt also erst 1 Sekunde lang ein Alarm, dann wird 1000 ms gewartet und es folgen noch 4 Wiederholungen.

Damit ist das Programm fertig. Du kannst es direkt starten. Makey:Lab sollte nun also ruhig starten, sobald du den Sensor allerdings anleuchtest, erklingt dein Alarmsound.

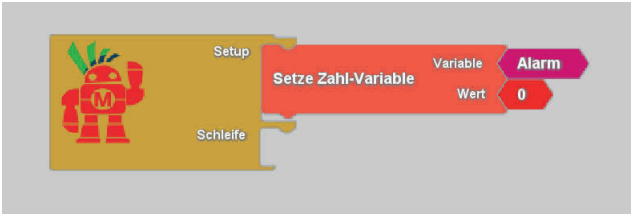
Nun wäre dein Tageslicht-Wecker eigentlich schon fertig. Im Moment reagiert er aber immer sofort, wenn es hell genug wird – egal, ob du gerade schläfst oder noch wach bist.

Damit er also **nicht schon beim abendlichen Licht anschaltet**, bauen wir noch einen kleinen Schalter ein. So kannst du ihn erst *scharfschalten*, wenn du wirklich schlafen gehst – und Makey:Lab bleibt sonst still.

Tageslicht-Wecker zum Scharfschalten

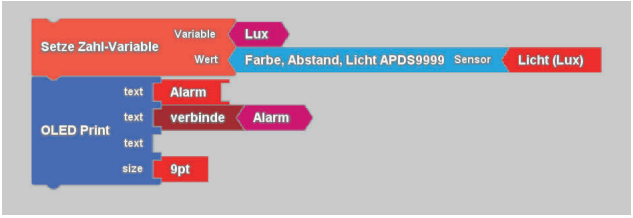
Damit das funktioniert, braucht dein Programm eine Variable, die sich merkt, ob der Alarm gerade aktiv ist oder nicht. Lege dafür ins Setup einen **Setze Zahl-Variable**-Block. Nenne die Variable *Alarm* und gibt ihr den

Startwert 0. Sie wird später zwischen 0 (aus) und 1 (an) wechseln und zeigt an, ob der Alarm scharfgeschaltet ist.



In der Programmschleife definierst du zu Beginn wieder deine Variable *Lux* und speist sie aus dem **Sensor**-Block.

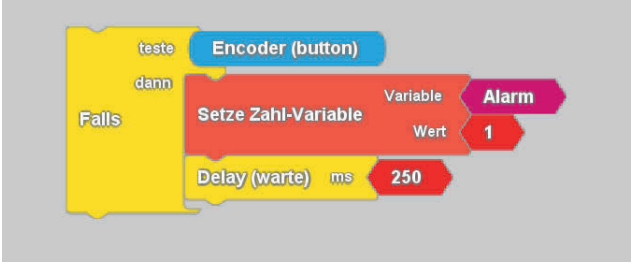
Dahinter kommt zum Testen ein **OLED Print**-Block – damit kannst du den aktuellen Status besser nachverfolgen.



Lege in die erste Zeile den Text "*Alarm*", die zweite Zeile füllst du mit einem **verbinde**-Block und deiner Variable *Alarm*. Damit steht auf dem Display in der ersten Zeile einfach nur worum es geht und in der zweiten Zeile kannst du sehen, ob der Alarm gerade 0 oder 1 ist.

Jetzt baust du dir den eigentlichen Schalter. Lege nach dem **OLED**-Block einen **Falls**-Block. Unter *teste* wählst du **Encoder (Button)** und als Aktion nutzt du den **Setze**

Zahl-Variable-Block, um deine Alarm variable auf 1 zu ändern.



Damit der Button eindeutig erkannt wird und nicht zu schnell hin und herschaltet beim Betätigen, folgt noch ein kurzes Delay von 250ms - sonst könnte es passieren, dass der Wert hin- und herspringt, solange du mit dem Finger noch auf den Button bist. Denn im Hintergrund läuft deine Programmschleife währenddessen etliche Male durch. Diesen Schutz nennt man auch "Entprellen".

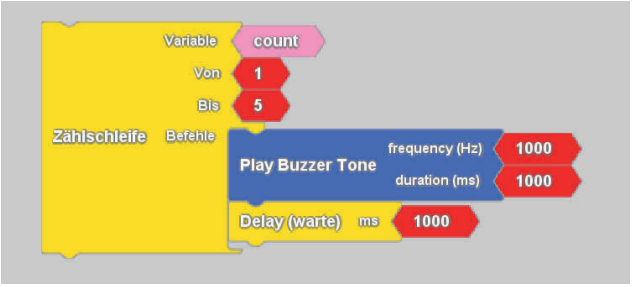
Nach der Button-Abfrage kommt jetzt der eigentliche Alarm, der durch eine weitere Bedingung erweitert wird.

Lege einen **Falls**-Block ans Ende deines Programms. Hier werden die beiden Bedingungen mit einem logischen **und**-Vergleich getestet.

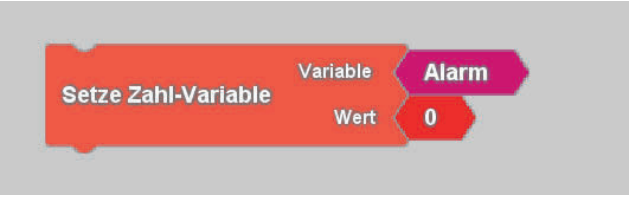
Als erste Bedingung wird geprüft, ob der *Lux*-Wert über 10 ist. Das dürfte ungefähr die Helligkeit sein, wenn es dämmert und der Tag langsam anbricht. Probiere das bei dir aber am besten nochmal aus mit deinem Helligkeitsmesser. Je nachdem, ob du noch eine Straßenlaterne vor dem Haus hast, Vorhänge, eine Außenjalousie, etc., kann das bei dir etwas variieren.

Als zweite Bedingung fragst du noch ab, ob deine Alarmvariable gerade bei 1 ist.
Also:
Alarm == 1

Wenn ja, dann folgt deine Alarmfunktion. Die kannst du genauso aufbauen wie zuvor, also **Zählschleife**, **Play Buzzer Tone**, **Delay**.



Danach wird der Alarm noch mit einer **Setze Zahl-Variable** auf 0 zurückgesetzt, damit du keinen Daueralarm bekommst.

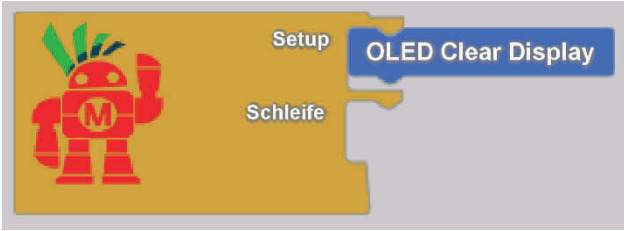


Das ist nun die erweiterte Variante deines Tageslicht-Weckers mit Aktivierungsbutton. Probiere es gleich mal aus. Du solltest den Alarm nun durch Drücken des Buttons scharf schalten können und auf dem Display eine 1 sehen. Leuchtest du den Sensor an, ertönt der Alarm und der Status springt im Anschluss automatisch wieder auf 0.

Du musst also erneut drücken, wenn dein Programm wieder laufen soll. Das ist im Prinzip wie bei einem echten Wecker: Die Uhrzeit bleibt eingestellt, aber der Alarm klingelt nur, wenn du ihn vorher aktivierst.
Praktisch, oder?

Programm abdunkeln

Damit dein Lichtsensor nicht vom **Display** durcheinandergebracht wird, kannst du jetzt noch etwas aufräumen. Den **OLED**-Block brauchten wir ja nur um zu testen, ob alles funktioniert. Den kannst du jetzt auch entfernen. Das typische Logo der *IoT-Werkstatt* bei Programmstart erlischt nach einigen Sekunden. Ansonsten kannst du auch direkt ein **OLED Clear Display**-Block ins Setup legen – dann ist es sofort dunkel. Du findest ihn unter dem Reiter *Externe Interfaces*.



Jetzt wird's nerdig - Aktivierungszeit einschränken

Es gibt noch ein kleines Problem, das wir lösen können. Stell dir vor, du legst dich um 20 Uhr schlafen und aktivierst deinen Alarm. Um 21 Uhr bekommst du Durst, schaltest dein Licht an, um etwas zu trinken. Was passiert? Richtig! Dein Wecker denkt, es sei Tag und er tönt los. Das ist nicht cool.

Mit einer Erweiterung können wir Makey:Lab noch die aktuelle Zeit aus dem Internet auslesen lassen. Damit kannst du den Alarm so eingrenzen, dass er nur losgeht, wenn es wirklich Tag ist und du wirklich aufstehen willst/musst.

ok, das ist wild!
aber du schaffst das

Setup

WLAN
Netzname: DEIN_WLAN
Passwort: DEIN_WLAN_PASSWORT

Uhrzeit, Abfrage Network Time Protokoll
Server: de.pool.ntp.org
Timezone: CET-1CEST,M3.5.0,M10.5.0/3

Setze Zahl-Variable
Variable: Alarm
Wert: 0

Schleife

Setze Zahl-Variable
Variable: Tagessekunden
Wert: Uhrzeit Now (Unix-Time) % 86400

Setze Zahl-Variable
Variable: Lux
Wert: Farbe, Abstand, Licht APDS9999 Sensor: Licht (Lux)

OLED Print
text: verbinde Alarm
text: verbinde Tagessekunden
text:
size: 9pt

Falls
teste: Encoder (button)
dann:
- Setze Zahl-Variable: Variable: Alarm, Wert: 1
- Delay (warte) ms: 250

Falls
teste: und
- Lux > 10
- und
  - Tagessekunden >= 21600
  - Tagessekunden <= 72000
- Alarm == 1

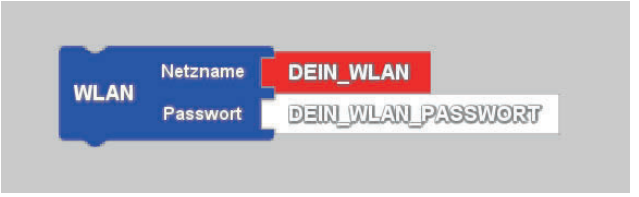
dann:
- Zählschleife: Variable: count, Von: 1, Bis: 5, Befehle:
  - Play Buzzer Tone: frequency (Hz): 1000, duration (ms): 1000
  - Delay (warte) ms: 1000
- Setze Zahl-Variable: Variable: Alarm, Wert: 0

Dafür bietet unsere **IoT-Werkstatt** genau den richtigen Block, mit dem du die Zeit aus dem Internet abfragen kannst. Das funktioniert über das sogenannte NTP-Protokoll (Network Time Protocol).

Damit fragt dein Gerät einen Zeitserver im Internet ab, der sich regelmäßig mit sehr genau laufenden Referenzuhren synchronisiert – meist mit Atomuhren als Grundlage. So kannst du die exakte Uhrzeit abrufen, auch wenn dein Makey:Lab keine eigene Echtzeituhr besitzt.

Dein nächstes Programm wird also nicht nur die Helligkeit prüfen, sondern zusätzlich anhand der Zeit aus dem Internet schauen, ob es gerade Tag oder Nacht ist. Denn nachts wollen wir keinen Alarm!

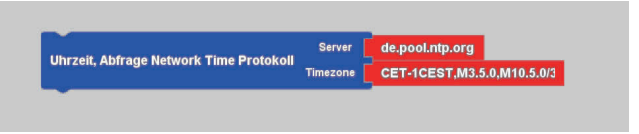
Um dein Programm dafür vorzubereiten, sind ein paar Schritte notwendig. Zunächst muss sich Makey:Lab mit deinem WLAN verbinden können. Dafür nutzt du den **WLAN**-Block unter *IoT: WiFi* und legst ihn ins Setup.



Hier gibst du unter *Netzname* den Namen deines WLANs ein - das ist quasi die SSID. Unter *Passwort* schreibst du dein WLAN-Passwort. Wenn du das nicht weißt, frag am besten deine Eltern danach.

Mehr als diesen Block brauchst du nicht, um Makey:Lab ans Internet anzubinden.

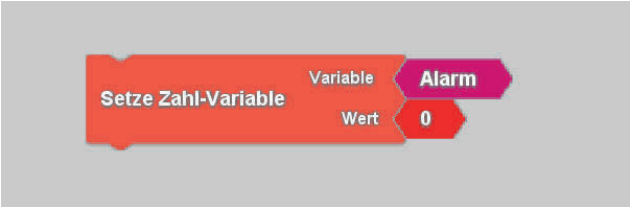
Als nächstes wollen wir noch eine Verbindung zum Zeitserver aufbauen. Dafür nimmst du den Block: **Uhrzeit, Abfrage Network Time Protokoll** (den findest du unter *IoT: E-Mail / NTP*). Die Standardwerte kannst du so eingestellt lassen - die stimmen, wenn du dich in Deutschland befindest.



Server: *de.pool.ntp.org*
Das ist ein sogenannter *Zeitserver-Pool*. Dahinter steckt eine Gruppe von zuverlässigen Servern in Deutschland, die immer die aktuelle Zeit bereitstellen. Der Pool verteilt automatisch die Anfragen auf verschiedene Server, damit alles stabil und genau bleibt.

Zeitzone: *CET-1CEST,M3.5.0,M10.5.0/3*
Das klingt kompliziert, bedeutet aber einfach: Mitteleuropäische Zeit (CET) und automatische Umstellung auf Sommerzeit (CEST). Der Teil M3.5.0,M10.5.0/3 beschreibt, **wann** die Zeit umgestellt wird – nämlich am letzten Sonntag im März bzw. Oktober um 3 Uhr.

Nach dem **NTP**-Block definierst du wieder deine *Alarm*-Variable auf 0. So startet das Programm.



Nun kommen wir in die Programmschleife. Es gibt in der **IoT-Werkstatt** zwei verschiedene Variablen-Blöcke, die die Uhrzeit angeben.

Uhrzeit als Text: Diese Angabe sieht für uns schön nachvollziehbar aus. Beispielsweise 06:05:20 - also 6:05 Uhr und 20 Sekunden. Zum Rechnen für unser Makey:Lab ist das aber unpraktisch, da wir leichter mit einer ganzen Zahl arbeiten können. Wir nutzen also einen anderen Block, dieser gibt uns die sogenannte Unix-Zeit aus. Er heißt: **Uhrzeit-Now (Unix-Time)**.

Unix-Zeit und Tagessekunden

Computer messen Zeit ein bisschen anders als wir. Sie zählen keine Stunden und Minuten, sondern die Anzahl der Sekunden, die seit dem 1. Januar 1970 vergangen sind. Dieses System nennt man Unix-Zeit. Der Vorteil: Sie läuft überall auf der Welt gleich und funktioniert auch über viele Jahrzehnte hinweg – egal, welcher Tag oder welches Jahr gerade ist.

Fun Fact:
Frühere 32-Bit-Computer konnten mit der Unix-Zeit nur bis zum Jahr 2038 zählen – dann „läuft ihre Uhr über“.

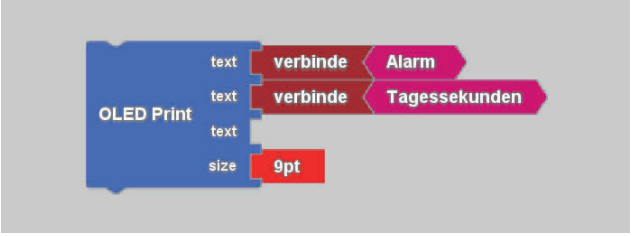
Moderne Systeme wie dein ESP32 nutzen aber längst 64-Bit-Zahlen, damit reicht die Zeitrechnung für viele Milliarden Jahre.

Ein bisschen Mathe: Um herauszufinden, wie viel Zeit heute schon vergangen ist, müssen wir diesen großen Zahlenwert einfach in einen Tagesrhythmus bringen. Ein Tag hat 86 400 Sekunden (24 × 60 × 60). Wenn wir also den Unix-Zeitwert durch 86 400 teilen und den Rest nehmen (% 86400), erhalten wir genau die Sekunden seit Mitternacht – also, wie weit der aktuelle Tag (in Sekunden) schon fortgeschritten ist.

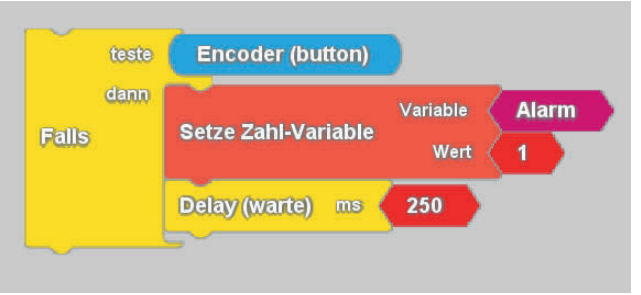
Diese kleine Rechnung kannst du direkt beim Setzen der neuen Zeit-Variable *Tagessekunden* einfügen und an den Anfang der Schleife setzen. So weiß dein Programm jederzeit, wie viele Sekunden des Tages bereits vergangen sind – und damit lässt sich doch einiges anfangen!

Danach kommen wir wieder zu unserem Programmaufbau von eben. Du setzt zunächst wieder die Variable *Lux* und fütterst sie mit dem Ausgang des APDS9999-Sensors, um dir später den Lichtwert anzeigen lassen zu können.

Um dein Programm während des Aufbaus besser verstehen und überprüfen zu können, solltest du direkt dahinter wieder ein **OLED Print**-Block einsetzen. Diesen verbindest du in der ersten Zeile mit deiner Status-Variable *Alarm* und in der zweiten Zeile mit deinen *Tagessekunden*. So kannst du direkt auf dem Display sehen, was dein Programm gerade macht. Dieser Vorgang wird auch *Debugging* genannt - dabei sucht man gezielt nach Bugs, also kleinen Programmfehlern, und kann sie Schritt für Schritt aufspüren. Kommen hier falsche Werte an, dann stimmt etwas noch nicht.



Als nächstes kannst du wieder deinen Button zum Scharfschalten einrichten. Also ein **Falls**-Block, der auf den Encoder-Button reagiert. Ein Druck auf den Button setzt die Variable *Alarm auf 1*. Mit dem kleinen 250ms-Delay entprellst du die Eingabe wieder – wie gehabt.



Abschließend kommt jetzt die Überprüfung, ob der Alarm losgehen soll. Nimm also einen weiteren **Falls**-Block. Getestet werden dieses Mal gleich mehrere Bedingungen, die kannst du aus mehreren **und**-Vergleichen zusammenbauen. Schau dir das am besten mal auf dem Bild genau an – du wirst sehen, das ist ganz einfach.

Du prüfst:
- den Status
- die Helligkeit
- die Zeit

Die erste Bedingung, die du abfragst, ist der Status. Ist der Alarm scharfgeschaltet?
Also: *Alarm == 1*

Dann willst du natürlich auch wissen, ob es bereits dämmert. Du kannst hier wieder mit dem Beispiel 10 anfangen oder deinen selbst gemessenen Wert eintragen.
Also: *Lux > 10*

Nun grenzt du noch die Zeit ein. Für ein Zeitfenster

brauchst du genau zwei Zeitangaben. Da dein Alarm nur am Tag stattfinden soll und nicht mitten in der Nacht (denk an das Beispiel, wenn du nachts das Licht zum Trinken anschaltest), stellst du quasi die Schlafenszeit für den Alarm ein.

Hierfür ist es hilfreich, wenn du dir vorher überlegst, **welche Uhrzeit wie viele Tagessekunden** hat.
Ein Tag hat 24 Stunden = 86 400 Sekunden.
Eine Stunde hat **60 Minuten × 60 Sekunden = 3600 Sekunden** – daher multiplizierst du einfach die Stunden mit 3600, um den passenden Wert zu erhalten.

Uhrzeit	Rechnung	Tagessekunden
0 Uhr	0 × 3600	**0**
3 Uhr	3 × 3600	**10 800**
6 Uhr	6 × 3600	**21 600**
12 Uhr	12 × 3600	**43 200**
20 Uhr	20 × 3600	**72 000**
24 Uhr (= 0 Uhr)	24 × 3600	**86 400** (= wieder 0 Uhr des nächsten Tages)

So kannst du ganz leicht herausfinden, wie viele Sekunden eines Tages bereits vergangen sind – und dein Makey:Lab weiß dann ganz genau, ob gerade früher Morgen oder später Abend ist.

Du könntest das Zeitfenster beispielsweise auf 6 Uhr morgens bis 20 Uhr abends einstellen. Nur dazwischen soll der Alarm angehen können. Die Werte dafür sind:

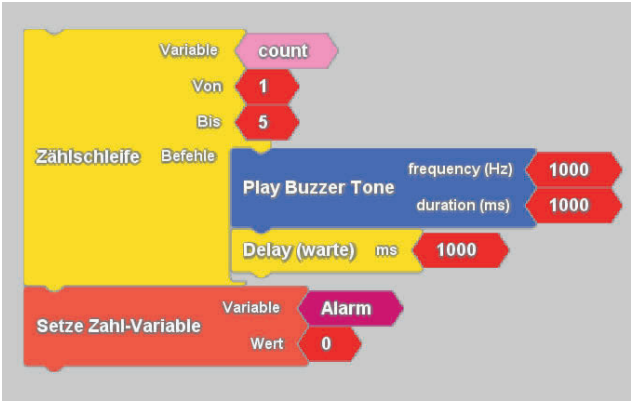
6 Uhr = 21600
20 Uhr = 72000

Prüfe also:
Tagessekunden >= 21600
Tagessekunden <= 72000

Deine **und**-Verschachtelung sieht im Blockformat vielleicht etwas kompliziert aus. Aber grundsätzlich werden hiermit einfach alle Bedingungen abgefragt.

Wenn sie alle erfüllt sind, also der Alarm-Status ist 1, Helligkeit ist über 10, es ist nach 6 Uhr und vor 20 Uhr, dann soll dein Alarm starten.

Den Alarm baust du einfach so auf wie zuvor, mit fünf Wiederholungen. Am Ende setzt du deine *Alarm*-Variable wieder auf den Status 0 zurück.



Fertig!

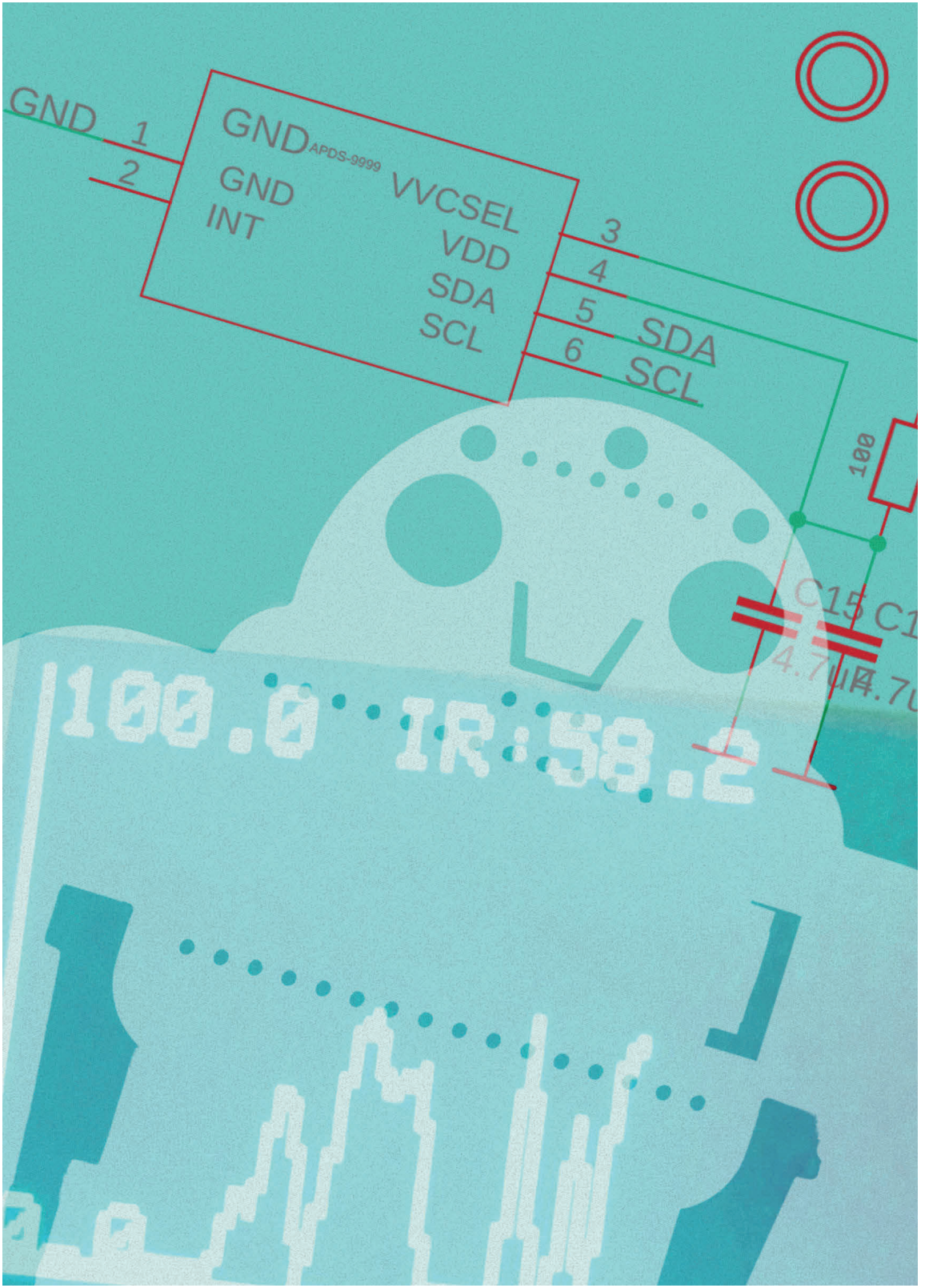
Jetzt hast du einen Tageslicht-Wecker, der seine Uhrzeit direkt aus dem Internet holt. Das war schon ein etwas anspruchsvollerer Aufbau – atme also ruhig einmal durch. Danach kannst du prüfen, ob alles wie geplant funktioniert.

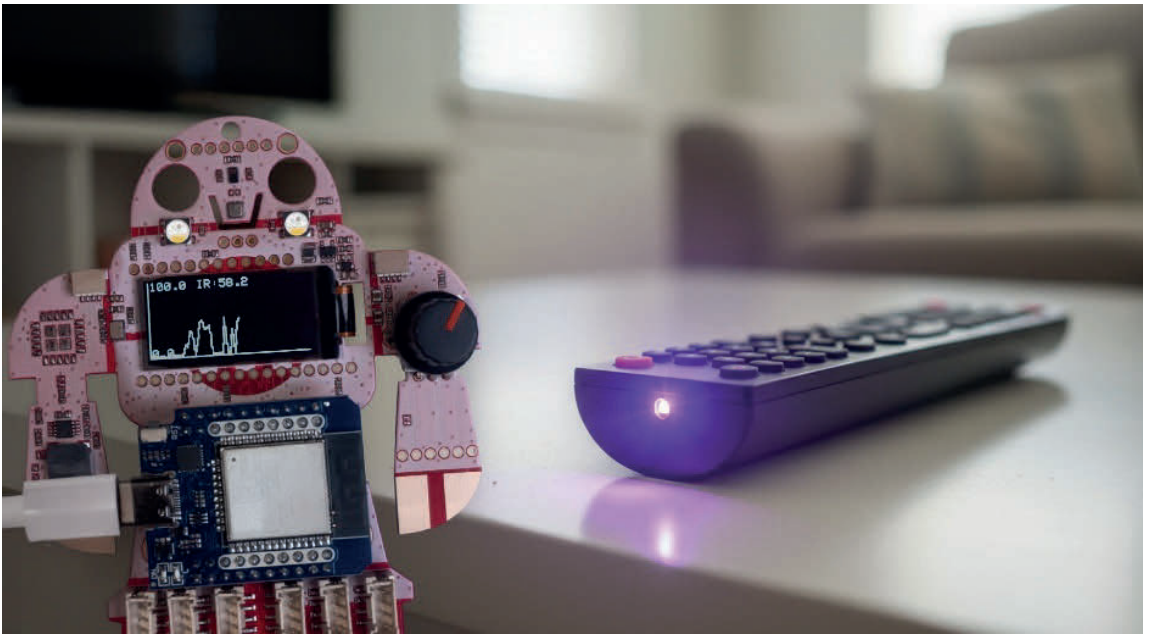
Wenn du gerade außerhalb deines Zeitfensters testest, kannst du die Werte einfach kurz anpassen. So lässt sich das Verhalten deines Weckers gut ausprobieren – spiel ruhig ein wenig mit den Zahlen!

Du willst noch mehr wissen?

SCAN MICH

Auch Insekten richten ihren Tag nach dem Sonnenlicht aus. Wann sind welche Arten eigentlich unterwegs? Am Umwelt-Campus Birkenfeld entwickelt die Arbeitsgruppe **Interdisziplinärer Umweltschutz** spezielle „Malaise-Fallen“, mit denen Insekten lebend gefangen, fotografiert und per KI automatisch bestimmt werden.
Scanne den QR-Code, um mehr darüber zu erfahren!


GND
1
2
GND
APDS-9999
GND
INT
VVCSEL
VDD
SDA
SCL
3
4
5
6
SDA
SCL
100
C15
4.7U
100.0 IR:58.2



#08 Makey:Lab sieht Farbe

Von farbigem und unsichtbarem Licht

Wenn wir von „Licht“ sprechen, denken wir meistens an das, was wir mit unseren Augen sehen können: hell, dunkel, Farben.
Doch in Wahrheit ist das nur ein winziger Ausschnitt dessen, was Licht eigentlich ist. Licht ist eine Form elektromagnetischer Strahlung – und die reicht weit über das sichtbare Spektrum hinaus.

Unser Auge ist auf einen schmalen Bereich zwischen etwa **380 und 780 Nanometern Wellenlänge** spezialisiert. In diesem Bereich sehen wir von Violett bis Rot alle Farben des Regenbogens.
Aber schon kurz *unterhalb* dieses Bereichs beginnt ultraviolettes Licht (UV), das du nicht sehen kannst, das

aber deine Haut spürt, wenn sie Sonnenbrand bekommt.

Und *oberhalb* davon liegt das **Infrarotlicht (IR)** – ebenfalls unsichtbar, aber voller Energie. Es steckt zum Beispiel in Fernbedienungen, Wärmelampen oder Nachtsichtgeräten.

Farben sehen wie ein Sensor

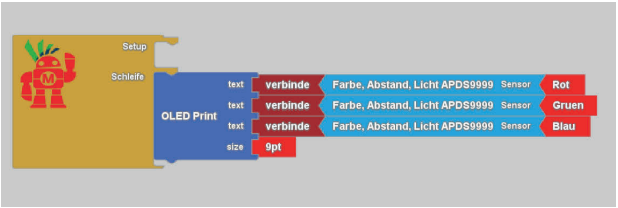
Dein Makey:Lab kann Licht auf eine ganz andere Art „sehen" als du. Der **APDS-9999-Sensor** auf deiner Platine misst, wie stark das Licht in den drei Grundfarben **Rot (R)**, **Grün (G)** und **Blau (B)** ist.
Diese drei Farbanteile nennt man kurz **RGB**, und sie bilden die Grundlage der **additiven Farbmischung**:
Mischt man alle drei in voller Stärke, entsteht Weiß.
Mischt man nur Rot und Grün, bekommt man Gelb – und so weiter.

Dein Auge macht im Prinzip genau das Gleiche, nur mit biologischen „Sensoren": Es besitzt drei verschiedene Zapfentypen für Rot-, Grün- und Blauanteile. Dein Gehirn rechnet dann aus dem Verhältnis dieser Signale die Farbe zusammen, die du siehst.

Bauen wir zunächst einen einfachen Farbmesser.

Farben als Zahlen

Das Programm zum Erkennen von Farben ist ganz einfach. Wir wollen einfach die RGB-Werte auf dem Display anzeigen lassen.



Nimm einen **OLED Print**-Block und lege ihn in deine Programmschleife. In jede Zeile hängst du einen verbinde-Block an und danach den **Farbe, Abstand, Licht APDS9999 Sensor**-Block. Als Ausgang des Sensors wählst du in der ersten Zeile *Rot*, in der zweiten *Gruen* und in der dritten *Blau*. Dann kannst du dein Programm bereits starten.

Dein Versuch zeigt dir die drei Farbwerte direkt auf dem Display.

Halte verschiedene Gegenstände oder farbige Flächen vor den Sensor – etwa ein roter Klemmbaustein, ein grünes Blatt oder ein blaues Stück Papier.

Die Zahlen verändern sich sofort: Bei Rot steigt der R-Wert, bei Grün der G-Wert und bei Blau der B-Wert.

So erkennst du, wie unterschiedlich Materialien Licht reflektieren – und wie dein Makey:Lab daraus die Farbe „sieht“.

Wirklich weiß?

Was du dabei auch beobachten kannst: Licht ist selten wirklich weiß.

Unser Auge gleicht Farbstiche automatisch aus – es führt sozusagen ständig einen Weißabgleich durch.

Das heißt: Dein Gehirn rechnet das Umgebungslicht so um, dass Weißflächen auch wirklich weiß aussehen – selbst wenn das Licht in Wirklichkeit leicht gelblich oder bläulich ist.

Kameras machen genau das Gleiche: Sie müssen den Weißabgleich oft automatisch oder manuell anpassen, damit die Farben auf einem Foto natürlich wirken.

Der Sensor hingegen macht keinen solchen Ausgleich. Er sieht das Licht, wie es wirklich ist.

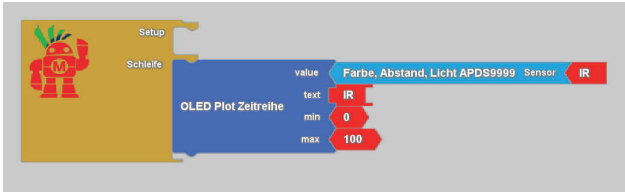
Darum unterscheiden sich die RGB-Werte je nach Beleuchtung deutlich:

Bei Tageslicht wirken sie recht ausgewogen, unter LED-Licht zeigt sich meist ein grünlicher Stich, und bei einer Glühlampe verschiebt sich alles sichtbar ins Rötlich-Gelbe.

Das unsichtbare Licht

Doch der APDS-9999 kann noch mehr. Er reagiert nicht nur auf sichtbares Licht, sondern auch auf **Infrarotstrahlung** – also Licht, das jenseits des roten Spektrums liegt und für unsere Augen unsichtbar ist.

Mit einem kleinen Programm kannst du den IR-Wert anzeigen lassen. Wenn du nun eine **TV-Fernbedienung** in Richtung Sensor drückst, wirst du im Wertverlauf einen kurzen Ausschlag sehen.



Erstelle in neues Programm. Lege in die Schleife einen **OLED Plot Zeitreihe**-Block. Als *value* legst du den **APDS9999**-Sensor an und wählst als Ausgang *IR* – also Infrarot.

Gib als *text* ebenfalls noch *IR* an. Als Wertebereich kannst du mal 0 bis 100 ausprobieren, das sollte für den Anfang ausreichen.

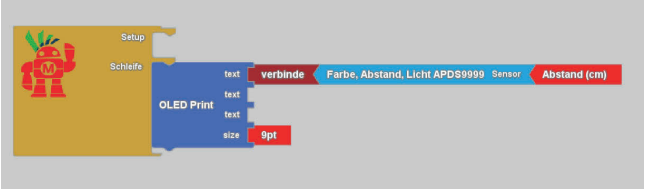
Was du damit beobachten kannst, ist echtes unsichtbares Licht – kleine modulierte Lichtblitze, mit denen deine Fernbedienung ihren Code an den Fernseher sendet. Halte die Fernbedienung an den Sensor und drücke ein

paar Mal eine Taste oder halte sie gedrückt. Das solltest du nun in deinem Verlauf sehen können.

Wie Makey:Lab mit Licht misst

Im Inneren des APDS-9999 sitzt eine winzige Infrarot-LED. Damit kann dein Makey:Lab also nicht nur Licht sehen, sondern auch selbst kleine Lichtblitze aussenden.

Das kannst du mit einem kleinen Trick sogar sichtbar machen.



Das Versuchsprogramm dazu ist ganz einfach: Füge einen **OLED Print**-Block in deine Schleife ein, **verbinde** ihn mit dem **APDS**-Sensor und wähle diesmal die Funktion *Abstand (cm)*.

Das kennst du vielleicht schon von der **Zimmer-Alarmanlage**. Wenn du das Programm startest, siehst du auf dem Display die gemessene Entfernung in Zentimetern.

Das Prinzip dahinter ist clever: Der Sensor sendet kurze IR-Lichtimpulse aus, die auf eine Fläche treffen und von dort reflektiert werden.

Aus der Stärke dieser Reflexion erkennt der Sensor, wie weit das Objekt entfernt ist.

Wenn du das einmal in Aktion sehen willst, schnapp dir dein Smartphone oder Tablet und öffne die Kamera. Viele Kameras können das Infrarotlicht als violette Blitze erkennen – besonders ältere oder einfachere Modelle.

Hinweis: Bei manchen neueren Smartphones (zum Beispiel iPhones) ist allerdings ein IR-Filter eingebaut, der das unsichtbare Licht fast vollständig blockiert. Falls du also nichts siehst, probier es einfach mit einem anderen Gerät.

Richte sie auf den Sensor – auf dem Bildschirm erkennst du nun kleine violette Blitze. Das sind die unsichtbaren Lichtimpulse, die dein Makey:Lab aussendet!

Das funktioniert, weil der Kamerasensor deines Smartphones Infrarotlicht noch erkennt, es aber nicht herausfiltert.

So wird das eigentlich unsichtbare Licht sichtbar – und du kannst beim Messen quasi mitsehen.

Experimentiere weiter

Jetzt kennst du drei Arten, wie dein Makey:Lab Licht wahrnehmen kann:
sichtbares Licht als Farbe, unsichtbares IR-Licht aus Fernbedienungen – und reflektiertes Infrarotlicht, das zur Abstandsmessung dient.

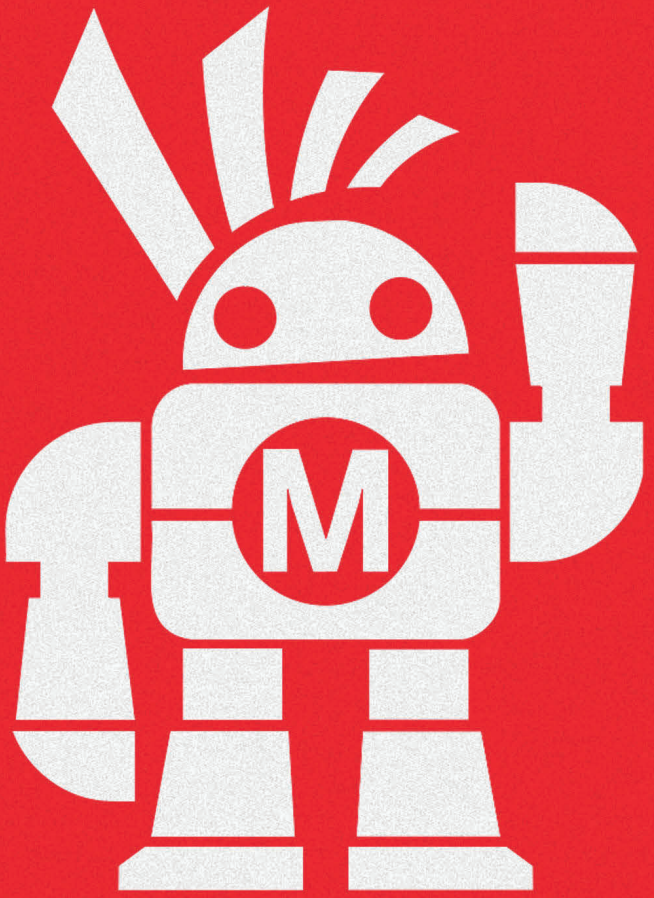
Du wirst schnell merken: Licht ist nicht einfach nur hell oder dunkel. Es verrät viel über seine Farbe und Intensität – und sogar, wie nah ein Gegenstand ist.
Dein Makey:Lab kann all das sichtbar machen – fast so, als hätte es ein eigenes kleines Auge.

Du willst noch mehr wissen?

SCAN MICH

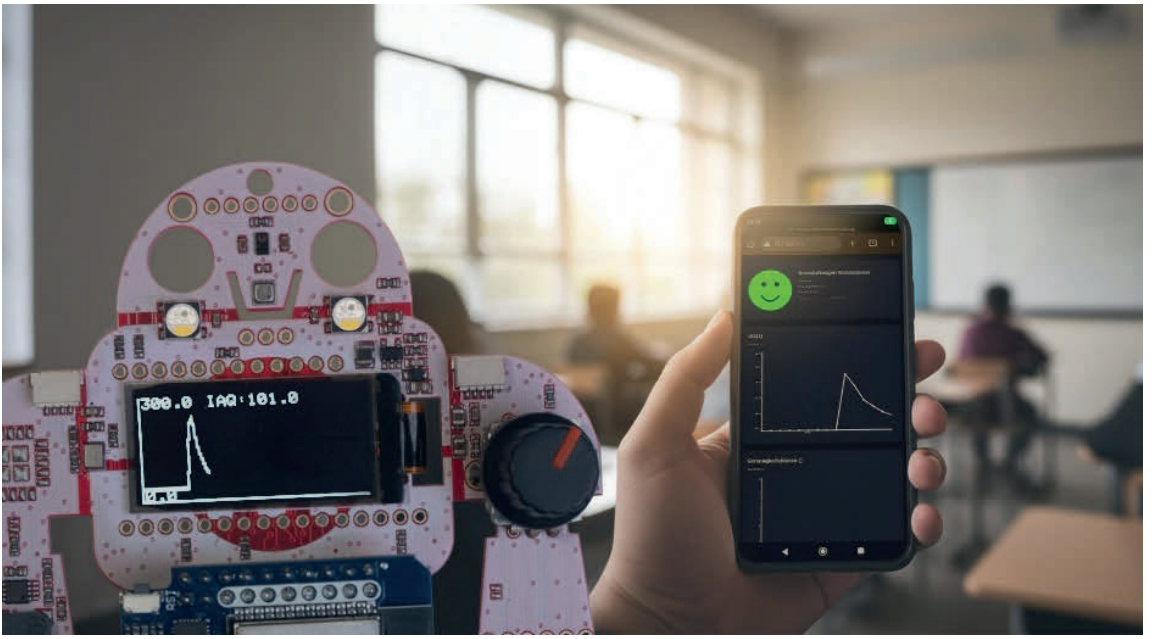
Hinter dem QR-Code findest du spannende Anwendungen rund um Farbmessung und Lichtanalyse – von Spektrometern in der Chemie bis zur Farbkalibrierung in den Medienlaboren.

Mehr dazu im Studiengang **Bio- und Pharmatechnik (B.Sc.)** am Umwelt-Campus Birkenfeld.



UND WEITER GEHTS!

IoT Werkstatt
TP3
TP4
TP5
TXD
RXD
IO22
SCL
IO21
SDA
IO17
TX
IO16
RX
GND2
VCC
5V
TD0
15
SD0
GND3
IO27
IO25
A1-G4
IO32
32
TDI
IO4
A5
IO0
IO2
SD1



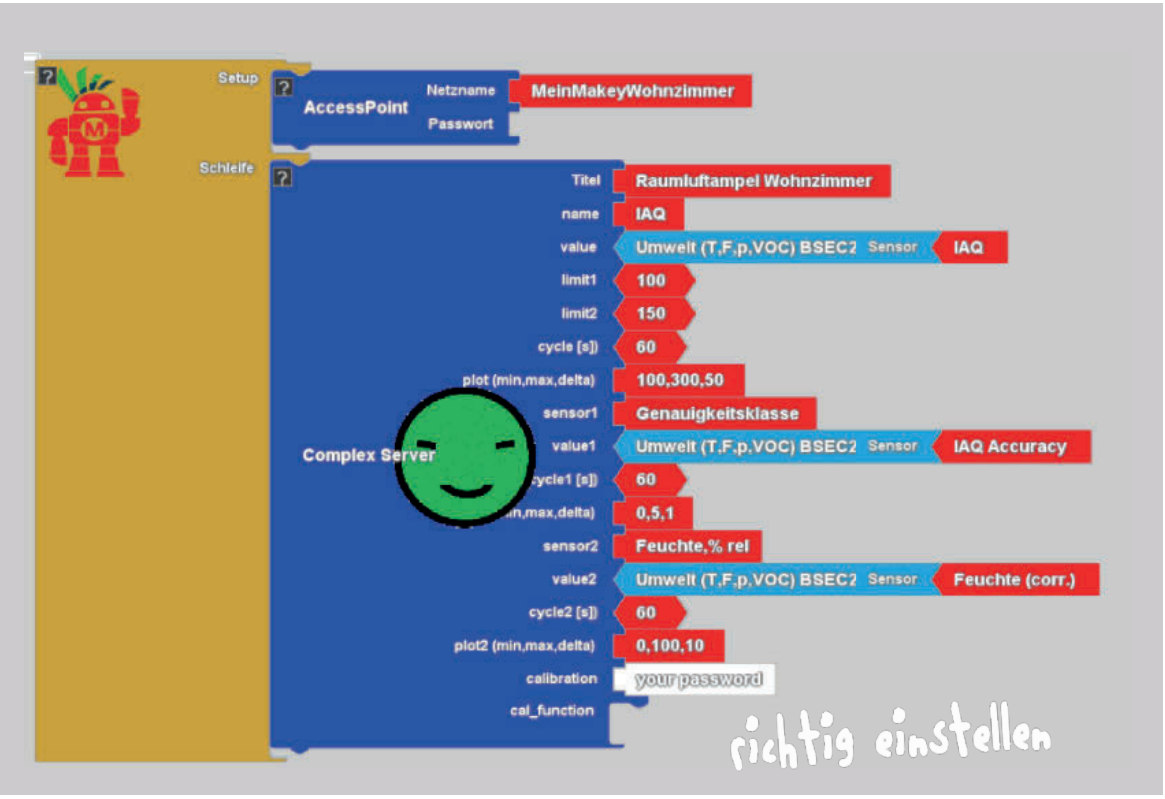
#09 Makey:Lab nutzt WLAN

So bringst du deine Messdaten ins WLAN

Im Kapitel *#02 Frische Luft fürs Gehirn* hast du deine eigene Frischluft-Ampel gebaut. Damit misst du die Luftqualität im Raum – zum Beispiel im Klassenzimmer oder zu Hause beim Lernen.

Die Anzeige funktioniert bisher über LEDs und ein kleines Kurvendiagramm auf dem OLED-Display. Aber wie wäre es, wenn du die Daten **direkt auf deinem Smartphone oder Tablet im Browser sehen könntest?**

Genau das geht! Dein Makey:Lab wird zum WLAN-Sender und stellt eine kleine Webseite bereit. Dort kannst du – und alle anderen im Raum – **live verfolgen**, wie gut die Luft gerade ist.



Schritt 1: WLAN starten

Unter dem Reiter *Iot: WiFi* findest du den Block **AccessPoint**.

Mit diesem Block startet dein Makey:Lab ein eigenes WLAN. Du brauchst also keinen Router oder Internetzugang! Leg ihn einfach unter *Setup* ab.

Hier trägst du ein:

- **Netzname** (SSID): z. B. *Klassenzimmer*
- **Passwort**: kann leer bleiben oder ein eigenes sein

Info: Was ist eine SSID?

SSID steht für *Service Set Identifier* – das ist einfach der Name des WLANs.

Du siehst ihn auf deinem Handy, wenn du nach verfügbaren Netzwerken suchst.

Schritt 2: Der Complex Server-Block

Jetzt brauchst du den **Complex Server**-Block. Du findest ihn unter dem Reiter *IoT: HTTP*.

Ja, der Block ist groß – aber lass dich nicht abschrecken!

Er funktioniert ähnlich wie der Block **OLED Plot Zeitreihe**, nur dass du damit mehrere Sensorwerte gleichzeitig im Browser als Diagramme anzeigen kannst.

Beispiel-Einstellungen

So lässt du dir den IAQ-Wert anzeigen:

- **Name**: IAQ
- **Sensorwert**: Umwelt (T,F,p,VOC) → IAQ
- **Limit 1**: 100 → (Grenze, ab der's kritisch wird)
- **Limit 2**: 150 → (Grenze, ab der's Alarm gibt)
- **Messintervall (Sekunden)**: 60
- **plot (min, max, delta)**: 100,300,50

Diese Angaben kannst du auch für weitere Werte machen, z. B.:

- **IAQ Accuracy** (Genauigkeit)
- **Feuchtigkeit % rel.**
- **Temperatur**

Der Smiley erklärt

Der Webserver zeigt neben den Diagrammen auch einen Smiley, der sich automatisch verfärbt – in unserem Beispiel abhängig von der Luftqualität. Nur der erste Sensorwert im Block bestimmt die Farbe des Smileys

IAQ-Wert	Farbzone	Bedeutung
unter 100	Grün	Alles ok
100–150	Orange	Wird kritisch
über 150	Rot	Lüften, bitte!

Du steuerst die Farben mit den Werten *Limit 1* (kritisch) und *Limit 2* (Alarm).

Schritt 3: Frischluft-Ampel weiterverwenden

Jetzt musst du nur noch deine bereits erstellten Frischluft-Ampel-Blöcke einfügen. Also wie in dem Kapitel *Frische Luft fürs Gehirn*. Oder du erweiterst dein

Programm mit dem Complex-Server-Block, wenn du es zuvor abgespeichert hast.

Dann funktioniert alles gleichzeitig:

- LED zeigt Ampelfarbe
- OLED zeigt Kurvenverlauf
- Webseite zeigt Diagramme

Die Darstellung des **Server**-Blocks läuft einfach parallel zur bisherigen Funktionsweise des Programms. Praktisch, oder?

Zugriff im Browser

So rufst du deine Daten im Browser auf:

- Verbinde dein Handy mit dem WLAN deines Makey:Lab: z. B. *Klassenzimmer*
- Öffne einen Browser (z. B. Firefox, Chrome)
- Gib diese Adresse ein: *http://192.168.4.1*

Was ist das für eine Zahl? *192.168.4.1* ist eine sogenannte IP-Adresse – die braucht der Browser, um das Makey:Lab zu erreichen.

Du musst sie genauso eingeben, inklusive *http://* am Anfang! Dann weiß der Browser, was du von ihm willst und du vermeidest Fehler.

Hinweis: Falls die Seite nicht lädt, dann kann das an einem Werbeblocker (Adblocker) liegen. Eventuell könntest du einen anderen Browser probieren.

Wichtig: Manchmal musst du an deinem Smartphone die mobilen Daten ausschalten. Wenn es nämlich merkt, dass das WLAN keine Internetverbindung hat, wechselt es sonst automatisch auf mobile Daten – und dein Browser verbindet sich dann nicht mehr mit deinem Makey:Lab!

Extra-Tipp für Experimente im Unterricht

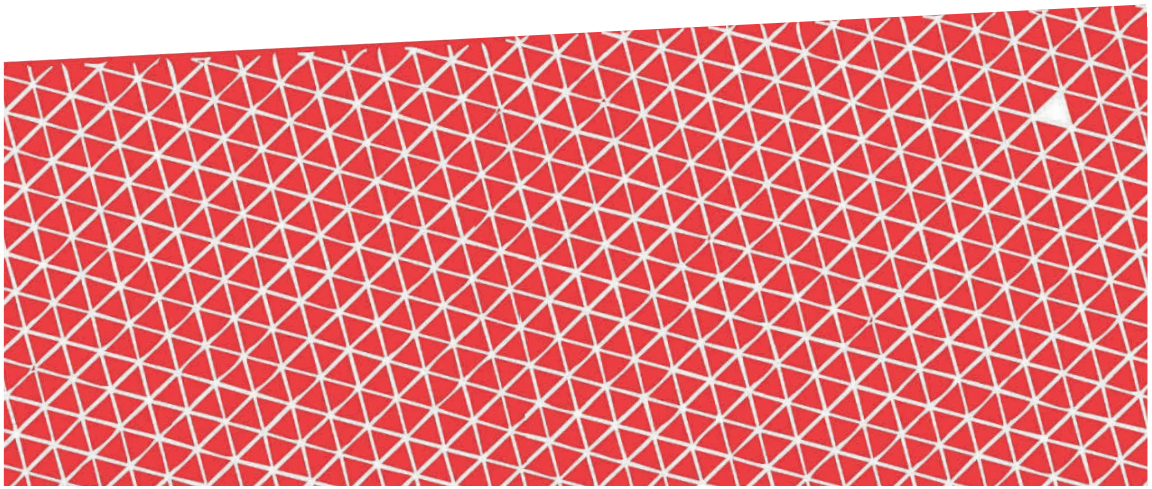
Das Webinterface eignet sich perfekt für eine kleine *Live-Demo im Klassenzimmer*:

- Alle, die möchten, verbinden sich mit dem Makey-WLAN.
- Ihr seht gemeinsam, wie viele Personen auf die Luftqualität auswirken.
- Die Diagramme aktualisieren sich automatisch!

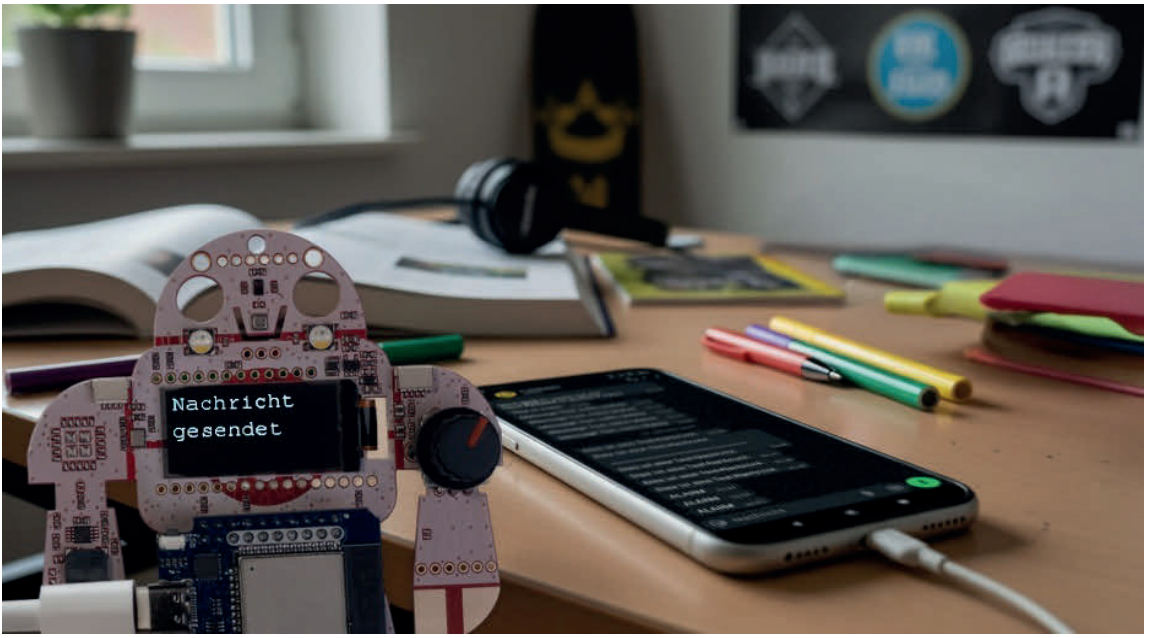
QR-Code-Idee: Erstelle einen QR-Code mit dem Link *http://192.168.4.1* und hänge ihn als Ausdruck an die Wand – dann können alle sofort loslegen.

Hinweis: Nur eine Handvoll Geräte kann sich gleichzeitig mit dem Makey-WLAN verbinden, da der eingebaute WLAN-Chip nur eine begrenzte Zahl an Verbindungen zulässt.

Dieser QR-Code führt direkt zum Ziel.



R9
12K
ADDR
VCC
GND
SDA
SCL
OLED_RST
R3
68R
3.3V
C6
1.0µF
VCOMH
+12V
OLED1
1*3
VSS
SDA/MOSI
SCK/SCL
DC
RES#
CS
IREF
IM1
VDD
VCOMH
VPP
128x32 OLED Display
OLED_128X64_SH1107
MT1
MT2
MT
CONN1G$1
CONN2G$1
V+
STEMMA_I2C_QT
Nachricht
gesendet
VBUS
R12
100K
A4
R13
220K



#10 Makey:Lab schreibt dir – übers Internet!

Wenn dein Projekt plötzlich mit dir chattet

Im letzten Kapitel hast du erlebt, wie dein Makey:Lab seine Messdaten per WLAN direkt anzeigt – als kleine Webseite, ganz ohne Internet.

Aber was wäre, wenn dein Makey:Lab dir **selbst eine Nachricht schicken** könnte?

Nicht nur an den Browser im gleichen WLAN, sondern an dein Smartphone, egal wo du gerade bist – bei deinen Freunden, im Verein oder unterwegs in der Stadt?

Genau das kann dein Makey:Lab!
Mit einem kleinen Trick und einem Dienst im Internet lernt dein Board, Nachrichten über WhatsApp zu verschicken. Und auch E-Mails kann es selbst absenden–

ganz ohne Umweg. So wirst du sofort benachrichtigt, wenn sich zu Hause etwas tut – ganz automatisch.

Das Prinzip ist einfach:
Makey:Lab spricht nicht direkt mit WhatsApp, sondern mit einem digitalen „Postboten“. Dieser heißt **CallMeBot** und hat eine Softwareschnittstelle, über die Computer und Programme miteinander reden können. Man nennt das auch Application Programming Interface, kurz API. Makey:Lab schickt also eine Nachricht an diesen Postboten, und der trägt sie für dich weiter – direkt auf dein Handy.

So wird aus deinem Makey:Lab ein echter Internet-Teilnehmer.

Ab jetzt kann es nicht nur messen und anzeigen, sondern auch Nachrichten über das Internet verschicken.

Damit werden deine bisherigen Programme richtig spannend – denn alles, was Makey:Lab erkennt oder misst, kann es nun auch nach draußen melden.

Das nennt man das **Internet der Dinge** – kurz **IoT**. Alltägliche Geräte, die ihre Umgebung wahrnehmen, Entscheidungen treffen und Nachrichten austauschen, alles vernetzt.

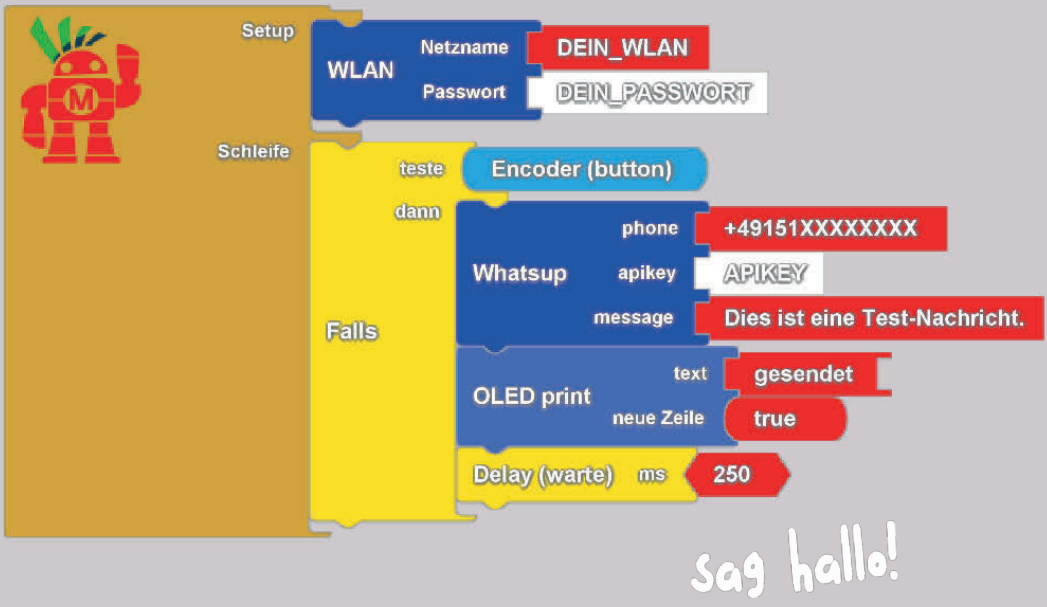
Und genau das lernst du jetzt: Wie dein Makey:Lab übers Internet mit dir spricht – und dich auf dem Laufenden hält.

Zuerst schauen wir uns an, **wie das über WhatsApp funktioniert**, und danach die klassische Variante **per E-Mail** – sie kommt ganz ohne zusätzliche Dienste aus.

Makey:Lab lernt WhatsApp
Jetzt wird's praktisch: Du richtest CallMeBot ein und lässt Makey:Lab seine erste WhatsApp-Nachricht verschicken.

Im nächsten Schritt erfährst du, wie du deine Telefonnummer registrierst und den API-Key bekommst – das kleine Passwort, das Makey:Lab für den Versand braucht.

In deinem Programm geht das ganz einfach:
Dafür gibt es einen speziellen Block, der mit CallMeBot spricht.



Du trägst dort später nur deine **Telefonnummer**, den **API-Key** (das ist eine Art Passwort für den Dienst) und den **Text der Nachricht** ein.

Was im Hintergrund alles passiert, übernimmt der Block für dich – du musst dich um keine komplizierten Datenformate kümmern.

Der Dienst ist kostenlos und wird seit vielen Jahren von Technikbegeisterten weltweit genutzt. Du gibst dabei keine persönlichen Daten weiter – nur deine Telefonnummer, damit CallMeBot weiß, wohin die Nachricht geschickt werden soll.

Danach probierst du das erste Beispielprogramm aus – und bekommst bald deine erste WhatsApp-Nachricht von Makey:Lab!

CallMeBot einrichten

Um CallMeBot nutzen zu können, musst du dort einmalig deine eigene Handynummer registrieren. Das geht ganz schnell – wichtig ist nur: Du kannst später **nur an diese Nummer** Nachrichten senden, nicht an beliebige andere.

Speichere dazu die CallMeBot-Nummer **+34 694 242 562** in deinem Handy.
Wähle einen Namen, den du leicht wiedererkennst – zum Beispiel „CallMeBot“ oder „Makey:Lab“.

Dann musst du dieser Nummer über WhatsApp **einmalig eine Nachricht** schicken.

Öffne also WhatsApp, tippe auf das Pluszeichen für eine neue Nachricht, wähle den eben angelegten Kontakt und schreibe:

I allow callmebot to send me messages

Nach kurzer Zeit solltest du eine Antwort bekommen:

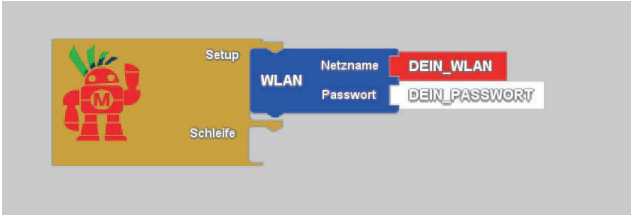
API Activated for your phone number. Your APIKEY is […]

Notiere dir den **API-Key**, den du soeben erhalten hast – den brauchst du gleich im Programm.

Hinweis: Eine genaue (englischsprachige) Anleitung findest du auch auf der offiziellen Webseite: *https://www.callmebot.com/blog/free-api-whatsapp-messages/*

Erster WhatsApp-Test

Jetzt ist es Zeit, dass Makey:Lab dir eine erste Nachricht schickt! Erstelle ein neues Programm und richte mit dem **WLAN**-Block im Setup als erstes dein Netzwerk ein. Gib hierfür also wieder Netzname und Passwort ein.



Danach nimmst du dir einen **Falls**-Block und legst ihn in die Programmschleife. Unter *teste* wählst du den **Encoder**

(button)-Block. Unter *dann* passiert jetzt die ganze Magie. Zieh dir aus dem Reiter *IoT: E-Mail / NTP* den **Whatsup**-Block (ja, der heißt wirklich so – angelehnt an WhatsApp) und leg ihn als Aktion ab.

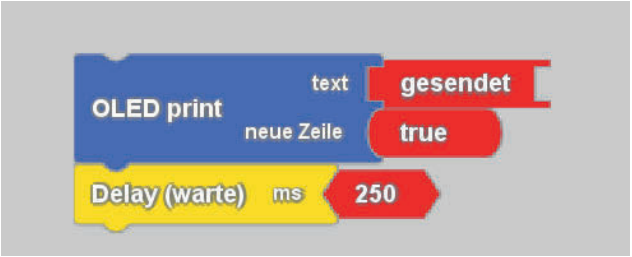
Der Block hat drei Parameter. Einmal natürlich unter *phone* deine Telefonnummer. Diese musst du in einem bestimmten Format schreiben. Statt der 0 am Anfang schreibst du die Länderkennung +49 (für Deutschland), danach folgt deine eigentliche Nummer, ohne Leerzeichen.

Also beispielsweise:

+49151XXXXXXXX *(Setze hier deine eigene Nummer statt der XXXX ein.)*

Dann folgt dein *apikey*. Trage hier die Zahlenfolge ein, die CallMeBot dir über WhatsApp geschickt hat.

Als letztes Feld trägst du unter *message* noch deine Nachricht ein, die an dein Handy geschickt werden soll.



Für eine kleine Debug-Anzeige (um Fehler auszuschließen) legst du nach dem **Whatsup**-Block noch einen **OLED Print**-Block. Hier trägst du unter *text* beispielsweise „gesendet“ ein.

Ein Delay von 250ms am Ende entprellt deine Button-Abfrage wieder, damit deine Schleife nur einmal reagiert.

Damit ist dein Programm fertig. Sobald du den Encoder-Button drückst, wird eine Nachricht an dein Handy geschickt. Wenn auf dem Bildschirm nicht „gesendet“ steht, dann prüfe bitte noch einmal genau deine **WLAN**- und **Whatsup**-Einstellungen im Programm.

Ein Hinweis noch: Manchmal kann so eine Nachricht einen Moment brauchen, bis sie auf deinem Handy erscheint. Das liegt daran, dass die API von vielen Menschen genutzt wird und dadurch manchmal etwas ausgelastet ist.

Prima – du solltest nun deine erste Nachricht von Makey:Lab empfangen haben!

So kannst du das nutzen

Jetzt, wo dein Makey:Lab übers Internet Nachrichten verschicken kann, kannst du viele deiner bisherigen Projekte erweitern.

Überlege: In welchen Situationen wäre es praktisch, wenn Makey:Lab dich informiert, auch wenn du nicht danebenstehst?

– *#1 Wetterfrosch*: schickt dir eine Nachricht, wenn sich der Luftdruck stark ändert.

– *#2 Lüftungsampel*: erinnert dich ans Lüften, wenn die die Luft schlecht wird.

– *#3 Makey:Lab schnuppert am Obstkorb*: meldet, wenn es (zu) reif riecht.

– *#4 Duschpetze*: warnt, wenn im Bad zu lange geduscht wird.

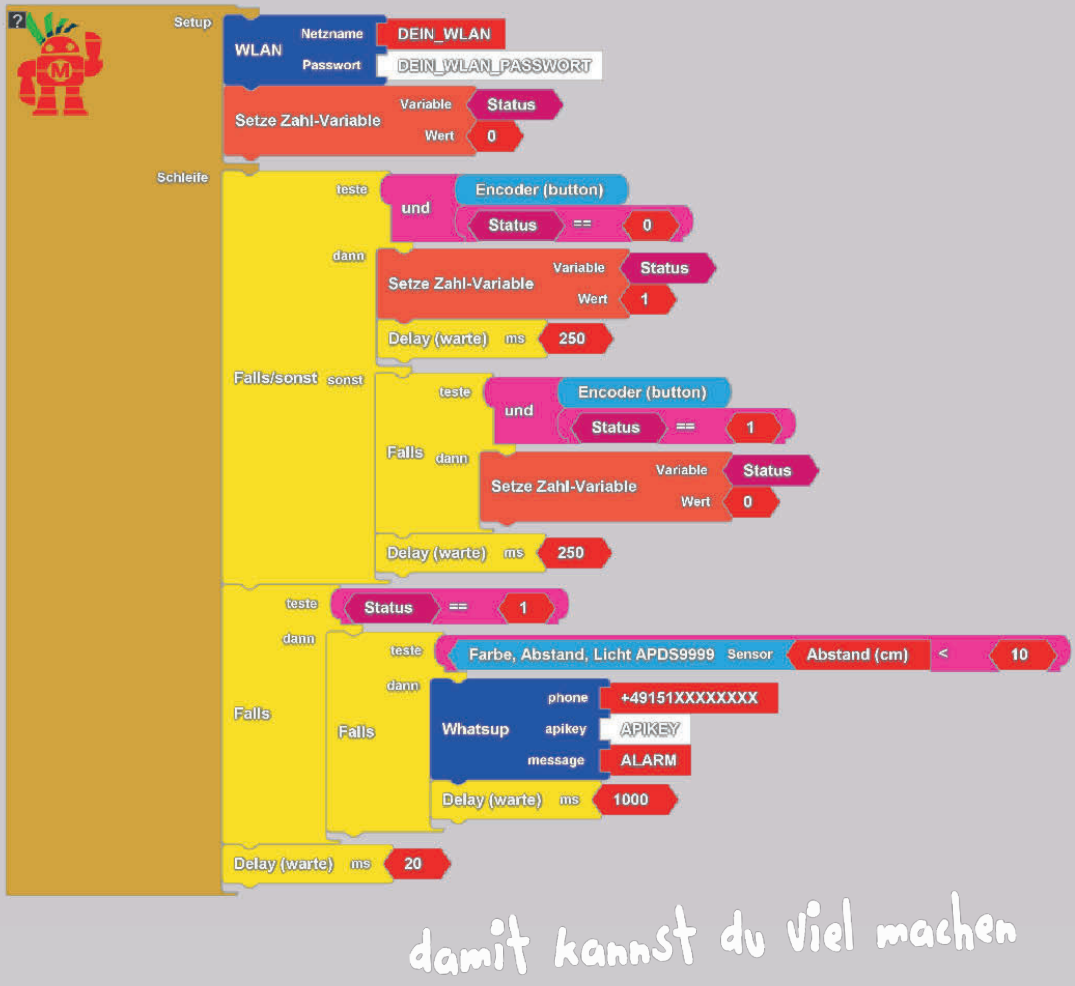
So lernst du: Fast jedes deiner Programme kann internetfähig werden – du musst nur wissen, was Makey:Lab dir mitteilen soll.

Ein Beispiel schauen wir uns jetzt gemeinsam an. Wenn du das Prinzip einmal verstanden hast, kannst du es ganz leicht auf deine anderen Programme übertragen.

Zimmer-Alarmanlage 2.0 – mit WhatsApp-Warnung

Es ist schon ziemlich praktisch, wenn dein Makey:Lab Alarm schlägt, sobald jemand dein Zimmer betritt.

Noch besser wäre es aber, wenn du sofort eine Nachricht auf dein Handy bekommst, oder? Genau das bauen wir jetzt!

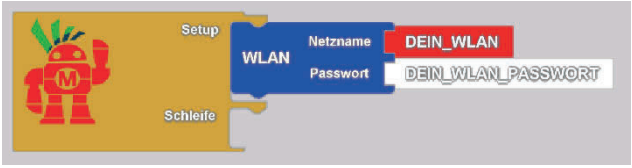


Für deine Zimmer-Alarmanlage 2.0 kombinierst du verschiedene Bausteine, die du schon kennst. Der Alarm bleibt diesmal still – kein Display, kein Buzzer, keine NeoPixel.

Damit du Makey:Lab trotzdem bequem aktivieren kannst, verwenden wir wieder den Encoder-Button.

Im Prinzip musst du nur den Teil mit dem akustischen Alarm durch den **Whatsup**-Block ersetzen. Dort trägst du deine Telefonnummer und den API-Key ein – wie zuvor – und schreibst als Nachricht zum Beispiel: „*Alarm!*“

Wichtig ist nur, dass du im Setup noch dein **WLAN**-Block einrichtest.



Nach dem Whatsup-Block folgt noch ein **Delay**, das dafür sorgt, dass innerhalb einer Sekunde nur **eine** Nachricht geschickt wird.

Bleibt der Eindringling länger vor Makey:Lab stehen, könnten natürlich mehrere Nachrichten kommen – aber eine Sekunde ist ein guter Richtwert.

Wie du siehst, ist das gar nicht kompliziert.

Du kannst ganz leicht eine Nachricht schicken lassen – und wenn du zum Beispiel einen Sensorwert weitergeben möchtest, trägst du im *message*-Feld des Whatsup-Blocks einfach deine entsprechende Variable ein.

So machst du aus jedem deiner Projekte ein echtes IoT-Gerät – eins, das dir schreibt, wenn etwas passiert.

E-Mail als stabile Alternative

WhatsApp ist praktisch, weil du sofort eine Nachricht bekommst.

Aber manchmal möchtest du lieber eine Nachricht verschicken, die unabhängig von Diensten wie CallMeBot funktioniert - zum Beispiel wenn du kein WhatsApp nutzt, der Dienst gerade nicht verfügbar ist – oder du deine Nachricht lieber direkt verschicken möchtest, ohne dass sie über einen zusätzlichen, fremden Server läuft.

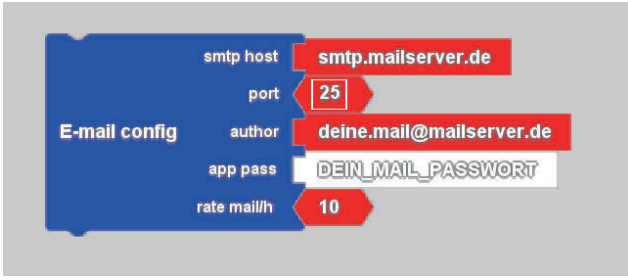
Genau dafür kannst du den **E-Mail send**-Block nutzen. Damit kann dein Makey:Lab direkt über das Internet eine Nachricht an eine E-Mail-Adresse senden – ganz ohne zusätzliche Dienste.

Im Prinzip funktioniert das fast genauso wie beim **Whatsup**-Block – nur dass die Nachricht diesmal nicht im Chat landet, sondern im Posteingang.

E-Mails sind zwar etwas altmodisch, aber äußerst zuverlässig. Und der Vorteil: Jeder Handy-Besitzer hat meist auch eine E-Mail-Adresse – selbst wenn kein Messenger funktioniert.

Wir werden nun ein Test-Programm erstellen, welches auf Knopfdruck eine E-Mail an eine bestimmte Adresse versendet.

Ins Setup fügst du dafür den **WLAN**-Block ein und richtest ihn ein. Danach folgt der **E-Mail config**-Block (den findest du unter *IoT: E-Mail / NTP*). Hier musst du die Einstellungen für dein Postfach hinterlegen.



Smtp host: Das ist die Serveradresse deines Anbieters, z.B. *mail.gmx.net* oder *smtp.web.de*.

port: Hier steht die Zahl 25. Sie zeigt an, über welchen „Datenkanal" die Nachricht verschickt wird. Viele Anbieter unterstützen diesen Standard-Port noch – bei manchen musst du eventuell einen anderen Port (z. B. 465 oder 587) verwenden.

author: Deine eigene E-Mail-Adresse.

app pass: Das Passwort für den Mailversand. Viele Anbieter verlangen dafür ein spezielles „App-Passwort".

rate mail/h: Gibt an, wie viele E-Mails pro Stunde gesendet werden dürfen – der Standardwert 10 ist ideal.

Hinweis: Einige Mail-Anbieter blockieren den direkten Versand über externe Geräte. Falls das Senden nicht funktioniert, liegt das fast immer an den

Sicherheitseinstellungen deines Postfachs. Aktiviere dort den Zugriff über App-Passwörter oder nutze einen Anbieter, der diese Funktion unterstützt (zum Beispiel *GMX* oder *Web.de*).

Nach diesen Einstellungen geht es nun in der Programmschleife weiter. Dort startest du mit einem **Falls**-Block. Als Auslöser setzt du unter *teste* den **Encoder (button)**. Als Aktion unter *dann* nutzt du den **E-mail send**-Block, der das eigentliche Senden übernimmt.

Unter *recipient* trägst du die Empfängeradresse ein. Beispielsweise könnte das sein: *deine.mail@mailserver.de*

Bei *subject* kannst du einen Betreff einfügen. Du könntest etwas schreiben wie:
Nachricht von Makey:Lab

Unter *message* steht deine eigentliche Textnachricht. So wie:
Hallo, dies ist eine Nachricht von Makey:Lab!

Nach dem **E-Mail**-Block setzt du ein **OLED print**-Block und nimmst als *text* eine Bestätigung: *gesendet*

Dann siehst du später direkt, ob das Senden geklappt hat.

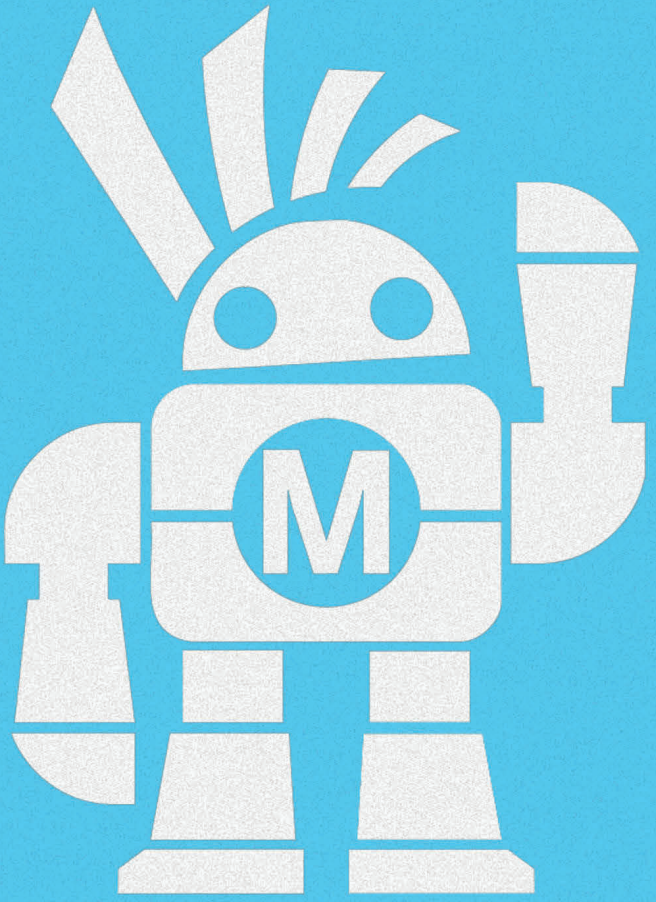
Abschließend setzt du ein **Delay** von 250 ms ein, um den Button zu entprellen. Damit ist dein Programm fertig. Du kannst nun statt WhatsApp-Nachrichten auch E-Mails versenden.

Du willst noch mehr wissen?

SCAN MICH

Hinter dem QR-Code findest du Einblicke in das Videolabor und Projekte zu Social Media am Umwelt-Campus Birkenfeld.

Mehr dazu im Studiengang **Medieninformatik (B.Sc.)**.



DANKE & BIS BALD!

Ende

Das war dein erster Blick ins Makey:Lab

Du bist am Ende des Playbooks angekommen.
Keine Sorge – es geht weiter!

Das Makey:Lab wächst ständig: mit neuen Projekten, neuen Ideen und neuen Playbooks.

In Zukunft wirst du noch mehr Möglichkeiten entdecken – vielleicht vernetzte Makey:Labs, größere IoT-Experimente oder sogar eigene Programme direkt im Code. Was daraus entsteht, liegt ganz bei dir – und bei der Community.

Bis dahin: Probier dich aus, experimentiere, verändere, bau um. Dein Makey:Lab ist dafür gemacht, mit dir zu lernen.

Danke, dass du dabei bist!

Bleib neugierig – und denk immer dran:
Du musst kein Profi sein, um Großes zu bauen.
Du brauchst nur eine Idee – und den Mut, einfach loszulegen.

Erzähl gern anderen vom Makey:Lab.
Denn gute Ideen werden besser, wenn man sie teilt.

ANHANG
BETRIEBSANLEITUNG

Betriebsanleitung – Makey:Lab

Dieses Dokument ist Bestandteil des Produkts. Bitte sorgfältig lesen und aufbewahren.

Sicherheitshinweise

- Betrieb ausschließlich mit **5 V Kleinspannung über USB** (mitgeliefertes Kabel).

- **Keine Netzspannung** anschließen – Gefahr durch elektrischen Schlag.

- Einsatz nur in **trockenen Innenräumen**. Bei Verwendung im Freien muss ein geeignetes, wettergeschütztes Gehäuse vorgesehen werden.

- Nicht geeignet für Kinder unter **14 Jahren**.

- Enthält **kleine Teile** – nicht für Kinder unter 3 Jahren (Verschluckungsgefahr).

- Verpackungsmaterialien (z. B. Plastiktüten) von Kindern fernhalten – **Erstickungsgefahr**.

- Umbauten und Änderungen sind nur zulässig, wenn sie in der Anleitung beschrieben sind. **Eigenständige Modifikationen erfolgen auf eigene Verantwortung.**

- Vor Feuchtigkeit, Hitze und starker mechanischer Belastung schützen.

- Das Gerät darf nicht in einer Umgebung mit brennbaren Gasen, Dämpfen oder Staub betrieben werden.

- Das Gerät ist nicht für Personen mit eingeschränkten physischen, sensorischen oder geistigen Fähigkeiten ohne Aufsicht oder entsprechende Unterweisung geeignet.

Bestimmungsgemäße Verwendung

- Makey:Lab ist eine **Elektronik- und Programmierplattform für Lern- und Experimentierzwecke.**

- Die Nutzung erfolgt im Rahmen von Ausbildung, Hobby und Forschung.

- Für Schäden, die durch unsachgemäße Nutzung entstehen, übernimmt der Hersteller keine Haftung. Die Gewährleistung erlischt bei eigenmächtigen Veränderungen, die nicht in der Anleitung vorgesehen sind.

Funkhinweise

- Enthält ein **ESP32-WROOM Funkmodul** (WLAN & Bluetooth).
- Frequenzband: **2,4 GHz ISM-Band (2400 – 2483,5 MHz)**
- Maximale Sendeleistung: **+20 dBm (100 mW EIRP)**

Nutzung in Flugzeugen, Krankenhäusern und in der Nähe empfindlicher medizinischer Geräte (z. B. Herzschrittmacher) ist nur eingeschränkt zulässig. Die jeweils geltenden Vorschriften sind zu beachten.

Technische Daten

- Betriebsspannung: 5 V (USB)
- Stromaufnahme @5V:

  - Leerlauf: typisch 80–90 mA
  - Normalbetrieb: typisch 80–250 mA
  - Volllast: bis 600 mA

- Funkstandards: WLAN 802.11 b/g/n, Bluetooth Classic & BLE
- Abmessungen: 86,5 × 108,5 × 26 mm
- Umgebung: 0 … 40 °C, nur Indoor-Betrieb oder im wettergeschützten Gehäuse

Lieferumfang: Makey:Lab Board, USB-Kabel, Playbook (Betriebsanleitung)

Entsorgung

Mit dem **WEEE-Symbol** gekennzeichnet, darf nicht im Hausmüll entsorgt werden. Abgabe bei kommunalen Sammelstellen.

Sofern **Batterien oder Akkus verwendet werden**, dürfen diese nicht im Hausmüll entsorgt werden, sondern sind bei Sammelstellen abzugeben.

EU-Konformitätserklärung

Makey:Lab erfüllt die grundlegenden Anforderungen der folgenden Richtlinien:

RED 2014/53/EU
EMV 2014/30/EU
RoHS 2011/65/EU
WEEE 2012/19/EU

Die vollständige EU-Konformitätserklärung ist abrufbar unter: ***https://heise.de/-10498312***

Herstellerangaben

Hersteller: **Maker Media GmbH**
Adresse: **Karl-Wiechert-Allee 10, 30625 Hannover**
Web: ***https://make-magazin.de/***

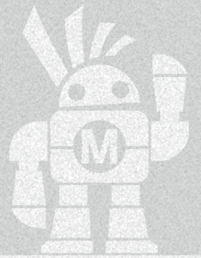
Symbolerklärung

CE-Kennzeichen: Konformität mit EU-Richtlinien

Durchgestrichene Mülltonne: Entsorgung über Sammelstellen (WEEE)

NOTIZEN

SCHREIB ES AUF



Scan mich

Das Playbook erklärt dir alles, was du brauchst, um sofort mit deinem Makey:Lab loszulegen. Scanne den QR-Code für weitere Infos, Software-Updates und den Zugang zur Makey:Lab-Seite.